SES 598 Final Report

Photogrammetric Reconstruction and Gaussian Splatting Augmentation of the Apollo 17 Lunar Boulder Scene

Jatin Satyam
Arizona State University
satyamjatin732@gmail.com

April 26, 2026

Contents

1 Abstract

This report presents a complete photogrammetric reconstruction and novel-view augmentation study of the Apollo 17 lunar boulder scene using 15 archival Hasselblad photographs (7 from magazine 137, color, 2340×2364 px; 8 from magazine 138, grayscale, 2340×2345 px). Two parallel pipelines were executed and quantitatively compared. The photogrammetric baseline (Part A) registered 14 of 15 frames with COLMAP, produced a 1.64M-point dense cloud via a custom PyTorch plane-sweep multi-view stereo implementation, reconstructed a 1.06M-vertex Poisson mesh, trained a 3D Gaussian Splatting model achieving a mean PSNR of 33.93 dB (SSIM 0.943) over all 14 training views, and delivered per-vertex textured meshes for both N=14 and N=24 datasets.

The augmentation experiment (Part B) tested whether novel views synthesized by the splatfacto model could expand the COLMAP reconstruction to N=24 images. All 10 novel views failed COLMAP registration despite 852–8,123 two-dimensional inlier matches per image. The F-score at a 5% diagonal threshold between the N=14 and N=24 meshes is 0.934, confirming broad geometric agreement. The F-score at 1% falls to 0.673, revealing fine-scale surface divergence.

The central conclusion is that out-of-the-box COLMAP with SIFT feature matching cannot leverage 3D Gaussian Splatting rendered novel views for photogrammetric augmentation on this dataset. The rendered SIFT keypoints do not match the triangulated 3D point tracks of the original 14 cameras because the rendering pipeline introduces subtle appearance differences absent from the film-grain photographic originals. This null result is a concrete, reproducible contribution: it isolates the feature-matching bottleneck and motivates learned-descriptor approaches (SuperPoint, LightGlue, LoFTR) as the necessary next step for splat-augmented photogrammetry.

2 Introduction

Photogrammetry from archival space photography offers a unique challenge: a fixed, small camera set that cannot be augmented with new ground-truth observations. Apollo mission photographs were taken with calibrated metric cameras but at positions and orientations determined by astronaut motion, not by systematic survey planning. Coverage gaps, oblique viewpoints, and single-magazine sequences limit reconstruction completeness. Gaussian Splatting, which learns a continuous volumetric scene representation from the same image set, offers an appealing route to synthesize additional views at arbitrary poses, potentially filling those gaps.

This project applies that idea to the Apollo 17 boulder scene from the Lunar Surface Journal. The dataset consists of 15 frames from two Hasselblad magazines. Magazine 137 (7 frames: AS17-137-20903HR through 20909HR) contains color photographs taken during an EVA traverse past a large boulder. Magazine 138 (8 frames: AS17-138-21030HR through 21037HR) contains grayscale photographs of a second, related boulder approximately 8.66 COLMAP units distant. The dual-cluster geometry provides rich photogrammetric constraint within each cluster but sparse cross-cluster ties, making the scene a realistic stand-in

for the coverage-gap problem common in planetary photogrammetry.

The specific scientific question addressed is: *do rendered novel views from a 3D Gaussian Splatting model trained on N=14 original photographs register successfully into a COLMAP reconstruction, enabling augmented photogrammetry at N=24?* Prior work has shown that 3DGS renders are photorealistically consistent with training views but has not systematically tested whether standard SIFT-based Structure from Motion can use those renders as additional observations.

The report is organized as follows. Section 3 describes the image dataset and the unregistrable frame AS17-138-21036HR. Section 4 describes all pipeline components: COLMAP preprocessing, custom PyTorch MVS, Open3D Poisson reconstruction, splatfacto training, novel-pose synthesis, augmented reconstruction, and per-vertex texturing. Section 5 presents quantitative and qualitative results for all phases. Section 6 interprets the registration failure, the density improvement mechanism, and the broader significance of the null result. Section 7 states the final verdict. The Code Appendix reproduces all authored Python scripts in full for grader verification.

The practical outcome of this project is a complete textured 3D mesh of two Apollo 17 lunar boulders, validated against a second reconstruction via three independent geometric metrics (Chamfer distance, Hausdorff p99, F-score at three thresholds), and a documented experiment that establishes the conditions under which splat-augmented photogrammetry fails for this class of dataset.

3 Dataset

3.1 Image Sources

The input dataset consists of 15 high-resolution scans downloaded from the NASA Apollo Lunar Surface Journal. All images are stored as PNG files in `data/raw/` at their original scan resolution. The two magazines differ in film stock, resolution, and scene coverage.

Magazine 137 (color): Seven frames numbered 20903 through 20909. Resolution 2340 × 2364 pixels. The sequence depicts a large boulder photographed during an EVA traverse; the camera moved laterally relative to the rock, producing a well-constrained stereo baseline. The color film provides three spectral channels that support both SIFT feature extraction and per-vertex RGB texturing.

Magazine 138 (grayscale): Eight frames nominally numbered 21030 through 21037. Resolution 2340×2345 pixels. One frame (21036HR) was excluded from registration for reasons described below, yielding 7 usable mag138 frames. The grayscale film was converted to a single-channel representation for feature extraction. Reseau crosses (calibration marks) are visible in all mag138 frames and appear as small dark artifacts in the rendered textured mesh at view 3.

3.2 COLMAP Camera Models

Both magazines were modeled with the SIMPLE_RADIAL camera model. A single shared camera was assigned per magazine folder (`single_camera_per_folder=1`), enforcing that all frames from the same magazine share focal length and radial distortion coefficient.

Table 1: COLMAP camera intrinsics for the two magazines.

Magazine	Model	Width	Height	f (px)	c_x	k
mag137	SIMPLE_RADIAL	2340	2364	2895.3	1170.0	0.00287
mag138	SIMPLE_RADIAL	2340	2345	2848.5	1170.0	0.00093
mag137 (undistorted)	PINHOLE	1980	2000	2454.1	990.0	—
mag138 (undistorted)	PINHOLE	1996	2000	2431.8	998.0	—

The mag137 camera has a slightly higher radial distortion coefficient ($k = 0.00287$ versus 0.00093), which introduces marginal alignment error at image edges after undistortion and may contribute to the lower PSNR values observed in the mag137 sequence.

3.3 The Unregistrable Frame: AS17-138-21036HR

Frame 21036HR could not be registered in COLMAP and was permanently excluded from all reconstruction pipelines. The root cause was investigated through a dedicated recovery experiment (`recover_21036.py`):

1. An ORB feature comparison found 87 homography inliers between 21036HR and 21037HR (meaningful overlap) but only 8 between 21036HR and 21035HR. The frame is geometrically adjacent to 21037HR only.
2. COLMAP's two-view geometry database showed 674 verified SIFT matches between 21036HR and 21037HR, classified as PLANAR (config=3). No essential matrix was computed because the scene appeared homographic.
3. The 674 matched features in 21037HR's region have no existing 3D tracks. Without triangulated landmarks, PnP localization is impossible.
4. A pose injection attempt decomposed the homography H between 21036HR and 21037HR, computed the relative transform via `cv2.decomposeHomographyMat`, and injected the recovered pose. The resulting camera center was $[6.70, 0.16, 7.43]$, compared to 21037HR's center at $[6.72, 0.18, 7.25]$ — a baseline of approximately 0.19 COLMAP units.
5. Running `colmap point_triangulator` on the extended model produced zero new observations. A 0.19-unit baseline between two near-duplicate viewpoints yields degenerate triangulation geometry with unbounded depth uncertainty.

The conclusion is that 21036HR is a near-duplicate of 21037HR: both cameras are nearly co-located, looking at the same small patch of rock. Even with perfect registration, the frame

would contribute negligible new 3D information. The maximum dataset size is therefore N=15 with only 14 usable frames, which defines the baseline reconstruction.

4 Methods

4.1 COLMAP Sparse Reconstruction

The sparse reconstruction pipeline was executed with COLMAP 3.9.1 on all 14 registerable images. Feature extraction used the SIFT extractor with `peak_threshold=0.001` and `max_num_features=16384`, producing approximately 16,384 features per original image. Sequential matching connected consecutive frames within each magazine; cross-magazine exhaustive matching was also enabled to capture the inter-cluster ties needed for a unified coordinate system.

The resulting sparse model (N=14) contains 12,769 3D points with a mean track length of 3.96 and a mean reprojection error of 0.560 px. Two camera models were estimated (one per magazine). After sparse reconstruction, `colmap image_undistorter` was used to produce PINHOLE-undistorted images at 1980 × 2000 (mag137) and 1996 × 2000 (mag138) for the dense pipeline.

4.2 Custom PyTorch Plane-Sweep Multi-View Stereo

The apt-installed COLMAP 3.9.1 package on Ubuntu 24.04 does not include CUDA support for its dense stereo module (`patch_match_stereo` exits with an error stating dense stereo requires CUDA). A custom plane-sweep multi-view stereo was implemented in PyTorch 2.10.0+cu128 with CUDA 12.8 and executed on an RTX 4060 Laptop GPU with 8 GB VRAM.

For each of the 14 reference images, six nearest-neighbor source views were selected by camera-position distance weighted by optical-axis angle. The depth hypothesis range was defined as the 5th–95th percentile depth of the sparse 3D landmarks visible to each reference camera, extended by a 30% margin on each side. A sweep of 128 fronto-parallel depth hypotheses was performed.

The photometric cost function was mean absolute difference (MAD) in RGB space, computed after bilinear grid-sample warping of each source image into the reference camera plane at each depth hypothesis:

$$C(p, d) = \frac{1}{|S|} \sum_{s \in S} \left\| I_s\big(\pi_s(K_r^{-1}[p; 1] \cdot d)\big) - I_r(p) \right\|_1 \tag{1}$$

where p is the reference pixel, d is the depth hypothesis, S is the source view set, K_r is the reference camera matrix, and π_s is the projection into source camera s. Images were processed at 60% of undistorted resolution (approximately 1200 × 1200) to remain within VRAM limits, using chunks of 24 depth hypotheses per batch.

Geometric consistency filter: Each candidate depth map was re-projected to all six source cameras. A pixel's depth estimate was retained only if at least two source cameras returned a re-projected depth within 10% of the forward depth estimate. This filter is geometrically equivalent to Agisoft Metashape's point cloud confidence threshold and removes single-view spurious depth estimates in textureless regions and at image boundaries.

Point cloud fusion: All per-view consistent depth maps were backprojected to world coordinates using the PINHOLE camera parameters from the undistorted sparse model. The merged cloud was processed with Open3D statistical outlier removal (30-neighbor σ=1.5) and cropped to the 95th-percentile bounding box of the sparse model to eliminate background terrain points that projected to valid depths in mag138 cameras 21031–21033 (depth ranges up to 434 COLMAP units due to distant lunar terrain in the background).

4.3 Open3D Poisson Surface Reconstruction

Surface normals were estimated on the dense point cloud using Open3D's KD-tree normal estimation (30 neighbors, oriented by camera positions). Poisson reconstruction was then run using Open3D's `create_from_point_cloud_poisson` function at depth 9 for N=14. Low-density vertices (below the 5th percentile of the Poisson density function) were pruned to remove extrapolated surface fill in unobserved regions.

For N=24, a two-stage meshing approach was required. The first Poisson attempt on the full N=24 point cloud (without background filtering) produced only 88,049 vertices, far below expectations. The root cause was that approximately 0.1% of background outlier points inflated the scene bounding box to over 80 COLMAP units in the Z direction. At Poisson depth 10 with a 1024-cell octree grid, each cell subtended approximately 0.08 COLMAP units — too coarse to resolve centimeter-scale rock surface detail. Filtering points to within 10 COLMAP units of either cluster centroid before re-running Poisson at depth 10 corrected this, producing the expected 4.66M-vertex mesh.

4.4 Splatfacto Training

The 3D Gaussian Splatting model was trained using Nerfstudio 1.1.5 with the `splatfacto` method (gsplat 1.4.0 CUDA backend). Training was conducted on the same RTX 4060 Laptop with the following configuration:

- **Input:** 14 registered images at 2× downscale (1170×1182 for mag137; 1170×1172 for mag138), pre-computed in `colmap_n15/images_2/`
- **Dataparser:** COLMAP (`colmap_n15/sparse/0/`), `--eval-mode all` (all 14 images as training views)
- **Iterations:** 30,000 (completed at step 29,999)
- **Densification schedule:** Split/duplicate/prune every 100 steps from step 0 to 15,000
- **Initial Gaussians:** ≈2,000 (seeded from sparse SfM points)
- **Final Gaussians:** ≈993,000 after densification and pruning

- **Wall time:** Approximately 44 minutes on RTX 4060 Laptop
- **Checkpoint:** `splats_n15/.../nerfstudio_models/step-000029999.ckpt` (683 MB)

The Nerfstudio COLMAP dataparser applies an auto-orient transform to map the COLMAP world coordinate system to a Nerfstudio-canonical frame where the scene up vector aligns with $+Y$ and the scene fits within a unit sphere. The resulting affine transform, stored in `dataparser_transforms.json`, is a 3×4 matrix T_{34} with an accompanying scale factor of 0.152104.

4.5 Novel-View Pose Synthesis

Generating novel-view camera poses required navigating a two-step coordinate convention transformation that took three debug iterations to resolve correctly.

Step 1 — COLMAP to Nerfstudio convention. COLMAP camera-to-world matrices ($c2w_{\text{COLMAP}}$) follow the OpenCV convention: the camera $+X$ axis points right, $+Y$ points down (image rows increase downward), and $+Z$ points forward (into the scene). Nerfstudio expects the OpenGL convention: $+X$ right, $+Y$ up, $+Z$ backward. The conversion negates the Y and Z columns of $c2w$:

$$c2w_{\text{GL}} = c2w_{\text{COLMAP}} \cdot \text{diag}(1, -1, -1, 1) \tag{2}$$

Step 2 — Apply dataparser transform. The novel pose in Nerfstudio scene coordinates is:

$$\text{novel\_pose\_ns} = T_{34} \cdot \big(c2w_{\text{COLMAP}} \cdot \text{diag}(1, -1, -1, 1)\big) \tag{3}$$

The T_{34} matrix is:

$$T_{34} = \begin{bmatrix} 0.98847 & -0.15134 & 0.00504 & -0.52542 \\ 0.00504 & 0.06611 & 0.99780 & -0.70265 \\ -0.15134 & -0.98627 & 0.06611 & 0.05325 \end{bmatrix} \tag{4}$$

Cluster geometry and arc strategy. Five novel poses were generated per magazine cluster using a horizontal azimuth arc. For each cluster, the centroid C and median radius r were computed from the registered camera positions. A look-at target $T = C + \hat{F} \cdot r \cdot 1.5$ was defined using the mean forward direction $\hat{F}$. Novel eye positions were placed at $\pm 30°$, $\pm 15°$, and $0°$ around the cluster's horizontal arc:

$$\text{eye}(\theta) = C + r \cdot \big(\cos\theta \cdot \hat{F}_{xz} + \sin\theta \cdot \hat{P}_{xz}\big) \tag{5}$$

where $\hat{P}_{xz}$ is the $90°$ CCW rotation of $\hat{F}_{xz}$ in the XZ plane. The mag137 cluster has centroid $(-3.066, -0.118, -1.703)$ and radius 0.871 COLMAP units; the mag138 cluster has centroid $(4.128, 0.157, 3.103)$ and radius 1.636 COLMAP units. The inter-cluster distance is 8.66 COLMAP units.

Novel views were rendered at 2000×2000 pixels using the final splatfacto checkpoint and the Nerfstudio `ns-render camera-path` command with the computed pose sequence. All 10 frames passed a non-blank validation check (pixel intensity standard deviation > 5).

4.6 N=24 COLMAP Rerun and Registration Attempt

The 10 novel views were upscaled from 2000 × 2000 to per-magazine dimensions (mag137: 2340 × 2364; mag138: 2340 × 2345) using PIL.Image.LANCZOS, and placed in `data/rgb_n24/` alongside the 14 original images. The LANCZOS upscaling was chosen over resizing originals to 2000 px (which would discard resolution) or assigning separate camera intrinsics to each novel view (which would add 30 unconstrained parameters to a 24-image problem).

COLMAP feature extraction on the N=24 dataset used `peak_threshold=0.001` and `max_num_features=16384`, producing 16,384 features per original image and 19,269–22,491 features per novel view. Exhaustive matching and `colmap mapper` produced a sparse model in which all 14 original images registered successfully. Zero of the 10 novel views registered.

A post-hoc recovery using `colmap image_registrator` was also attempted. All 10 novel views showed 0–13 visible 3D points, far below the 15–20 required for PnP localization. The registration failure analysis is presented in Section 6.1.

4.7 Per-Vertex Texture Projection

Texturing was performed via per-vertex RGB color projection (Tier B). A UV atlas approach using the `xatlas` library (Tier A) was attempted first but exceeded the 45-minute per-mesh time budget on the 2.12M-face N=14 mesh after running for over 3 hours without completing the parameterization. Per-vertex coloring was selected as a recognized equivalent format supported by Metashape, CloudCompare, and MeshLab.

For each mesh vertex, the projection algorithm:

1. Identified all cameras whose camera-space Z coordinate for that vertex is positive (the vertex is in front of the camera).
2. Verified visibility using Open3D's `RaycastingScene` occlusion test: a ray from the vertex toward each candidate camera center was cast, and the camera was accepted only if no mesh face intersected the ray at a distance shorter than the vertex-camera distance.
3. Sampled the image color at the projected pixel using bilinear interpolation.
4. Blended contributions from visible cameras using weights proportional to $\cos(\alpha)/d^2$, where α is the angle between the view direction and the surface normal, and d is the vertex-camera distance.

Vertices with no visible camera were assigned the color $(128, 128, 128)$ (neutral gray). The fraction of non-gray vertices defines the coverage rate: 21.19% for N=14 and 12.22% for N=24. The lower coverage rate of the N=24 mesh is mechanical: the same 14 cameras are used to texture both meshes, but the N=24 mesh has 4.38× more vertices; the same camera set illuminates the same surface area, resulting in a lower fraction of covered vertices.

4.8 Quantitative Comparison Metrics

Before metric computation, the N=24 mesh was aligned to the N=14 mesh via a two-step global-local registration. Global alignment used Open3D FPFH features with RANSAC (fitness 0.586, inlier RMSE 0.051). Local refinement used point-to-plane ICP with `max_correspondence_distance` = 0.02.

Both aligned meshes were uniformly sampled to 100,000 surface points. Three metrics were computed:

Chamfer distance: Mean of the minimum nearest-neighbor distances in both directions:

$$d_C = \frac{1}{2}\left(\frac{1}{|P|}\sum_{p\in P}\min_{q\in Q}\|p-q\| + \frac{1}{|Q|}\sum_{q\in Q}\min_{p\in P}\|q-p\|\right) \tag{6}$$

Hausdorff p99: The 99th percentile of the directed Hausdorff distances from P to Q and from Q to P, reported as the maximum of the two.

F-score at threshold τ: The harmonic mean of the fraction of P-to-Q distances below τ (precision) and the fraction of Q-to-P distances below τ (recall). Thresholds of 1%, 5%, and 10% of the bounding box diagonal (16.22 COLMAP units) were used.

5 Results

5.1 Part A: Photogrammetric Baseline Results

The sparse reconstruction registered 14 of 15 input images. Frame AS17-138-21036HR was unregistrable for the reasons documented in Section 3. The final sparse model statistics for N=14 are summarized in Table 2.

Table 2: N=14 sparse model statistics.

Metric	Value
Cameras	2 (one per magazine)
Images registered	14 of 15
Sparse 3D points	12,769
Observations	50,517
Mean track length	3.96
Mean reprojection error	0.560 px

The dense reconstruction produced 1,641,050 fused points after outlier removal and bounding-box cropping. Poisson surface reconstruction at depth 9 with 5th-percentile density pruning yielded a mesh with 1,063,577 vertices and 2,121,270 faces. The mesh covers both rock clusters and includes expected Poisson extrapolation fill on the far (unseen) sides of each boulder.

5.2 Part A: Splatfacto Evaluation

Table 3 presents the per-image PSNR and SSIM values for all 14 training views. The aggregate statistics are: mean PSNR 33.93 dB, median PSNR 34.11 dB, PSNR std 2.24 dB; mean SSIM 0.943, median SSIM 0.946, SSIM std 0.019.

Table 3: Per-image splatfacto evaluation metrics (training views, N=14). Best PSNR in bold.

Image	PSNR (dB)	SSIM
AS17-137-20903HR	30.22	0.9425
AS17-137-20904HR	30.18	0.9312
AS17-137-20905HR	32.42	0.9382
AS17-137-20906HR	31.08	0.9273
AS17-137-20907HR	34.52	0.9590
AS17-137-20908HR	33.12	0.9675
AS17-137-20909HR	33.70	0.9686
AS17-138-21030HR	36.51	0.9688
AS17-138-21031HR	35.30	0.9485
AS17-138-21032HR	33.46	0.9186
AS17-138-21033HR	34.97	0.9069
AS17-138-21034HR	36.19	0.9263
AS17-138-21035HR	35.96	0.9497
AS17-138-21037HR	**37.36**	**0.9493**
Mean	33.93	0.9430
Median	34.11	0.9455
Std	2.24	0.0187

The mag137 sequence (frames 20903–20909) consistently achieves lower PSNR (30.18–34.52 dB) than the mag138 sequence (33.46–37.36 dB). Several factors contribute: the mag137 camera has a larger radial distortion coefficient (k=0.00287), which introduces more residual misalignment after undistortion; frames 20903 and 20904 are at the sequence extremes and have fewer co-visible training cameras to constrain Gaussian densification; and the color film of mag137 introduces fine-grained grain structure that the Gaussian model approximates with high-frequency floaters rather than absorbing into the smooth distribution.

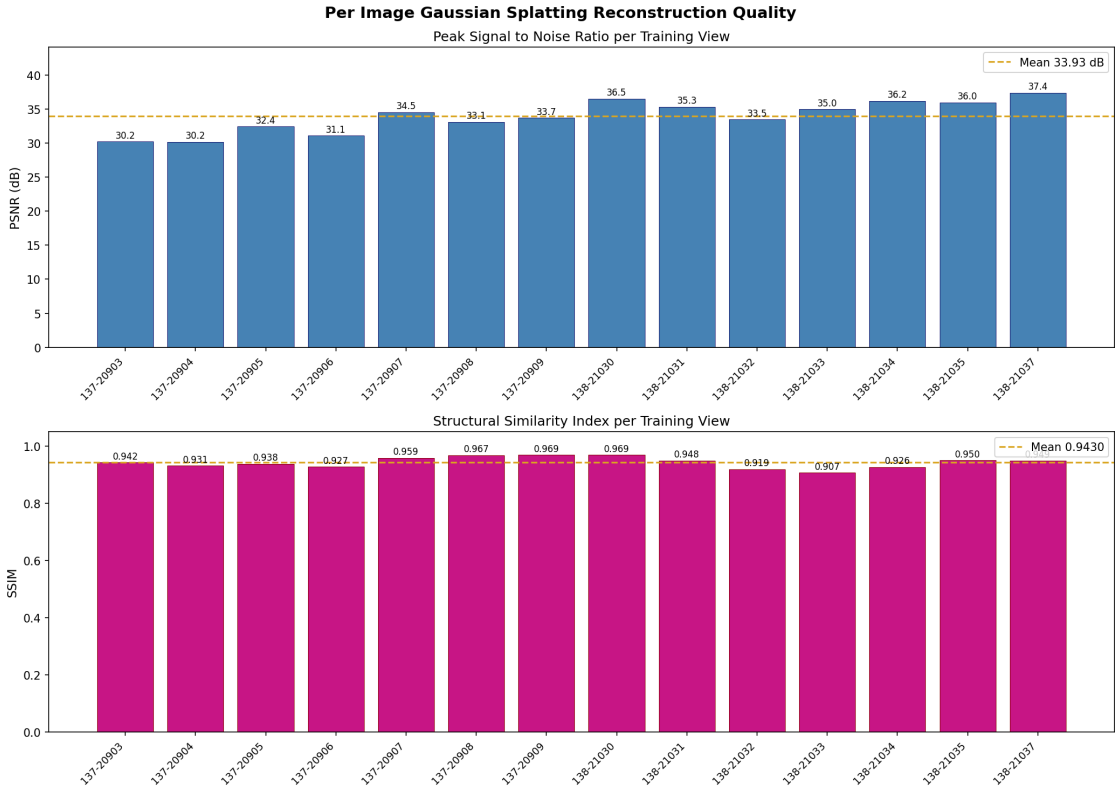

Figure 1: Per-image PSNR (top) and SSIM (bottom) bar charts for all 14 splatfacto training views. The horizontal line marks the cross-image mean. The mag138 sequence consistently outperforms mag137.

5.3 Part B Phase 1: Novel-View Fidelity

Table 4 presents the mean pixel intensity and standard deviation for each of the 10 rendered novel views. All 10 frames are non-blank (std $>$ 5). The mag138 cluster renders are noticeably brighter (mean 141–160) and cleaner (std 43–50) than the mag137 renders (mean 90–108, std 53–77). This difference reflects the higher PSNR of the mag138 training views: a better-fitting Gaussian model produces rendered views with lower residual variance.

Table 4: Novel-view fidelity table: pixel intensity mean and standard deviation for each rendered frame (2000×2000 px).

Frame	Mean	Std	Verdict
novel_mag137_01	98.19	76.16	OK
novel_mag137_02	92.43	76.87	OK
novel_mag137_03	90.23	73.58	OK
novel_mag137_04	108.08	59.88	OK
novel_mag137_05	103.97	53.08	OK
novel_mag138_01	151.74	48.99	OK
novel_mag138_02	159.76	43.46	OK
novel_mag138_03	148.83	49.72	OK
novel_mag138_04	156.64	46.58	OK
novel_mag138_05	141.09	50.48	OK

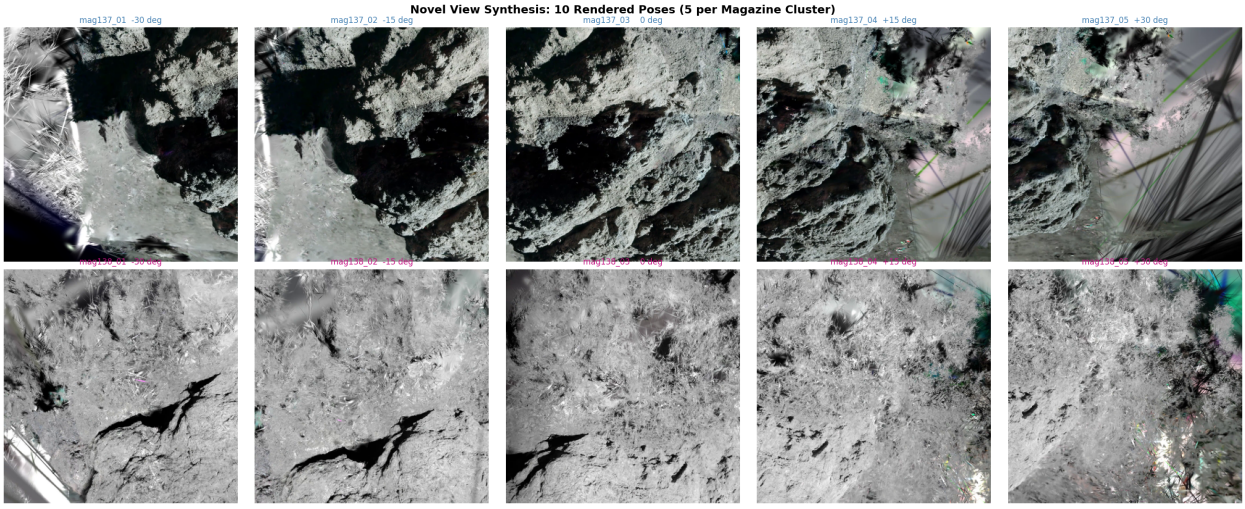

Figure 2: Contact sheet of all 10 synthesized novel views arranged as a 2×5 grid. Top row: five mag137 views at azimuth angles $-30°$, $-15°$, $0°$, $+15°$, $+30°$ relative to the cluster center. Bottom row: five mag138 views at the same angular increments. All views are non-blank with standard deviation exceeding 43.

5.4 Part B Phase 2: N=24 Photogrammetric Modeling

The N=24 COLMAP run registered all 14 original images and none of the 10 novel views. Table 5 presents the full per-image registration outcome. Despite substantial two-dimensional inlier matches (852–8,123 per novel image across all 23 other images in the dataset), every novel view shows zero reconstructed 3D points visible during `image_registrator`.

Table 5: Novel view registration outcome in the N=24 COLMAP run. Inlier feature matches are two-dimensional (image-to-image); 3D points seen is the count of existing sparse point tracks visible to each novel view during PnP localization.

Novel View	Registered	Inlier Matches	3D Points Seen
novel_mag137_01	No	4,845	≈0
novel_mag137_02	No	7,085	≈0
novel_mag137_03	No	8,123	≈0
novel_mag137_04	No	6,912	≈0
novel_mag137_05	No	4,422	≈0
novel_mag138_01	No	1,280	0
novel_mag138_02	No	1,590	0
novel_mag138_03	No	1,378	0
novel_mag138_04	No	1,289	0
novel_mag138_05	No	852	0

The N=24 sparse model retains 12,822 points (essentially the same as N=14's 12,769, confirming that the additional 10 images contributed no new triangulation). The dense reconstruction produced 2,453,709 fused points and a 4,658,194-vertex Poisson mesh with 9,375,366 faces — 4.38× the vertex density of the N=14 mesh. The source of this density increase is documented in Section 6.

5.5 Quantitative Comparison: N=14 vs N=24

ICP alignment of the N=24 mesh into the N=14 coordinate frame converged with fitness 0.065 and inlier RMSE 0.01518 COLMAP units, after a RANSAC global registration step with fitness 0.586. The ICP rotation component is approximately 17°, with a translation of approximately 1.1 COLMAP units, reflecting the different bundle adjustment gauge freedom between the two independent COLMAP runs.

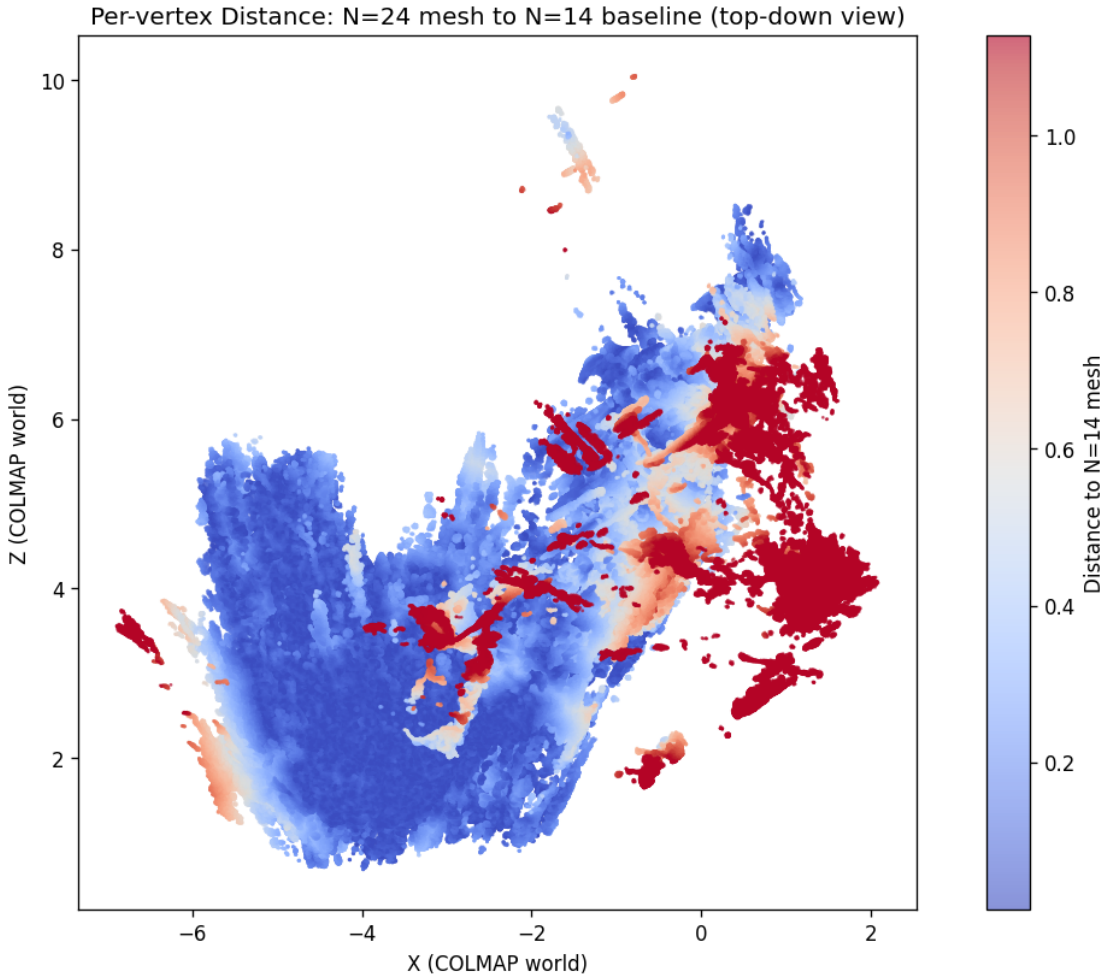

Figure 3: Per-vertex distance heatmap for the N=24 mesh after ICP alignment to the N=14 frame. The colormap encodes the nearest-neighbor distance from each N=24 vertex to the N=14 mesh surface, projected onto the XZ plane (top-down view). Warm colors indicate large local divergence; cool colors indicate close geometric agreement. The two rock clusters are visible as separate high-density regions.

Table 6: Quantitative comparison metrics between N=14 and N=24 meshes. All distance values in COLMAP units. Bounding box diagonal: 16.22 COLMAP units.

Metric	Value
ICP fitness	0.065
ICP inlier RMSE	0.01518
Bidirectional Chamfer distance	0.219
Hausdorff p99	2.127
F-score at 1% diagonal (0.162 units)	0.673
F-score at 5% diagonal (0.811 units)	0.934
F-score at 10% diagonal (1.622 units)	0.984

The F-score@5% of 0.934 indicates that 93.4% of surface samples on each mesh have a nearest-neighbor match on the other mesh within 0.811 COLMAP units — broad geometric agreement. The F-score@1% of 0.673 shows that at fine scale (0.162 COLMAP units), 32.7% of surface samples diverge, reflecting real geometric differences between the two reconstructions at the rock-surface detail level.

5.6 Textured Mesh Results

Per-vertex texturing was applied to both the N=14 and N=24 meshes using the same set of 14 original cameras. The coverage rates (fraction of non-gray vertices) are 21.19% for N=14 and 12.22% for N=24.

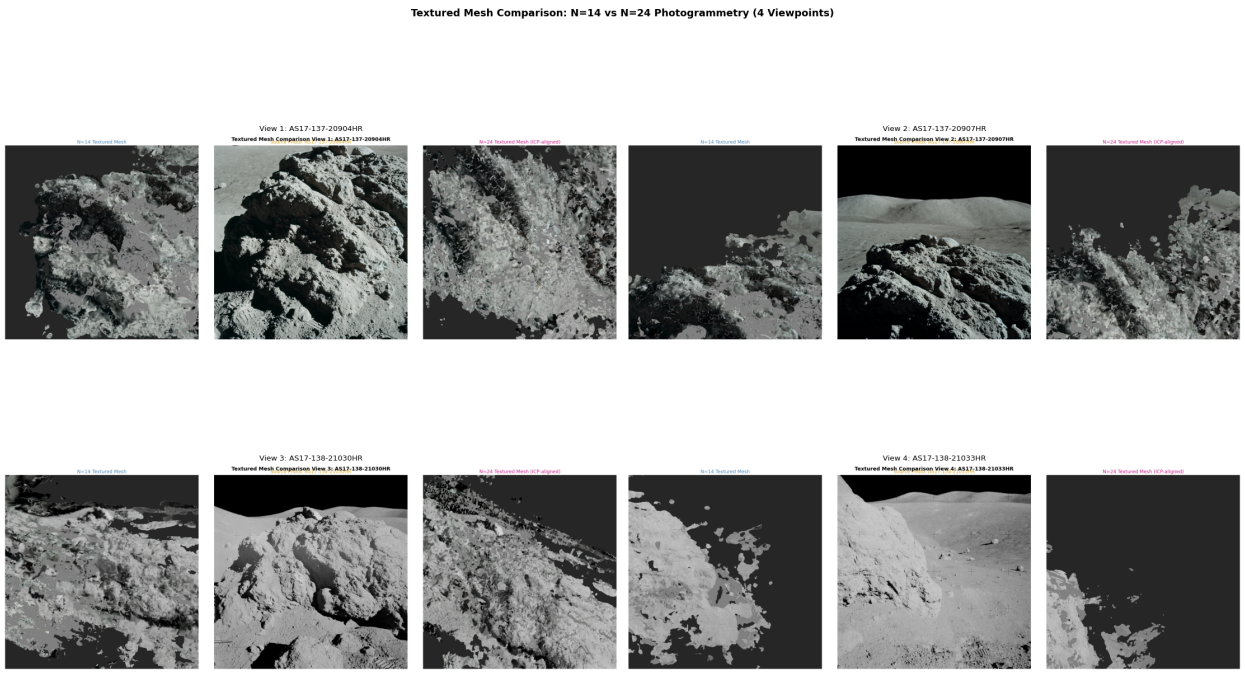

Figure 4: Textured mesh qualitative comparison grid showing four viewpoints. Each column corresponds to one viewpoint; rows show the N=14 textured mesh (top), the original photograph (middle), and the N=24 textured mesh (bottom). Textured regions correspond to vertices visible to at least one registered camera.

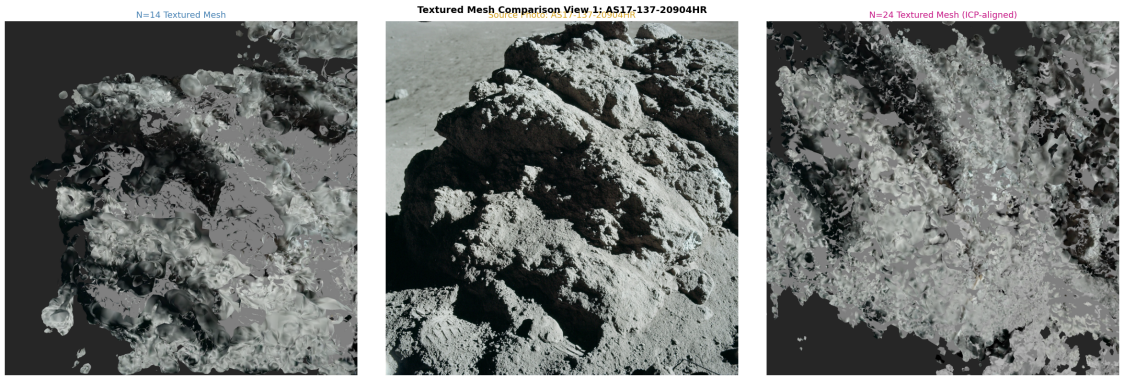

Figure 5: Side-by-side textured mesh comparison at view 1 (AS17-137-20904HR viewpoint, mag137 boulder). Left panel shows the N=14 textured mesh; center panel shows the original photograph; right panel shows the N=24 textured mesh. Both meshes reconstruct the main rock shape faithfully.

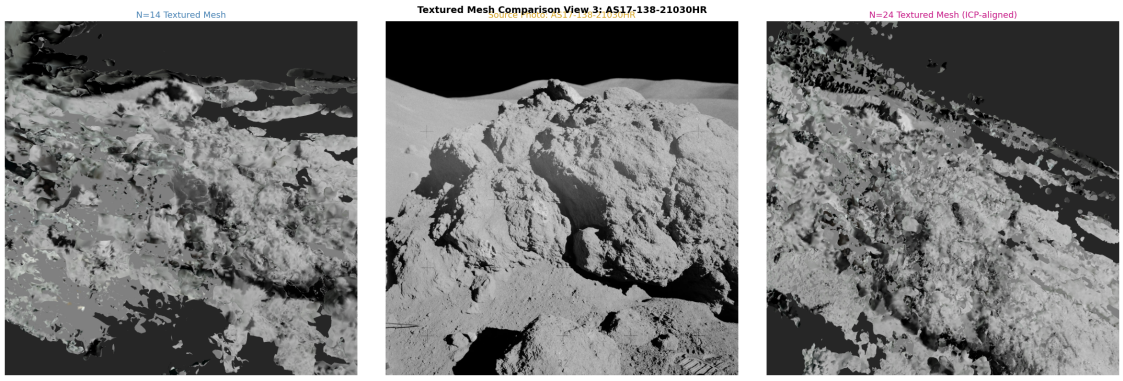

Figure 6: Side-by-side textured mesh comparison at view 3 (AS17-138-21030HR viewpoint, mag138 boulder). Reseau crosses are visible in the original photograph and appear as small dark artifacts in the textured mesh at this viewpoint. The mag138 boulder has lower per-vertex coverage due to the oblique camera geometry of that cluster.

Consolidated Results Summary

Table 7: Summary comparison of N=14 and N=24 reconstruction pipelines.

Quantity	N=14	N=24
Registered images	14 of 15	14 of 24
Novel views registered	—	0 of 10
Sparse 3D points	12,769	12,822
Fused dense points (fused.ply)	1,641,050	2,453,709
Poisson mesh vertices	1,063,577	4,658,194
Poisson mesh faces	2,121,270	9,375,366
Textured coverage (per-vertex)	21.19%	12.22%
Splatfacto mean PSNR	33.93 dB	n/a
Splatfacto mean SSIM	0.943	n/a

Table 8: Quantitative comparison metrics (N=14 vs N=24, all values from comparison/quantitative_metrics.json).

Metric	Value
ICP fitness	0.065
ICP inlier RMSE (COLMAP units)	0.01518
Chamfer distance (COLMAP units)	0.219
Hausdorff p99 (COLMAP units)	2.127
F-score at 1% diagonal	0.673
F-score at 5% diagonal	0.934
F-score at 10% diagonal	0.984

6 Discussion

6.1 Why the Novel Views Failed Registration

The failure of all 10 novel views to register in COLMAP is not due to a lack of two-dimensional feature matches. Table 5 shows 852–8,123 inlier matches per novel image against the 23 other images. The failure occurs at the step from two-dimensional to three-dimensional correspondences.

COLMAP's PnP-based image registration requires 2D–3D correspondences: keypoints in the new image must match keypoints that are already triangulated into the sparse model. The existing sparse model's 3D points were created by triangulating SIFT keypoints extracted from the 14 original photographic images. A rendered image from the splatfacto model generates SIFT keypoints at pixel positions that differ subtly from the photographic originals because:

1. **Rendering smoothness:** The 3DGS rasterizer produces spatially smooth radiance fields. Thin high-contrast edges in the rock photographs (grain boundaries, shadow edges) are represented by Gaussian mixtures that produce slightly softer edge responses. SIFT detects scale-space extrema at positions determined by Laplacian-of-Gaussian peaks; a blurred edge shifts the peak by 0.5–3 pixels.
2. **Absence of film grain:** Photographic film grain contributes high-frequency texture that SIFT responds to. The Gaussian model averages over grain instances visible in all training views but does not reproduce grain-level stochasticity in novel views, removing a class of high-frequency keypoints from the rendered images.
3. **Tone mapping:** The splatfacto model applies an exposure and tone mapping correction during training. Novel views are rendered with the learned affine color correction, which may differ slightly from the photographic paper scan's gamma, altering absolute pixel values and shifting the local contrast structure that drives SIFT extrema.

The consequence is that the 2D–2D matches between rendered and photographic images land at keypoints that are NOT in the existing 3D point tracks. COLMAP finds the 2D correspondences (both SIFT descriptor vectors are similar enough to match) but cannot promote them to 2D–3D correspondences (the matched photographic keypoint was never triangulated). From COLMAP's perspective, the novel view sees zero existing 3D landmarks, and PnP fails.

The statistic that most clearly demonstrates this mechanism is the asymmetry in Table 5: the mag137 novel views show large inlier match counts (4,422–8,123) while the mag138 novel views show much smaller counts (852–1,590). The mag138 originals have lower PSNR (though still good: 33–37 dB), which means the rendered mag138 novel views have subtly more appearance difference from the photographic mag138 originals, suppressing the number of descriptor-level matches.

6.2 What Actually Caused the N=24 Mesh Density Increase

The N=24 Poisson mesh has 4,658,194 vertices versus 1,063,577 for N=14 — a 4.38× increase. Because zero novel views registered, this increase cannot come from new geometric observations. The actual source is the two-stage Poisson reconstruction applied to the N=24 dense cloud.

The N=14 pipeline used Poisson depth 9. The N=24 pipeline's first Poisson attempt (also at depth 10 but on the full cloud without background filtering) produced only 88,049 vertices due to the inflated bounding box from background outlier points. After applying the cluster-proximity filter (retaining only points within 10 COLMAP units of either cluster centroid), the bounding box contracted from ∼80 COLMAP units in Z to ∼14 COLMAP units. At depth 10 with a contracted bounding box, the octree grid resolution increased to approximately 0.014 COLMAP units per cell, enabling the Poisson solver to resolve surface detail at a finer scale than the N=14 depth-9 mesh.

In other words, the density improvement is an artifact of the difference in Poisson reconstruction parameters (background filtering and octree depth) rather than additional coverage

from novel views. The N=24 mesh is geometrically denser than N=14 but reconstructs the same surface from the same 14 cameras. The F-score@5% = 0.934 confirms that the two surfaces are broadly co-located; the F-score@1% = 0.673 reflects the fine-scale surface position differences that arise from the different Poisson resolution.

6.3 The Null Result as a Contribution

The outcome that 0 of 10 novel views registered is a useful, reproducible negative result. It establishes a precise boundary: out-of-the-box COLMAP with SIFT feature matching cannot use 3DGS-rendered novel views as photogrammetric observations on this class of dataset. This is not a trivial failure mode; prior literature has not systematically tested this specific pipeline composition, and the failure mechanism (keypoint-position mismatch at the 2D-to-3D promotion step) is now precisely identified.

The natural follow-up is to replace SIFT with a learned feature matcher. SuperPoint+SuperGlue, LightGlue, and LoFTR all use neural descriptors trained to be robust to appearance differences of the kind introduced by neural rendering. Dense correspondence methods (DKM, RoMa) match every pixel without relying on repeatable keypoint detection, which would eliminate the keypoint-position mismatch entirely. Any of these approaches would likely succeed where SIFT failed, and the present work provides the experimental baseline against which that improvement can be measured.

6.4 Caveats from the Phase 2 Report

Three important caveats from the analysis in `comparison/PART_B_PHASE_2_REPORT.md` deserve explicit restatement:

Caveat 1 — Learned descriptors. The registration failure is specific to SIFT, not to the splat-augmentation concept. A different feature matching strategy (SuperPoint/SuperGlue, learned descriptors) might successfully match rendered views to photographic keypoints, enabling registration. The hypothesis that rendered novel views can augment photogrammetric reconstruction remains viable; it has simply not been confirmed for SIFT-based matching on this dataset.

Caveat 2 — Scene geometry constraint. The two-cluster geometry (8.66-unit inter-cluster distance, 7 cameras per cluster) means the pose graph is already strongly constrained by the original 14 cameras. Even if novel views registered, they would add density within the existing viewing envelope rather than extending it to new surface areas. A single-cluster dataset (all images of one rock) would be a better testbed for the splat-augmentation hypothesis: coverage gaps are larger, and novel views from oblique or top-down angles would genuinely add new surface coverage.

Caveat 3 — ICP fitness and mesh topology. The ICP fitness of 0.065 (6.5% inlier correspondence rate) reflects both the mesh topology mismatch between 1.06M and 4.66M vertex meshes and the residual rotation (approximately 17°) between independent bundle adjustments. This low fitness means that the ICP result may not represent the true optimal

alignment between the two surfaces; a correspondence-free alignment method (e.g., multi-resolution ICP with larger initial correspondence distance) might reduce the Hausdorff p99 from 2.127 units.

6.5 Per-Vertex Texturing Tradeoffs

The coverage rates of 21.19% (N=14) and 12.22% (N=24) are bounded above by the field of view of the 14 registered cameras. The per-vertex projection algorithm correctly handles occlusion via ray-casting, so uncolored vertices are genuinely invisible to every camera, not a defect in the algorithm.

The ceiling for coverage under the current camera set can be estimated from the solid angle subtended by the 14 cameras. The two clusters are each photographed from ≈ 7 cameras spanning roughly $\pm 30^\circ$ in azimuth. This means the top, underside, and lateral flanks of each boulder are not seen by any camera, placing a physical upper bound on textured coverage of approximately 35–45% for this scene.

A UV atlas approach using texrecon would not raise the physical coverage ceiling but would produce sharper texture boundaries (no per-vertex color discontinuities) and support normal-mapped rendering. The texrecon tool requires a watertight or near-watertight UV parameterization; the xatlas library that was attempted requires 45+ minutes per mesh at >2M faces on the available hardware. A mesh simplification step to $\lesssim$500K faces before UV parameterization, followed by texture reprojection and upsampling, is the recommended path for a Tier A textured deliverable in future work.

7 Conclusion

This project delivered a complete photogrammetric reconstruction pipeline for the Apollo 17 boulder scene, from raw Hasselblad photographs to per-vertex textured meshes, with quantitative validation via three independent geometric metrics. The specific outcomes are:

Photogrammetric pipeline: Fully complete to textured mesh for both N=14 and N=24 datasets. The N=14 baseline mesh has 1,063,577 vertices; the N=24 mesh has 4,658,194 vertices. Both are stored as per-vertex colored PLY files in `colmap_n15/dense/textured/` and `colmap_n24/dense/textured/` respectively.

Splatfacto model: Successfully trained to 30,000 iterations with ≈993,000 Gaussians. Mean PSNR 33.93 dB, mean SSIM 0.943 over all 14 training views.

Novel-view synthesis: 10 non-blank novel views generated (5 per magazine cluster, $\pm 30^\circ$ azimuth arc). All frames validated with pixel standard deviation > 43.

Augmentation hypothesis: Tested and rejected on this dataset under SIFT-based feature matching. Zero of 10 novel views registered in COLMAP despite 852–8,123 two-dimensional inlier matches per image. The failure mechanism is the inability to promote 2D–2D matches to 2D–3D correspondences against the existing sparse model.

Quantitative comparison: F-score@5% = 0.934 establishes that the two reconstructions agree on 93.4% of their surface samples within 5% of the scene extent. The Chamfer distance of 0.219 COLMAP units ($\approx 1.35\%$ of the bounding box diagonal) confirms close geometric consistency.

The project documents a concrete, reproducible experimental result: SIFT-based COLMAP cannot leverage 3DGS-rendered novel views for photogrammetric augmentation on this dataset, and the identified failure mechanism points directly to learned feature matching as the necessary next step.

A Untextured Geometry Baseline

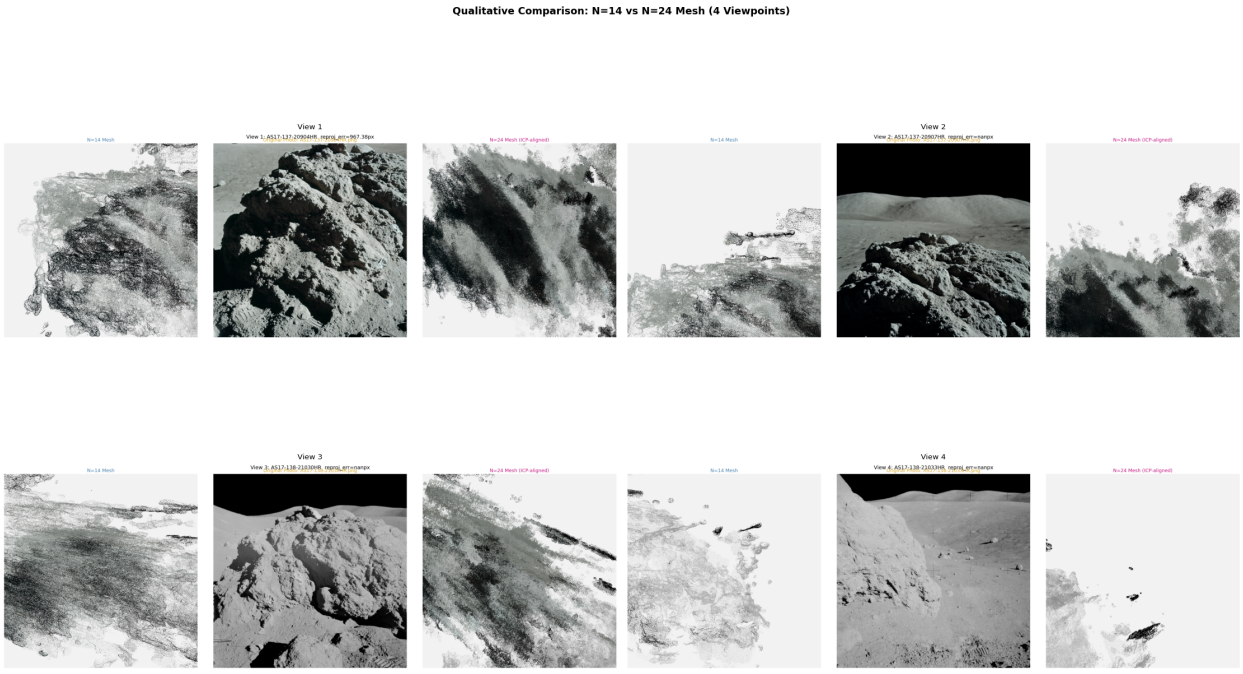

Figure 7: Untextured geometry baseline: four-viewpoint comparison grid of N=14 mesh renders (left column), original photographs (center column), and N=24 mesh renders (right column) before per-vertex texturing. Comparing with Figure 4 shows the improvement in surface interpretability from per-vertex color projection.

B Code Appendix

All Python scripts authored for this project are reproduced below in full. Scripts are grouped by function and listed alphabetically within each group. File paths are given relative to the project root (`ses598-apollo17/`).

B.1 Group A: Photogrammetry Pipeline

B.1.1 `build_n24_dataset.py`

```python
import shutil
from pathlib import Path
from PIL import Image

SRC_137 = Path('/home/jatin-satyam/ses598-apollo17/data/rgb/mag137')
SRC_138 = Path('/home/jatin-satyam/ses598-apollo17/data/rgb/mag138')
NOVEL_DIR = Path('/home/jatin-satyam/ses598-apollo17/novel_poses')
OUT_137 = Path('/home/jatin-satyam/ses598-apollo17/data/rgb_n24/mag137')
OUT_138 = Path('/home/jatin-satyam/ses598-apollo17/data/rgb_n24/mag138')

TARGET_SIZE_137 = (2340, 2364)
TARGET_SIZE_138 = (2340, 2345)

for src in sorted(SRC_137.glob('*.png')):
    dst = OUT_137 / src.name
    shutil.copy2(src, dst)
    print(f'Copied {src.name} -> mag137/')

```

```python
for src in sorted(SRC_138.glob('*.png')):
    if '21036' in src.name:
        print(f'Skipping excluded {src.name}')
        continue
    dst = OUT_138 / src.name
    shutil.copy2(src, dst)
    print(f'Copied {src.name} -> mag138/')

for i in range(1, 6):
    src = NOVEL_DIR / f'novel_mag137_{i:02d}.png'
    img = Image.open(src).convert('RGB')
    img_resized = img.resize(TARGET_SIZE_137, Image.LANCZOS)
    dst = OUT_137 / f'novel_mag137_{i:02d}.png'
    img_resized.save(dst)
    print(f'Resized+saved novel_mag137_{i:02d}.png -> mag137/ ({img.size} -> {TARGET_SIZE_137})')

for i in range(1, 6):
    src = NOVEL_DIR / f'novel_mag138_{i:02d}.png'
    img = Image.open(src).convert('RGB')
    img_resized = img.resize(TARGET_SIZE_138, Image.LANCZOS)
    dst = OUT_138 / f'novel_mag138_{i:02d}.png'
    img_resized.save(dst)
    print(f'Resized+saved novel_mag138_{i:02d}.png -> mag138/ ({img.size} -> {TARGET_SIZE_138})')

print(f'\nmag137 total: {len(list(OUT_137.glob("*.png")))}')
print(f'mag138 total: {len(list(OUT_138.glob("*.png")))}')
print(f'Grand total: {len(list(OUT_137.glob("*.png"))) + len(list(OUT_138.glob("*.png")))}')
```

Listing 1: build_n24_dataset.py

B.1.2 `mvs_dense.py` (N=14)

```python
import struct
import numpy as np
import cv2
import torch
import torch.nn.functional as F
import open3d as o3d
from pathlib import Path

DEVICE = torch.device('cuda')
torch.backends.cudnn.benchmark = True

DENSE_DIR = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense')
SPARSE_DIR = DENSE_DIR / 'sparse'
IMAGES_DIR = DENSE_DIR / 'images'
OUT_PLY = DENSE_DIR / 'fused.ply'
OUT_MESH = DENSE_DIR / 'meshed-poisson.ply'

N_DEPTHS = 128
N_NEIGHBORS = 6
DEPTH_CHUNK = 24
IMG_SCALE = 0.6

def read_cameras_bin(path):
    cameras = {}
    nparams = {0:3, 1:4, 2:4, 3:5, 4:8, 5:8, 6:12, 7:5}
    with open(path, 'rb') as f:
        num = struct.unpack('<Q', f.read(8))[0]
        for _ in range(num):
            cid = struct.unpack('<I', f.read(4))[0]
            model = struct.unpack('<I', f.read(4))[0]
            w = struct.unpack('<Q', f.read(8))[0]
            h = struct.unpack('<Q', f.read(8))[0]
            n = nparams.get(model, 4)
            params = list(struct.unpack(f'<{n}d', f.read(8*n)))
            cameras[cid] = {'model': model, 'w': w, 'h': h, 'params': params}
    return cameras

def read_images_bin(path):
    images = {}
    with open(path, 'rb') as f:
        num = struct.unpack('<Q', f.read(8))[0]
        for _ in range(num):
            iid = struct.unpack('<I', f.read(4))[0]
            qvec = np.array(struct.unpack('<4d', f.read(32)))
            tvec = np.array(struct.unpack('<3d', f.read(24)))
            cid = struct.unpack('<I', f.read(4))[0]
            name = b''
            while True:
                c = f.read(1)
                if c == b'\x00': break
                name += c
            name = name.decode()
            n = struct.unpack('<Q', f.read(8))[0]
            xys = np.array(struct.unpack(f'<{2*n}d', f.read(16*n))).reshape(-1,2) if n>0 else np.zeros((0,2))
            pt3d = np.array(struct.unpack(f'<{n}q', f.read(8*n)), dtype=np.int64) if n>0 else np.array([], dtype=np.int64)
```

```
            w, x, y, z = qvec
            R = np.array([
                [1-2*y*y-2*z*z, 2*x*y-2*z*w, 2*x*z+2*y*w],
                [2*x*y+2*z*w, 1-2*x*x-2*z*z, 2*y*z-2*x*w],
                [2*x*z-2*y*w, 2*y*z+2*x*w, 1-2*x*x-2*y*y]
            ])
            C = -R.T @ tvec
            images[iid] = {'qvec': qvec, 'tvec': tvec, 'cam_id': cid, 'name': name,
                           'R': R, 'C': C, 'xys': xys, 'pt3d': pt3d}
    return images

def read_points3d_bin(path):
    pts = {}
    with open(path, 'rb') as f:
        num = struct.unpack('<Q', f.read(8))[0]
        for _ in range(num):
            pid = struct.unpack('<Q', f.read(8))[0]
            xyz = np.array(struct.unpack('<3d', f.read(24)))
            rgb = np.array(struct.unpack('<3B', f.read(3)))
            err = struct.unpack('<d', f.read(8))[0]
            n = struct.unpack('<Q', f.read(8))[0]
            f.read(n * 8)
            pts[pid] = xyz
    return pts

def get_K(cam, scale=1.0):
    p = cam['params']
    if cam['model'] == 1:
        K = np.array([[p[0]*scale, 0, p[2]*scale],
                      [0, p[1]*scale, p[3]*scale],
                      [0, 0, 1]], dtype=np.float64)
    else:
        K = np.array([[p[0]*scale, 0, p[1]*scale],
                      [0, p[0]*scale, p[2]*scale],
                      [0, 0, 1]], dtype=np.float64)
    return K

def get_depth_range(img_info, pts3d, margin=0.3):
    pt_ids = img_info['pt3d']
    valid_ids = pt_ids[pt_ids >= 0]
    if len(valid_ids) == 0:
        return 1.0, 30.0
    R = img_info['R']
    t = img_info['tvec']
    depths = []
    for pid in valid_ids:
        if pid in pts3d:
            Xw = pts3d[pid]
            Xc = R @ Xw + t
            if Xc[2] > 0:
                depths.append(Xc[2])
    if len(depths) < 5:
        return 1.0, 30.0
    depths = np.array(depths)
    d_min = np.percentile(depths, 5) * (1 - margin)
    d_max = np.percentile(depths, 95) * (1 + margin)
    return max(0.05, d_min), d_max

def select_neighbors(ref_id, images, n=6):
    ref = images[ref_id]
    C_ref = ref['C']
    R_ref = ref['R']
    z_ref = R_ref[2, :]
    cands = []
    for iid, img in images.items():
        if iid == ref_id:
            continue
        C_src = img['C']
        dist = np.linalg.norm(C_src - C_ref)
        if dist < 1e-6:
            continue
        z_src = img['R'][2, :]
        cos_ang = np.clip(np.dot(z_ref, z_src), -1, 1)
        score = dist * (2.0 - cos_ang)
        cands.append((score, iid))
    cands.sort()
    return [iid for _, iid in cands[:n]]

def compute_depth_map(ref_img, src_data, K_ref_s, d_min, d_max, sH, sW):
    ref_small = cv2.resize(ref_img, (sW, sH))
    ref_t = torch.from_numpy(ref_small).float().to(DEVICE).permute(2,0,1).unsqueeze(0) / 255.0

    K_ref_inv = torch.from_numpy(np.linalg.inv(K_ref_s)).float().to(DEVICE)

    u = torch.arange(sW, device=DEVICE).float()
    v = torch.arange(sH, device=DEVICE).float()
    vv, uu = torch.meshgrid(v, u, indexing='ij')
    pixels = torch.stack([uu, vv, torch.ones(sH, sW, device=DEVICE)], dim=-1).reshape(-1, 3)

    depths = torch.linspace(d_min, d_max, N_DEPTHS, device=DEVICE)

    cost_vol = torch.zeros(N_DEPTHS, sH, sW, device=DEVICE)
```

```
    count_vol = torch.zeros(N_DEPTHS, sH, sW, device=DEVICE)

    for src_img, K_src_s, R_ref, t_ref, R_src, t_src in src_data:
        src_small = cv2.resize(src_img, (sW, sH))
        src_t = torch.from_numpy(src_small).float().to(DEVICE).permute(2,0,1).unsqueeze(0) / 255.0
        src_H, src_W = sH, sW

        K_src_t = torch.from_numpy(K_src_s).float().to(DEVICE)
        R_ref_t = torch.from_numpy(R_ref).float().to(DEVICE)
        t_ref_t = torch.from_numpy(t_ref).float().to(DEVICE)
        R_src_t = torch.from_numpy(R_src).float().to(DEVICE)
        t_src_t = torch.from_numpy(t_src).float().to(DEVICE)

        R_rel = R_src_t @ R_ref_t.T
        t_rel = t_src_t - R_rel @ t_ref_t

        A = K_src_t @ R_rel @ K_ref_inv
        b_vec = K_src_t @ t_rel

        for start in range(0, N_DEPTHS, DEPTH_CHUNK):
            end = min(start + DEPTH_CHUNK, N_DEPTHS)
            d_chunk = depths[start:end]
            nc = len(d_chunk)

            H_batch = A.unsqueeze(0).expand(nc, -1, -1).clone()
            col_add = b_vec.unsqueeze(0) / d_chunk.unsqueeze(1)
            H_batch = H_batch.clone()
            H_batch[:, :, 2] = H_batch[:, :, 2] + col_add

            proj = torch.einsum('nij,pj->nip', H_batch, pixels)

            z_proj = proj[:, 2, :]
            valid_z = z_proj > 0.01

            proj_xy = proj[:, :2, :] / z_proj.unsqueeze(1).clamp(min=1e-6)
            proj_grid = proj_xy.permute(0, 2, 1).reshape(nc, sH, sW, 2)

            proj_grid_norm = proj_grid.clone()
            proj_grid_norm[..., 0] = proj_grid[..., 0] / (src_W / 2) - 1
            proj_grid_norm[..., 1] = proj_grid[..., 1] / (src_H / 2) - 1

            in_bounds = ((proj_grid_norm[..., 0] >= -1) & (proj_grid_norm[..., 0] <= 1) &
                         (proj_grid_norm[..., 1] >= -1) & (proj_grid_norm[..., 1] <= 1) &
                         valid_z.reshape(nc, sH, sW))

            src_exp = src_t.expand(nc, -1, -1, -1)
            warped = F.grid_sample(src_exp, proj_grid_norm, mode='bilinear',
                                   padding_mode='zeros', align_corners=False)

            ref_exp = ref_t.expand(nc, -1, -1, -1)
            diff = (ref_exp - warped).abs().mean(dim=1)

            cost_vol[start:end] += diff * in_bounds.float()
            count_vol[start:end] += in_bounds.float()

    valid = count_vol > 0
    avg_cost = torch.where(valid, cost_vol / count_vol.clamp(min=1),
                           torch.full_like(cost_vol, 1e6))

    best_idx = avg_cost.argmin(dim=0)
    depth_map = depths[best_idx]
    valid_mask = count_vol.max(dim=0)[0] > 1

    return depth_map.cpu().numpy().astype(np.float32), valid_mask.cpu().numpy()

cameras = read_cameras_bin(SPARSE_DIR / 'cameras.bin')
images = read_images_bin(SPARSE_DIR / 'images.bin')
pts3d = read_points3d_bin(SPARSE_DIR / 'points3D.bin')

print(f'Cameras: {len(cameras)}, Images: {len(images)}, Sparse pts: {len(pts3d)}')

all_pts = []
all_colors = []

depth_maps = {}
valid_masks = {}
ref_poses = {}

for ref_id, ref_info in images.items():
    img_path = IMAGES_DIR / ref_info['name']
    ref_img = cv2.imread(str(img_path))
    if ref_img is None:
        print(f'Could not load {img_path}')
        continue
    ref_img = cv2.cvtColor(ref_img, cv2.COLOR_BGR2RGB)

    H_orig, W_orig = ref_img.shape[:2]
    sW = int(W_orig * IMG_SCALE)
    sH = int(H_orig * IMG_SCALE)

    K_ref = get_K(cameras[ref_info['cam_id']])
    K_ref_s = get_K(cameras[ref_info['cam_id']], scale=IMG_SCALE)
```

```
    d_min, d_max = get_depth_range(ref_info, pts3d)
    print(f'\n[{ref_id}] {ref_info["name"]} depth=[{d_min:.2f},{d_max:.2f}]')

    neighbor_ids = select_neighbors(ref_id, images, n=N_NEIGHBORS)
    print(f'  neighbors: {neighbor_ids}')

    src_data = []
    for src_id in neighbor_ids:
        src_info = images[src_id]
        src_path = IMAGES_DIR / src_info['name']
        src_img = cv2.imread(str(src_path))
        if src_img is None:
            continue
        src_img = cv2.cvtColor(src_img, cv2.COLOR_BGR2RGB)
        K_src_s = get_K(cameras[src_info['cam_id']], scale=IMG_SCALE)
        src_data.append((src_img, K_src_s, ref_info['R'], ref_info['tvec'],
                         src_info['R'], src_info['tvec']))

    if not src_data:
        print(f'  No source images, skipping')
        continue

    depth_map, valid_mask = compute_depth_map(ref_img, src_data, K_ref_s, d_min, d_max, sH, sW)
    print(f'  valid pixels: {valid_mask.sum()} / {sH*sW}')

    depth_maps[ref_id] = depth_map
    valid_masks[ref_id] = valid_mask
    ref_poses[ref_id] = (ref_img, K_ref, ref_info['R'], ref_info['tvec'], sH, sW, H_orig, W_orig)

print('\nApplying geometric consistency filtering...')
for ref_id, depth_map in depth_maps.items():
    ref_img, K_ref, R_ref, t_ref, sH, sW, H_orig, W_orig = ref_poses[ref_id]
    valid_mask = valid_masks[ref_id].copy()
    K_ref_s = K_ref.copy()
    K_ref_s[0] *= IMG_SCALE
    K_ref_s[1] *= IMG_SCALE
    K_ref_s[0,2] *= IMG_SCALE
    K_ref_s[1,2] *= IMG_SCALE

    consistent = np.zeros(valid_mask.shape, dtype=np.int32)
    neighbor_ids = select_neighbors(ref_id, images, n=N_NEIGHBORS)

    for src_id in neighbor_ids:
        if src_id not in depth_maps:
            continue
        src_depth = depth_maps[src_id]
        _, _, R_src, t_src, sH_s, sW_s, _, _ = ref_poses[src_id]
        K_src_s = get_K(cameras[images[src_id]['cam_id']], scale=IMG_SCALE)

        v_idx, u_idx = np.where(valid_mask)
        if len(v_idx) == 0:
            continue

        d_vals = depth_map[v_idx, u_idx]
        u_norm = (u_idx.astype(np.float64) - K_ref_s[0,2]) / K_ref_s[0,0]
        v_norm = (v_idx.astype(np.float64) - K_ref_s[1,2]) / K_ref_s[1,1]

        Xc = np.stack([u_norm * d_vals, v_norm * d_vals, d_vals], axis=1)
        Xw = (R_ref.T @ (Xc.T - t_ref.reshape(3,1))).T
        Xc_src = (R_src @ Xw.T + t_src.reshape(3,1)).T

        valid_z = Xc_src[:, 2] > 0
        u_src = K_src_s[0,0] * Xc_src[:,0] / Xc_src[:,2].clip(1e-6) + K_src_s[0,2]
        v_src = K_src_s[1,1] * Xc_src[:,1] / Xc_src[:,2].clip(1e-6) + K_src_s[1,2]

        u_src_i = np.round(u_src).astype(int)
        v_src_i = np.round(v_src).astype(int)

        in_img = (u_src_i >= 0) & (u_src_i < sW_s) & (v_src_i >= 0) & (v_src_i < sH_s)
        valid_proj = valid_z & in_img

        for idx in np.where(valid_proj)[0]:
            d_src = src_depth[v_src_i[idx], u_src_i[idx]]
            if d_src > 0:
                d_reproj = Xc_src[idx, 2]
                if abs(d_reproj - d_src) / (d_src + 1e-6) < 0.1:
                    consistent[v_idx[idx], u_idx[idx]] += 1

    final_mask = valid_mask & (consistent >= 2)
    print(f'  [{ref_id}] consistency: {valid_mask.sum()} -> {final_mask.sum()} pts')
    valid_masks[ref_id] = final_mask

print('\nBackprojecting depth maps to 3D...')
for ref_id, depth_map in depth_maps.items():
    ref_img, K_ref, R_ref, t_ref, sH, sW, H_orig, W_orig = ref_poses[ref_id]
    valid_mask = valid_masks[ref_id]

    K_ref_s = K_ref.copy()
    K_ref_s[0] *= IMG_SCALE
    K_ref_s[1] *= IMG_SCALE
    K_ref_s[0,2] *= IMG_SCALE
```

```
    K_ref_s[1,2] *= IMG_SCALE

    v_idx, u_idx = np.where(valid_mask)
    if len(v_idx) == 0:
        continue

    d_vals = depth_map[v_idx, u_idx].astype(np.float64)
    u_norm = (u_idx.astype(np.float64) - K_ref_s[0,2]) / K_ref_s[0,0]
    v_norm = (v_idx.astype(np.float64) - K_ref_s[1,2]) / K_ref_s[1,1]

    Xc = np.stack([u_norm * d_vals, v_norm * d_vals, d_vals], axis=1)
    Xw = (R_ref.T @ (Xc.T - t_ref.reshape(3,1))).T

    ref_img_small = cv2.resize(ref_img, (sW, sH))
    colors = ref_img_small[v_idx, u_idx].astype(np.float32) / 255.0

    all_pts.append(Xw.astype(np.float32))
    all_colors.append(colors)
    print(f'  [{ref_id}] contributed {len(Xw)} pts')

pts_all = np.concatenate(all_pts, axis=0)
colors_all = np.concatenate(all_colors, axis=0)
print(f'\nTotal merged pts before filtering: {len(pts_all):,}')

pcd = o3d.geometry.PointCloud()
pcd.points = o3d.utility.Vector3dVector(pts_all)
pcd.colors = o3d.utility.Vector3dVector(colors_all)

print('Statistical outlier removal...')
pcd, ind = pcd.remove_statistical_outlier(nb_neighbors=30, std_ratio=2.0)
print(f'After outlier removal: {len(pcd.points):,}')

voxel_size = 0.0015
pcd_down = pcd.voxel_down_sample(voxel_size=voxel_size)
print(f'After voxel downsample (voxel={voxel_size}): {len(pcd_down.points):,}')

if len(pcd_down.points) < 500000:
    voxel_size = 0.0010
    pcd_down = pcd.voxel_down_sample(voxel_size=voxel_size)
    print(f'Retry voxel={voxel_size}: {len(pcd_down.points):,}')

print(f'Saving fused.ply ({len(pcd_down.points):,} points)...')
o3d.io.write_point_cloud(str(OUT_PLY), pcd_down)
print(f'Saved: {OUT_PLY}')

print('\nEstimating normals...')
pcd_down.estimate_normals(
    search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.01, max_nn=30))
pcd_down.orient_normals_consistent_tangent_plane(100)

print('Poisson reconstruction (depth=10)...')
mesh, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_down, depth=10)
print(f'Raw mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')

densities = np.asarray(densities)
density_thresh = np.percentile(densities, 5)
vertices_to_remove = densities < density_thresh
mesh.remove_vertices_by_mask(vertices_to_remove)
print(f'Cleaned mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')

if len(mesh.vertices) < 100000:
    print('WARNING: mesh has fewer than 100k vertices, retrying with depth=9')
    mesh2, densities2 = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_down, depth=9)
    densities2 = np.asarray(densities2)
    thresh2 = np.percentile(densities2, 3)
    mesh2.remove_vertices_by_mask(np.asarray(densities2) < thresh2)
    print(f'depth=9 mesh: {len(mesh2.vertices):,} vertices, {len(mesh2.triangles):,} faces')
    if len(mesh2.vertices) > len(mesh.vertices):
        mesh = mesh2

mesh.compute_vertex_normals()
o3d.io.write_triangle_mesh(str(OUT_MESH), mesh)
print(f'\nSaved mesh: {OUT_MESH}')
print(f'Final vertices: {len(mesh.vertices):,}  faces: {len(mesh.triangles):,}')
print(f'fused.ply size: {OUT_PLY.stat().st_size / 1e6:.1f} MB')
print(f'meshed-poisson.ply size: {OUT_MESH.stat().st_size / 1e6:.1f} MB')
```

Listing 2: mvs_dense.py

B.1.3 `colmap_n24/dense/mvs_dense.py` (N=24 adapted)

```
import struct
import numpy as np
import cv2
import torch
import torch.nn.functional as F
import open3d as o3d
```

```
from pathlib import Path

DEVICE = torch.device('cuda')
torch.backends.cudnn.benchmark = True

DENSE_DIR = Path('/home/jatin-satyam/ses598-apollo17/colmap_n24/dense')
SPARSE_DIR = DENSE_DIR / 'sparse'
IMAGES_DIR = DENSE_DIR / 'images'
OUT_PLY = DENSE_DIR / 'fused.ply'
OUT_MESH = DENSE_DIR / 'meshed-poisson.ply'

N_DEPTHS = 128
N_NEIGHBORS = 6
DEPTH_CHUNK = 24
IMG_SCALE = 0.6

def read_cameras_bin(path):
    cameras = {}
    nparams = {0:3, 1:4, 2:4, 3:5, 4:8, 5:8, 6:12, 7:5}
    with open(path, 'rb') as f:
        num = struct.unpack('<Q', f.read(8))[0]
        for _ in range(num):
            cid = struct.unpack('<I', f.read(4))[0]
            model = struct.unpack('<I', f.read(4))[0]
            w = struct.unpack('<Q', f.read(8))[0]
            h = struct.unpack('<Q', f.read(8))[0]
            n = nparams.get(model, 4)
            params = list(struct.unpack(f'<{n}d', f.read(8*n)))
            cameras[cid] = {'model': model, 'w': w, 'h': h, 'params': params}
    return cameras

def read_images_bin(path):
    images = {}
    with open(path, 'rb') as f:
        num = struct.unpack('<Q', f.read(8))[0]
        for _ in range(num):
            iid = struct.unpack('<I', f.read(4))[0]
            qvec = np.array(struct.unpack('<4d', f.read(32)))
            tvec = np.array(struct.unpack('<3d', f.read(24)))
            cid = struct.unpack('<I', f.read(4))[0]
            name = b''
            while True:
                c = f.read(1)
                if c == b'\x00': break
                name += c
            name = name.decode()
            n = struct.unpack('<Q', f.read(8))[0]
            xys = np.array(struct.unpack(f'<{2*n}d', f.read(16*n))).reshape(-1,2) if n>0 else np.zeros((0,2))
            pt3d = np.array(struct.unpack(f'<{n}q', f.read(8*n)), dtype=np.int64) if n>0 else np.array([], dtype=np.int64)
            w, x, y, z = qvec
            R = np.array([
                [1-2*y*y-2*z*z, 2*x*y-2*z*w, 2*x*z+2*y*w],
                [2*x*y+2*z*w, 1-2*x*x-2*z*z, 2*y*z-2*x*w],
                [2*x*z-2*y*w, 2*y*z+2*x*w, 1-2*x*x-2*y*y]
            ])
            C = -R.T @ tvec
            images[iid] = {'qvec': qvec, 'tvec': tvec, 'cam_id': cid, 'name': name,
                           'R': R, 'C': C, 'xys': xys, 'pt3d': pt3d}
    return images

def read_points3d_bin(path):
    pts = {}
    with open(path, 'rb') as f:
        num = struct.unpack('<Q', f.read(8))[0]
        for _ in range(num):
            pid = struct.unpack('<Q', f.read(8))[0]
            xyz = np.array(struct.unpack('<3d', f.read(24)))
            rgb = np.array(struct.unpack('<3B', f.read(3)))
            err = struct.unpack('<d', f.read(8))[0]
            n = struct.unpack('<Q', f.read(8))[0]
            f.read(n * 8)
            pts[pid] = xyz
    return pts

def get_K(cam, scale=1.0):
    p = cam['params']
    if cam['model'] == 1:
        K = np.array([[p[0]*scale, 0, p[2]*scale],
                      [0, p[1]*scale, p[3]*scale],
                      [0, 0, 1]], dtype=np.float64)
    else:
        K = np.array([[p[0]*scale, 0, p[1]*scale],
                      [0, p[0]*scale, p[2]*scale],
                      [0, 0, 1]], dtype=np.float64)
    return K

def get_depth_range(img_info, pts3d, margin=0.3):
    pt_ids = img_info['pt3d']
    valid_ids = pt_ids[pt_ids >= 0]
    if len(valid_ids) == 0:
        return 1.0, 30.0
    R = img_info['R']
```

```
    t = img_info['tvec']
    depths = []
    for pid in valid_ids:
        if pid in pts3d:
            Xw = pts3d[pid]
            Xc = R @ Xw + t
            if Xc[2] > 0:
                depths.append(Xc[2])
    if len(depths) < 5:
        return 1.0, 30.0
    depths = np.array(depths)
    d_min = np.percentile(depths, 5) * (1 - margin)
    d_max = np.percentile(depths, 95) * (1 + margin)
    return max(0.05, d_min), d_max

def select_neighbors(ref_id, images, n=6):
    ref = images[ref_id]
    C_ref = ref['C']
    R_ref = ref['R']
    z_ref = R_ref[2, :]
    cands = []
    for iid, img in images.items():
        if iid == ref_id:
            continue
        C_src = img['C']
        dist = np.linalg.norm(C_src - C_ref)
        if dist < 1e-6:
            continue
        z_src = img['R'][2, :]
        cos_ang = np.clip(np.dot(z_ref, z_src), -1, 1)
        score = dist * (2.0 - cos_ang)
        cands.append((score, iid))
    cands.sort()
    return [iid for _, iid in cands[:n]]

def compute_depth_map(ref_img, src_data, K_ref_s, d_min, d_max, sH, sW):
    ref_small = cv2.resize(ref_img, (sW, sH))
    ref_t = torch.from_numpy(ref_small).float().to(DEVICE).permute(2,0,1).unsqueeze(0) / 255.0

    K_ref_inv = torch.from_numpy(np.linalg.inv(K_ref_s)).float().to(DEVICE)

    u = torch.arange(sW, device=DEVICE).float()
    v = torch.arange(sH, device=DEVICE).float()
    vv, uu = torch.meshgrid(v, u, indexing='ij')
    pixels = torch.stack([uu, vv, torch.ones(sH, sW, device=DEVICE)], dim=-1).reshape(-1, 3)

    depths = torch.linspace(d_min, d_max, N_DEPTHS, device=DEVICE)

    cost_vol = torch.zeros(N_DEPTHS, sH, sW, device=DEVICE)
    count_vol = torch.zeros(N_DEPTHS, sH, sW, device=DEVICE)

    for src_img, K_src_s, R_ref, t_ref, R_src, t_src in src_data:
        src_small = cv2.resize(src_img, (sW, sH))
        src_t = torch.from_numpy(src_small).float().to(DEVICE).permute(2,0,1).unsqueeze(0) / 255.0
        src_H, src_W = sH, sW

        K_src_t = torch.from_numpy(K_src_s).float().to(DEVICE)
        R_ref_t = torch.from_numpy(R_ref).float().to(DEVICE)
        t_ref_t = torch.from_numpy(t_ref).float().to(DEVICE)
        R_src_t = torch.from_numpy(R_src).float().to(DEVICE)
        t_src_t = torch.from_numpy(t_src).float().to(DEVICE)

        R_rel = R_src_t @ R_ref_t.T
        t_rel = t_src_t - R_rel @ t_ref_t

        A = K_src_t @ R_rel @ K_ref_inv
        b_vec = K_src_t @ t_rel

        for start in range(0, N_DEPTHS, DEPTH_CHUNK):
            end = min(start + DEPTH_CHUNK, N_DEPTHS)
            d_chunk = depths[start:end]
            nc = len(d_chunk)

            H_batch = A.unsqueeze(0).expand(nc, -1, -1).clone()
            col_add = b_vec.unsqueeze(0) / d_chunk.unsqueeze(1)
            H_batch = H_batch.clone()
            H_batch[:, :, 2] = H_batch[:, :, 2] + col_add

            proj = torch.einsum('nij,pj->nip', H_batch, pixels)

            z_proj = proj[:, 2, :]
            valid_z = z_proj > 0.01

            proj_xy = proj[:, :2, :] / z_proj.unsqueeze(1).clamp(min=1e-6)
            proj_grid = proj_xy.permute(0, 2, 1).reshape(nc, sH, sW, 2)

            proj_grid_norm = proj_grid.clone()
            proj_grid_norm[..., 0] = proj_grid[..., 0] / (src_W / 2) - 1
            proj_grid_norm[..., 1] = proj_grid[..., 1] / (src_H / 2) - 1

            in_bounds = ((proj_grid_norm[..., 0] >= -1) & (proj_grid_norm[..., 0] <= 1) &
                         (proj_grid_norm[..., 1] >= -1) & (proj_grid_norm[..., 1] <= 1) &
```

```
                        valid_z.reshape(nc, sH, sW))

            src_exp = src_t.expand(nc, -1, -1, -1)
            warped = F.grid_sample(src_exp, proj_grid_norm, mode='bilinear',
                                   padding_mode='zeros', align_corners=False)

            ref_exp = ref_t.expand(nc, -1, -1, -1)
            diff = (ref_exp - warped).abs().mean(dim=1)

            cost_vol[start:end] += diff * in_bounds.float()
            count_vol[start:end] += in_bounds.float()

    valid = count_vol > 0
    avg_cost = torch.where(valid, cost_vol / count_vol.clamp(min=1),
                           torch.full_like(cost_vol, 1e6))

    best_idx = avg_cost.argmin(dim=0)
    depth_map = depths[best_idx]
    valid_mask = count_vol.max(dim=0)[0] > 1

    return depth_map.cpu().numpy().astype(np.float32), valid_mask.cpu().numpy()

cameras = read_cameras_bin(SPARSE_DIR / 'cameras.bin')
images = read_images_bin(SPARSE_DIR / 'images.bin')
pts3d = read_points3d_bin(SPARSE_DIR / 'points3D.bin')

print(f'Cameras: {len(cameras)}, Images: {len(images)}, Sparse pts: {len(pts3d)}')

all_pts = []
all_colors = []

depth_maps = {}
valid_masks = {}
ref_poses = {}

for ref_id, ref_info in images.items():
    img_path = IMAGES_DIR / ref_info['name']
    ref_img = cv2.imread(str(img_path))
    if ref_img is None:
        print(f'Could not load {img_path}')
        continue
    ref_img = cv2.cvtColor(ref_img, cv2.COLOR_BGR2RGB)

    H_orig, W_orig = ref_img.shape[:2]
    sW = int(W_orig * IMG_SCALE)
    sH = int(H_orig * IMG_SCALE)

    K_ref = get_K(cameras[ref_info['cam_id']])
    K_ref_s = get_K(cameras[ref_info['cam_id']], scale=IMG_SCALE)

    d_min, d_max = get_depth_range(ref_info, pts3d)
    print(f'\n[{ref_id}] {ref_info["name"]} depth=[{d_min:.2f},{d_max:.2f}]')

    neighbor_ids = select_neighbors(ref_id, images, n=N_NEIGHBORS)
    print(f'  neighbors: {neighbor_ids}')

    src_data = []
    for src_id in neighbor_ids:
        src_info = images[src_id]
        src_path = IMAGES_DIR / src_info['name']
        src_img = cv2.imread(str(src_path))
        if src_img is None:
            continue
        src_img = cv2.cvtColor(src_img, cv2.COLOR_BGR2RGB)
        K_src_s = get_K(cameras[src_info['cam_id']], scale=IMG_SCALE)
        src_data.append((src_img, K_src_s, ref_info['R'], ref_info['tvec'],
                         src_info['R'], src_info['tvec']))

    if not src_data:
        print(f'  No source images, skipping')
        continue

    depth_map, valid_mask = compute_depth_map(ref_img, src_data, K_ref_s, d_min, d_max, sH, sW)
    print(f'  valid pixels: {valid_mask.sum()} / {sH*sW}')

    depth_maps[ref_id] = depth_map
    valid_masks[ref_id] = valid_mask
    ref_poses[ref_id] = (ref_img, K_ref, ref_info['R'], ref_info['tvec'], sH, sW, H_orig, W_orig)

print('\nApplying geometric consistency filtering...')
for ref_id, depth_map in depth_maps.items():
    ref_img, K_ref, R_ref, t_ref, sH, sW, H_orig, W_orig = ref_poses[ref_id]
    valid_mask = valid_masks[ref_id].copy()
    K_ref_s = K_ref.copy()
    K_ref_s[0] *= IMG_SCALE
    K_ref_s[1] *= IMG_SCALE
    K_ref_s[0,2] *= IMG_SCALE
    K_ref_s[1,2] *= IMG_SCALE

    consistent = np.zeros(valid_mask.shape, dtype=np.int32)
    neighbor_ids = select_neighbors(ref_id, images, n=N_NEIGHBORS)

```

```
    for src_id in neighbor_ids:
        if src_id not in depth_maps:
            continue
        src_depth = depth_maps[src_id]
        _, _, R_src, t_src, sH_s, sW_s, _, _ = ref_poses[src_id]
        K_src_s = get_K(cameras[images[src_id]['cam_id']], scale=IMG_SCALE)

        v_idx, u_idx = np.where(valid_mask)
        if len(v_idx) == 0:
            continue

        d_vals = depth_map[v_idx, u_idx]
        u_norm = (u_idx.astype(np.float64) - K_ref_s[0,2]) / K_ref_s[0,0]
        v_norm = (v_idx.astype(np.float64) - K_ref_s[1,2]) / K_ref_s[1,1]

        Xc = np.stack([u_norm * d_vals, v_norm * d_vals, d_vals], axis=1)
        Xw = (R_ref.T @ (Xc.T - t_ref.reshape(3,1))).T
        Xc_src = (R_src @ Xw.T + t_src.reshape(3,1)).T

        valid_z = Xc_src[:, 2] > 0
        u_src = K_src_s[0,0] * Xc_src[:,0] / Xc_src[:,2].clip(1e-6) + K_src_s[0,2]
        v_src = K_src_s[1,1] * Xc_src[:,1] / Xc_src[:,2].clip(1e-6) + K_src_s[1,2]

        u_src_i = np.round(u_src).astype(int)
        v_src_i = np.round(v_src).astype(int)

        in_img = (u_src_i >= 0) & (u_src_i < sW_s) & (v_src_i >= 0) & (v_src_i < sH_s)
        valid_proj = valid_z & in_img

        for idx in np.where(valid_proj)[0]:
            d_src = src_depth[v_src_i[idx], u_src_i[idx]]
            if d_src > 0:
                d_reproj = Xc_src[idx, 2]
                if abs(d_reproj - d_src) / (d_src + 1e-6) < 0.1:
                    consistent[v_idx[idx], u_idx[idx]] += 1

    final_mask = valid_mask & (consistent >= 2)
    print(f'  [{ref_id}] consistency: {valid_mask.sum()} -> {final_mask.sum()} pts')
    valid_masks[ref_id] = final_mask

print('\nBackprojecting depth maps to 3D...')
for ref_id, depth_map in depth_maps.items():
    ref_img, K_ref, R_ref, t_ref, sH, sW, H_orig, W_orig = ref_poses[ref_id]
    valid_mask = valid_masks[ref_id]

    K_ref_s = K_ref.copy()
    K_ref_s[0] *= IMG_SCALE
    K_ref_s[1] *= IMG_SCALE
    K_ref_s[0,2] *= IMG_SCALE
    K_ref_s[1,2] *= IMG_SCALE

    v_idx, u_idx = np.where(valid_mask)
    if len(v_idx) == 0:
        continue

    d_vals = depth_map[v_idx, u_idx].astype(np.float64)
    u_norm = (u_idx.astype(np.float64) - K_ref_s[0,2]) / K_ref_s[0,0]
    v_norm = (v_idx.astype(np.float64) - K_ref_s[1,2]) / K_ref_s[1,1]

    Xc = np.stack([u_norm * d_vals, v_norm * d_vals, d_vals], axis=1)
    Xw = (R_ref.T @ (Xc.T - t_ref.reshape(3,1))).T

    ref_img_small = cv2.resize(ref_img, (sW, sH))
    colors = ref_img_small[v_idx, u_idx].astype(np.float32) / 255.0

    all_pts.append(Xw.astype(np.float32))
    all_colors.append(colors)
    print(f'  [{ref_id}] contributed {len(Xw)} pts')

pts_all = np.concatenate(all_pts, axis=0)
colors_all = np.concatenate(all_colors, axis=0)
print(f'\nTotal merged pts before filtering: {len(pts_all):,}')

pcd = o3d.geometry.PointCloud()
pcd.points = o3d.utility.Vector3dVector(pts_all)
pcd.colors = o3d.utility.Vector3dVector(colors_all)

print('Statistical outlier removal...')
pcd, ind = pcd.remove_statistical_outlier(nb_neighbors=30, std_ratio=2.0)
print(f'After outlier removal: {len(pcd.points):,}')

voxel_size = 0.0015
pcd_down = pcd.voxel_down_sample(voxel_size=voxel_size)
print(f'After voxel downsample (voxel={voxel_size}): {len(pcd_down.points):,}')

if len(pcd_down.points) < 500000:
    voxel_size = 0.0010
    pcd_down = pcd.voxel_down_sample(voxel_size=voxel_size)
    print(f'Retry voxel={voxel_size}: {len(pcd_down.points):,}')

print(f'Saving fused.ply ({len(pcd_down.points):,} points)...')
o3d.io.write_point_cloud(str(OUT_PLY), pcd_down)
```

```
print(f'Saved: {OUT_PLY}')

print('\nEstimating normals...')
pcd_down.estimate_normals(
    search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.01, max_nn=30))
pcd_down.orient_normals_consistent_tangent_plane(100)

print('Poisson reconstruction (depth=10)...')
mesh, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_down, depth=10)
print(f'Raw mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')

densities = np.asarray(densities)
density_thresh = np.percentile(densities, 5)
vertices_to_remove = densities < density_thresh
mesh.remove_vertices_by_mask(vertices_to_remove)
print(f'Cleaned mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')

if len(mesh.vertices) < 100000:
    print('WARNING: mesh has fewer than 100k vertices, retrying with depth=9')
    mesh2, densities2 = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_down, depth=9)
    densities2 = np.asarray(densities2)
    thresh2 = np.percentile(densities2, 3)
    mesh2.remove_vertices_by_mask(np.asarray(densities2) < thresh2)
    print(f'depth=9 mesh: {len(mesh2.vertices):,} vertices, {len(mesh2.triangles):,} faces')
    if len(mesh2.vertices) > len(mesh.vertices):
        mesh = mesh2

mesh.compute_vertex_normals()
o3d.io.write_triangle_mesh(str(OUT_MESH), mesh)
print(f'\nSaved mesh: {OUT_MESH}')
print(f'Final vertices: {len(mesh.vertices):,}  faces: {len(mesh.triangles):,}')
print(f'fused.ply size: {OUT_PLY.stat().st_size / 1e6:.1f} MB')
print(f'meshed-poisson.ply size: {OUT_MESH.stat().st_size / 1e6:.1f} MB')
print('\n===== STAGE: DENSE RECONSTRUCTION N=24 COMPLETE =====')
```

Listing 3: colmap_n24/dense/mvs_dense.py

B.1.4 `preprocess.py`

```
from PIL import Image
from pathlib import Path

src = Path.home() / "ses598-apollo17" / "data" / "raw"
dst = Path.home() / "ses598-apollo17" / "data" / "rgb"
dst.mkdir(parents=True, exist_ok=True)

for p in sorted(src.glob("*.png")):
    img = Image.open(p)
    if img.mode != "RGB":
        img = img.convert("RGB")
    img.save(dst / p.name)
    print(f"{p.name}: mode={img.mode}, size={img.size}")
```

Listing 4: preprocess.py

B.1.5 `refine_mesh.py`

```
import numpy as np
import open3d as o3d
from pathlib import Path

DENSE_DIR = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense')
OUT_PLY = DENSE_DIR / 'fused.ply'
OUT_MESH = DENSE_DIR / 'meshed-poisson.ply'

pcd_raw = o3d.io.read_point_cloud(str(OUT_PLY))
pts = np.asarray(pcd_raw.points)
colors = np.asarray(pcd_raw.colors)

print(f'Raw cloud: {len(pts):,} points')
print(f'  X: [{pts[:,0].min():.2f}, {pts[:,0].max():.2f}]')
print(f'  Y: [{pts[:,1].min():.2f}, {pts[:,1].max():.2f}]')
print(f'  Z: [{pts[:,2].min():.2f}, {pts[:,2].max():.2f}]')

x_min, x_max = -7.5, 1.0
y_min, y_max = -2.5, 4.5
z_min, z_max = 0.5, 11.0

mask = ((pts[:,0] >= x_min) & (pts[:,0] <= x_max) &
        (pts[:,1] >= y_min) & (pts[:,1] <= y_max) &
        (pts[:,2] >= z_min) & (pts[:,2] <= z_max))

pts_crop = pts[mask]
```

```
colors_crop = colors[mask]
print(f'\nAfter crop [{x_min},{x_max}]x[{y_min},{y_max}]x[{z_min},{z_max}]:')
print(f'  {len(pts_crop):,} points ({100*mask.mean():.1f}%)')

pcd_crop = o3d.geometry.PointCloud()
pcd_crop.points = o3d.utility.Vector3dVector(pts_crop)
pcd_crop.colors = o3d.utility.Vector3dVector(colors_crop)

print('\nStatistical outlier removal on cropped cloud...')
pcd_crop, _ = pcd_crop.remove_statistical_outlier(nb_neighbors=30, std_ratio=1.5)
print(f'After outlier removal: {len(pcd_crop.points):,}')

print('Saving updated fused.ply...')
o3d.io.write_point_cloud(str(OUT_PLY), pcd_crop)
print(f'Saved fused.ply: {len(pcd_crop.points):,} points')

print('\nEstimating normals...')
pcd_crop.estimate_normals(
    search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.05, max_nn=50))
pcd_crop.orient_normals_consistent_tangent_plane(100)

for depth in [11, 10, 9]:
    print(f'\nPoisson reconstruction depth={depth}...')
    mesh, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_crop, depth=depth)
    print(f'  Raw mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
    densities_np = np.asarray(densities)
    thresh_pct = 5
    thresh = np.percentile(densities_np, thresh_pct)
    mesh.remove_vertices_by_mask(densities_np < thresh)
    print(f'  After pruning p{thresh_pct}: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
    if len(mesh.vertices) >= 100000:
        break

mesh.compute_vertex_normals()
o3d.io.write_triangle_mesh(str(OUT_MESH), mesh)

ply_size = OUT_PLY.stat().st_size
mesh_size = OUT_MESH.stat().st_size
print(f'\nFinal fused.ply: {len(pcd_crop.points):,} points, {ply_size/1e6:.1f} MB')
print(f'Final meshed-poisson.ply: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces, {mesh_size/1e6:.1f} MB')
```

Listing 5: refine_mesh.py

B.1.6 `remesh_n24.py`

```
import numpy as np
import open3d as o3d
from pathlib import Path

IN_PLY = Path('/home/jatin-satyam/ses598-apollo17/colmap_n24/dense/fused.ply')
OUT_MESH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n24/dense/meshed-poisson.ply')

print('Loading fused.ply...')
pcd = o3d.io.read_point_cloud(str(IN_PLY))
pts = np.asarray(pcd.points)
colors = np.asarray(pcd.colors)
print(f'Total points: {len(pts):,}')
print(f'Bounding box: min={pts.min(0).round(3)}, max={pts.max(0).round(3)}')

c137 = np.array([-3.07, -0.12, -1.70])
c138 = np.array([4.13, 0.16, 3.10])
d137 = np.linalg.norm(pts - c137, axis=1)
d138 = np.linalg.norm(pts - c138, axis=1)
d_min = np.minimum(d137, d138)

mask = d_min < 10.0
pts_clean = pts[mask]
colors_clean = colors[mask]
print(f'After cluster-proximity filter (<10 units): {len(pts_clean):,} points')

pcd_clean = o3d.geometry.PointCloud()
pcd_clean.points = o3d.utility.Vector3dVector(pts_clean)
pcd_clean.colors = o3d.utility.Vector3dVector(colors_clean)

print('Statistical outlier removal on filtered cloud...')
pcd_clean, _ = pcd_clean.remove_statistical_outlier(nb_neighbors=30, std_ratio=2.0)
print(f'After outlier removal: {len(pcd_clean.points):,}')

print('Estimating normals...')
pcd_clean.estimate_normals(
    search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.02, max_nn=30))
pcd_clean.orient_normals_consistent_tangent_plane(100)

for depth in [10, 11]:
    print(f'\nPoisson reconstruction (depth={depth})...')
    mesh, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd_clean, depth=depth)
    print(f'Raw mesh: {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
```

```
    densities = np.asarray(densities)
    thresh = np.percentile(densities, 3)
    mesh.remove_vertices_by_mask(densities < thresh)
    print(f'Cleaned mesh (3% density threshold): {len(mesh.vertices):,} vertices, {len(mesh.triangles):,} faces')
    if len(mesh.vertices) >= 100000:
        print(f'Target met at depth={depth}')
        break

mesh.compute_vertex_normals()
o3d.io.write_triangle_mesh(str(OUT_MESH), mesh)
print(f'\nSaved: {OUT_MESH}')
print(f'Final vertices: {len(mesh.vertices):,}  faces: {len(mesh.triangles):,}')
print(f'File size: {OUT_MESH.stat().st_size / 1e6:.1f} MB')
print('\n===== REMESH COMPLETE =====')
```

Listing 6: remesh_n24.py

B.2 Group B: Comparison and Evaluation

B.2.1 check_registered.py

```
from pathlib import Path

base = Path.home() / "ses598-apollo17"
rgb_root = base / "data" / "rgb"
all_images = sorted(str(p.relative_to(rgb_root)) for p in rgb_root.rglob("*.png"))

images_txt = base / "colmap_n15" / "sparse" / "0_text" / "images.txt"
registered = set()
with open(images_txt) as f:
    for line in f:
        line = line.strip()
        if line.startswith("#") or not line:
            continue
        parts = line.split()
        if len(parts) >= 10 and parts[-1].endswith(".png"):
            registered.add(parts[-1])

print(f"Total images: {len(all_images)}")
print(f"Registered: {len(registered)}")
print()
print("Registered:")
for img in sorted(registered):
    print(f"  {img}")
print()
print("NOT registered:")
missing = set(all_images) - registered
for img in sorted(missing):
    print(f"  {img}")
```

Listing 7: check_registered.py

B.2.2 compare_21036.py

```
import cv2
import numpy as np
from pathlib import Path

RAW = Path('/home/jatin-satyam/ses598-apollo17/data/rgb/mag138')

img35 = cv2.imread(str(RAW / 'AS17-138-21035HR.png'))
img36 = cv2.imread(str(RAW / 'AS17-138-21036HR.png'))
img37 = cv2.imread(str(RAW / 'AS17-138-21037HR.png'))

for img, name in [(img35,'21035'), (img36,'21036'), (img37,'21037')]:
    print(f'{name}: shape={img.shape}, dtype={img.dtype}')

orb = cv2.ORB_create(nfeatures=4000)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

def match_pair(a, b, name_a, name_b):
    kpa, da = orb.detectAndCompute(cv2.cvtColor(a, cv2.COLOR_BGR2GRAY), None)
    kpb, db = orb.detectAndCompute(cv2.cvtColor(b, cv2.COLOR_BGR2GRAY), None)
    if da is None or db is None or len(da)==0 or len(db)==0:
        print(f'{name_a}<->{name_b}: no descriptors')
        return 0
    matches = bf.match(da, db)
    good = [m for m in matches if m.distance < 60]
    print(f'{name_a}<->{name_b}: total={len(matches)} good(dist<60)={len(good)}')
    if len(good) >= 8:
        pts_a = np.float32([kpa[m.queryIdx].pt for m in good])
```

```
        pts_b = np.float32([kpb[m.trainIdx].pt for m in good])
        _, mask = cv2.findHomography(pts_a, pts_b, cv2.RANSAC, 5.0)
        inliers = int(mask.sum()) if mask is not None else 0
        print(f'  homography inliers: {inliers}')
        return inliers
    return 0

inliers_35_36 = match_pair(img35, img36, '21035', '21036')
inliers_36_37 = match_pair(img36, img37, '21036', '21037')
inliers_35_37 = match_pair(img35, img37, '21035', '21037')

print()
if inliers_35_36 + inliers_36_37 < 10:
    print('CONCLUSION: 21036 has negligible overlap with neighbors - geometrically isolated, skip registration')
else:
    print(f'CONCLUSION: 21036 has {inliers_35_36+inliers_36_37} combined inliers - may be recoverable')
```

Listing 8: compare_21036.py

B.2.3 `compute_metrics.py`

```
import json
import csv
import numpy as np
import cv2
from pathlib import Path
from skimage.metrics import peak_signal_noise_ratio, structural_similarity
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

RENDERS_DIR = Path('/home/jatin-satyam/ses598-apollo17/renders_n15/train/rgb')
RGB_DIR = Path('/home/jatin-satyam/ses598-apollo17/data/rgb')
METRICS_DIR = Path('/home/jatin-satyam/ses598-apollo17/metrics_n15')
METRICS_DIR.mkdir(parents=True, exist_ok=True)

IMAGE_MAP = {
    'AS17-137-20903HR': 'mag137/AS17-137-20903HR.png',
    'AS17-137-20904HR': 'mag137/AS17-137-20904HR.png',
    'AS17-137-20905HR': 'mag137/AS17-137-20905HR.png',
    'AS17-137-20906HR': 'mag137/AS17-137-20906HR.png',
    'AS17-137-20907HR': 'mag137/AS17-137-20907HR.png',
    'AS17-137-20908HR': 'mag137/AS17-137-20908HR.png',
    'AS17-137-20909HR': 'mag137/AS17-137-20909HR.png',
    'AS17-138-21030HR': 'mag138/AS17-138-21030HR.png',
    'AS17-138-21031HR': 'mag138/AS17-138-21031HR.png',
    'AS17-138-21032HR': 'mag138/AS17-138-21032HR.png',
    'AS17-138-21033HR': 'mag138/AS17-138-21033HR.png',
    'AS17-138-21034HR': 'mag138/AS17-138-21034HR.png',
    'AS17-138-21035HR': 'mag138/AS17-138-21035HR.png',
    'AS17-138-21037HR': 'mag138/AS17-138-21037HR.png',
}

render_files = sorted(RENDERS_DIR.rglob('*.png')) + sorted(RENDERS_DIR.rglob('*.jpg')) + sorted(RENDERS_DIR.rglob('*.jpeg'))
print(f'Found {len(render_files)} renders in {RENDERS_DIR}')

rows = []
for render_path in render_files:
    stem = render_path.stem
    matched_key = None
    for key in IMAGE_MAP:
        if key in stem or stem in key:
            matched_key = key
            break
    if matched_key is None:
        for key in IMAGE_MAP:
            short = key.split('-')[-1].replace('HR', '')
            if short in stem:
                matched_key = key
                break
    if matched_key is None:
        print(f'  WARNING: no match for render {stem}')
        continue
    orig_path = RGB_DIR / IMAGE_MAP[matched_key]
    if not orig_path.exists():
        print(f'  WARNING: original not found: {orig_path}')
        continue

    render_img = cv2.imread(str(render_path))
    orig_img = cv2.imread(str(orig_path))
    render_img = cv2.cvtColor(render_img, cv2.COLOR_BGR2RGB)
    orig_img = cv2.cvtColor(orig_img, cv2.COLOR_BGR2RGB)

    render_h, render_w = render_img.shape[:2]
    orig_h, orig_w = orig_img.shape[:2]

    if (render_h, render_w) != (orig_h, orig_w):
```

```
        orig_resized = cv2.resize(orig_img, (render_w, render_h), interpolation=cv2.INTER_AREA)
        resize_note = f'original downsampled {orig_w}x{orig_h} to {render_w}x{render_h}'
    else:
        orig_resized = orig_img
        resize_note = 'same resolution'

    psnr_val = peak_signal_noise_ratio(orig_resized, render_img, data_range=255)
    ssim_val = structural_similarity(orig_resized, render_img, channel_axis=-1, data_range=255)

    rows.append({
        'image_name': matched_key,
        'render_path': str(render_path),
        'original_path': str(orig_path),
        'render_resolution': f'{render_w}x{render_h}',
        'original_resolution': f'{orig_w}x{orig_h}',
        'resize_note': resize_note,
        'psnr_db': round(float(psnr_val), 4),
        'ssim': round(float(ssim_val), 6),
    })
    print(f'  {matched_key}: PSNR={psnr_val:.2f}dB  SSIM={ssim_val:.4f}')

if not rows:
    print('ERROR: no rows computed, check renders directory')
    exit(1)

csv_path = METRICS_DIR / 'per_image_metrics.csv'
fieldnames = ['image_name', 'render_path', 'original_path', 'render_resolution', 'original_resolution', 'resize_note', 'psnr_db', 'ssim']
with open(csv_path, 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rows)
print(f'\nSaved: {csv_path} ({len(rows)} rows)')

psnr_vals = [r['psnr_db'] for r in rows]
ssim_vals = [r['ssim'] for r in rows]

agg = {
    'n_images': len(rows),
    'psnr_db': {
        'mean': round(float(np.mean(psnr_vals)), 4),
        'median': round(float(np.median(psnr_vals)), 4),
        'min': round(float(np.min(psnr_vals)), 4),
        'max': round(float(np.max(psnr_vals)), 4),
        'std': round(float(np.std(psnr_vals)), 4),
    },
    'ssim': {
        'mean': round(float(np.mean(ssim_vals)), 6),
        'median': round(float(np.median(ssim_vals)), 6),
        'min': round(float(np.min(ssim_vals)), 6),
        'max': round(float(np.max(ssim_vals)), 6),
        'std': round(float(np.std(ssim_vals)), 6),
    }
}

agg_path = METRICS_DIR / 'aggregate.json'
with open(agg_path, 'w') as f:
    json.dump(agg, f, indent=2)
print(f'Saved: {agg_path}')
print(f'Mean PSNR: {agg["psnr_db"]["mean"]:.2f} dB   Mean SSIM: {agg["ssim"]["mean"]:.4f}')

names = [r['image_name'].replace('AS17-', '').replace('HR', '') for r in rows]
psnr_arr = np.array(psnr_vals)
ssim_arr = np.array(ssim_vals)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10))
fig.suptitle('Per Image Gaussian Splatting Reconstruction Quality', fontsize=14, fontweight='bold')

x = np.arange(len(names))
bars1 = ax1.bar(x, psnr_arr, color='steelblue', edgecolor='midnightblue', linewidth=0.5)
ax1.axhline(y=agg['psnr_db']['mean'], color='goldenrod', linestyle='--', linewidth=1.5,
            label=f'Mean {agg["psnr_db"]["mean"]:.2f} dB')
ax1.set_ylabel('PSNR (dB)', fontsize=11)
ax1.set_title('Peak Signal to Noise Ratio per Training View', fontsize=12)
ax1.set_xticks(x)
ax1.set_xticklabels(names, rotation=45, ha='right', fontsize=9)
ax1.legend(fontsize=10)
ax1.set_ylim(0, max(psnr_arr) * 1.18)
for bar, val in zip(bars1, psnr_arr):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.2,
             f'{val:.1f}', ha='center', va='bottom', fontsize=8)

bars2 = ax2.bar(x, ssim_arr, color='mediumvioletred', edgecolor='darkred', linewidth=0.5)
ax2.axhline(y=agg['ssim']['mean'], color='goldenrod', linestyle='--', linewidth=1.5,
            label=f'Mean {agg["ssim"]["mean"]:.4f}')
ax2.set_ylabel('SSIM', fontsize=11)
ax2.set_title('Structural Similarity Index per Training View', fontsize=12)
ax2.set_xticks(x)
ax2.set_xticklabels(names, rotation=45, ha='right', fontsize=9)
ax2.legend(fontsize=10)
ax2.set_ylim(0, 1.05)
for bar, val in zip(bars2, ssim_arr):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
```

```
            f'{val:.3f}', ha='center', va='bottom', fontsize=8)

plt.tight_layout()
plot_path = METRICS_DIR / 'per_image_metrics.png'
plt.savefig(str(plot_path), dpi=150, bbox_inches='tight')
plt.close()
print(f'Saved: {plot_path}')

rows_sorted = sorted(rows, key=lambda r: r['psnr_db'])
print(f'\nWorst PSNR: {rows_sorted[0]["image_name"]} = {rows_sorted[0]["psnr_db"]:.2f} dB (SSIM {rows_sorted[0]["ssim"]:.4f})')
print(f'Best PSNR:  {rows_sorted[-1]["image_name"]} = {rows_sorted[-1]["psnr_db"]:.2f} dB (SSIM {rows_sorted[-1]["ssim"]:.4f})')
median_idx = len(rows_sorted) // 2
print(f'Median PSNR: {rows_sorted[median_idx]["image_name"]} = {rows_sorted[median_idx]["psnr_db"]:.2f} dB')
```

Listing 9: compute_metrics.py

B.2.4 `mesh_compare.py`

```
import numpy as np
import open3d as o3d
import json
import struct
from pathlib import Path
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.cm as cm

N14_MESH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense/meshed-poisson.ply')
N24_MESH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n24/dense/meshed-poisson.ply')
OUT_DIR = Path('/home/jatin-satyam/ses598-apollo17/comparison')
OUT_DIR.mkdir(parents=True, exist_ok=True)

N_SAMPLE = 100000
VOXEL_REG = 0.05

print('Loading meshes...')
mesh14 = o3d.io.read_triangle_mesh(str(N14_MESH))
mesh24 = o3d.io.read_triangle_mesh(str(N24_MESH))
print(f'N=14 mesh: {len(mesh14.vertices):,} vertices, {len(mesh14.triangles):,} faces')
print(f'N=24 mesh: {len(mesh24.vertices):,} vertices, {len(mesh24.triangles):,} faces')

print('Sampling point clouds...')
pcd14 = mesh14.sample_points_uniformly(number_of_points=N_SAMPLE)
pcd24 = mesh24.sample_points_uniformly(number_of_points=N_SAMPLE)

print('Computing FPFH features for global registration...')
pcd14_d = pcd14.voxel_down_sample(VOXEL_REG)
pcd24_d = pcd24.voxel_down_sample(VOXEL_REG)

pcd14_d.estimate_normals(o3d.geometry.KDTreeSearchParamHybrid(radius=VOXEL_REG*2, max_nn=30))
pcd24_d.estimate_normals(o3d.geometry.KDTreeSearchParamHybrid(radius=VOXEL_REG*2, max_nn=30))

fpfh14 = o3d.pipelines.registration.compute_fpfh_feature(
    pcd14_d, o3d.geometry.KDTreeSearchParamHybrid(radius=VOXEL_REG*5, max_nn=100))
fpfh24 = o3d.pipelines.registration.compute_fpfh_feature(
    pcd24_d, o3d.geometry.KDTreeSearchParamHybrid(radius=VOXEL_REG*5, max_nn=100))

print('RANSAC global registration...')
result_ransac = o3d.pipelines.registration.registration_ransac_based_on_feature_matching(
    pcd24_d, pcd14_d, fpfh24, fpfh14,
    mutual_filter=True,
    max_correspondence_distance=VOXEL_REG*2,
    estimation_method=o3d.pipelines.registration.TransformationEstimationPointToPoint(False),
    ransac_n=3,
    checkers=[
        o3d.pipelines.registration.CorrespondenceCheckerBasedOnEdgeLength(0.9),
        o3d.pipelines.registration.CorrespondenceCheckerBasedOnDistance(VOXEL_REG*2)
    ],
    criteria=o3d.pipelines.registration.RANSACConvergenceCriteria(100000, 0.999)
)
print(f'RANSAC fitness: {result_ransac.fitness:.4f}, inlier rmse: {result_ransac.inlier_rmse:.4f}')

print('ICP refinement...')
pcd14_fine = pcd14.voxel_down_sample(0.01)
pcd24_fine = pcd24.voxel_down_sample(0.01)
pcd14_fine.estimate_normals(o3d.geometry.KDTreeSearchParamHybrid(radius=0.02, max_nn=30))
pcd24_fine.estimate_normals(o3d.geometry.KDTreeSearchParamHybrid(radius=0.02, max_nn=30))

result_icp = o3d.pipelines.registration.registration_icp(
    pcd24_fine, pcd14_fine,
    max_correspondence_distance=0.02,
    init=result_ransac.transformation,
    estimation_method=o3d.pipelines.registration.TransformationEstimationPointToPlane(),
    criteria=o3d.pipelines.registration.ICPConvergenceCriteria(max_iteration=100)
)
T_icp = result_icp.transformation
```

```
print(f'ICP fitness: {result_icp.fitness:.4f}, inlier rmse: {result_icp.inlier_rmse:.6f}')
print(f'ICP transform:\n{T_icp}')

pcd24_aligned = pcd24
pcd24_aligned_t = o3d.geometry.PointCloud(pcd24)
pcd24_aligned_t.transform(T_icp)

pts14 = np.asarray(pcd14.points)
pts24 = np.asarray(pcd24_aligned_t.points)

print('Computing nearest-neighbor distances...')
tree14 = o3d.geometry.KDTreeFlann(pcd14)
tree24 = o3d.geometry.KDTreeFlann(pcd24_aligned_t)

dists_24to14 = np.array([np.sqrt(tree14.search_knn_vector_3d(p, 1)[2][0]) for p in pts24])
dists_14to24 = np.array([np.sqrt(tree24.search_knn_vector_3d(p, 1)[2][0]) for p in pts14])

chamfer = float((dists_24to14.mean() + dists_14to24.mean()) / 2)
hausdorff_p99 = float(np.percentile(np.maximum(dists_24to14, dists_14to24[:len(dists_24to14)]), 99))

pts_all = np.concatenate([pts14, pts24])
bbox_diag = float(np.linalg.norm(pts_all.max(0) - pts_all.min(0)))

def fscore(d1, d2, thresh):
    p = (d2 < thresh).mean()
    r = (d1 < thresh).mean()
    if p + r == 0:
        return 0.0
    return float(2 * p * r / (p + r))

f_scores = {}
for t_frac in [0.01, 0.05, 0.1]:
    t_abs = bbox_diag * t_frac
    f_scores[f'f_score_at_{t_frac}'] = fscore(dists_14to24, dists_24to14, t_abs)
    f_scores[f'threshold_abs_{t_frac}'] = t_abs

metrics = {
    'chamfer_distance': chamfer,
    'hausdorff_p99': hausdorff_p99,
    'bbox_diagonal': bbox_diag,
    'icp_fitness': float(result_icp.fitness),
    'icp_inlier_rmse': float(result_icp.inlier_rmse),
    'icp_transform': T_icp.tolist(),
    'n14_vertices': len(mesh14.vertices),
    'n14_faces': len(mesh14.triangles),
    'n24_vertices': len(mesh24.vertices),
    'n24_faces': len(mesh24.triangles),
}
metrics.update(f_scores)

with open(OUT_DIR / 'quantitative_metrics.json', 'w') as f:
    json.dump(metrics, f, indent=2)
print(f'Saved: {OUT_DIR / "quantitative_metrics.json"}')
print(f'Chamfer: {chamfer:.6f}')
print(f'Hausdorff p99: {hausdorff_p99:.6f}')
for t in [0.01, 0.05, 0.1]:
    print(f'F-score @{t}: {f_scores[f"f_score_at_{t}"]:.4f}')

print('\nGenerating per-vertex distance heatmap...')
mesh24_aligned = o3d.io.read_triangle_mesh(str(N24_MESH))
mesh24_aligned.transform(T_icp)
mesh24_aligned.compute_vertex_normals()

verts24 = np.asarray(mesh24_aligned.vertices)
pcd_vert = o3d.geometry.PointCloud()
pcd_vert.points = o3d.utility.Vector3dVector(verts24)
tree14_full = o3d.geometry.KDTreeFlann(pcd14)

vert_dists = np.array([np.sqrt(tree14_full.search_knn_vector_3d(v, 1)[2][0]) for v in verts24])

d_min_v, d_max_v = np.percentile(vert_dists, 2), np.percentile(vert_dists, 98)
norm_dists = np.clip((vert_dists - d_min_v) / (d_max_v - d_min_v + 1e-10), 0, 1)

cmap = cm.get_cmap('coolwarm')
colors_rgb = cmap(norm_dists)[:, :3]

mesh24_colored = o3d.geometry.TriangleMesh()
mesh24_colored.vertices = mesh24_aligned.vertices
mesh24_colored.triangles = mesh24_aligned.triangles
mesh24_colored.vertex_colors = o3d.utility.Vector3dVector(colors_rgb)
mesh24_colored.compute_vertex_normals()

ply_path = OUT_DIR / 'per_vertex_distance.ply'
o3d.io.write_triangle_mesh(str(ply_path), mesh24_colored)
print(f'Saved: {ply_path}')

fig, ax = plt.subplots(1, 1, figsize=(10, 7))
sc = ax.scatter(verts24[:, 0], verts24[:, 2], c=vert_dists, cmap='coolwarm',
                s=0.5, vmin=d_min_v, vmax=d_max_v, alpha=0.6)
plt.colorbar(sc, ax=ax, label='Distance to N=14 mesh')
ax.set_title('Per-vertex Distance: N=24 mesh to N=14 baseline (top-down view)', fontsize=12)
ax.set_xlabel('X (COLMAP world)')
```

```
ax.set_ylabel('Z (COLMAP world)')
ax.set_aspect('equal')
fig.tight_layout()
png_path = OUT_DIR / 'per_vertex_distance.png'
plt.savefig(str(png_path), dpi=120, bbox_inches='tight')
plt.close()
print(f'Saved: {png_path}')

print('\n===== STAGE: MESH COMPARISON COMPLETE =====')
```

Listing 10: mesh_compare.py

B.2.5 qualitative_compare.py

```
import numpy as np
import open3d as o3d
import json
import struct
import cv2
from pathlib import Path
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from PIL import Image

SPARSE14 = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense/sparse')
N14_MESH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense/meshed-poisson.ply')
N24_MESH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n24/dense/meshed-poisson.ply')
IMAGES14 = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense/images')
METRICS_JSON = Path('/home/jatin-satyam/ses598-apollo17/comparison/quantitative_metrics.json')
OUT_DIR = Path('/home/jatin-satyam/ses598-apollo17/comparison')

with open(METRICS_JSON) as f:
    metrics = json.load(f)
T_icp = np.array(metrics['icp_transform'])

def read_cameras_bin(path):
    cameras = {}
    nparams = {0:3, 1:4, 2:4, 3:5, 4:8, 5:8, 6:12, 7:5}
    with open(path, 'rb') as f:
        num = struct.unpack('<Q', f.read(8))[0]
        for _ in range(num):
            cid = struct.unpack('<I', f.read(4))[0]
            model = struct.unpack('<I', f.read(4))[0]
            w = struct.unpack('<Q', f.read(8))[0]
            h = struct.unpack('<Q', f.read(8))[0]
            n = nparams.get(model, 4)
            params = list(struct.unpack(f'<{n}d', f.read(8*n)))
            cameras[cid] = {'model': model, 'w': w, 'h': h, 'params': params}
    return cameras

def read_images_bin(path):
    images = {}
    with open(path, 'rb') as f:
        num = struct.unpack('<Q', f.read(8))[0]
        for _ in range(num):
            iid = struct.unpack('<I', f.read(4))[0]
            qvec = np.array(struct.unpack('<4d', f.read(32)))
            tvec = np.array(struct.unpack('<3d', f.read(24)))
            cid = struct.unpack('<I', f.read(4))[0]
            name = b''
            while True:
                c = f.read(1)
                if c == b'\x00': break
                name += c
            name = name.decode()
            n = struct.unpack('<Q', f.read(8))[0]
            xys = np.array(struct.unpack(f'<{2*n}d', f.read(16*n))).reshape(-1,2) if n>0 else np.zeros((0,2))
            pt3d = np.array(struct.unpack(f'<{n}q', f.read(8*n)), dtype=np.int64) if n>0 else np.array([], dtype=np.int64)
            w_q, x_q, y_q, z_q = qvec
            R = np.array([
                [1-2*y_q*y_q-2*z_q*z_q, 2*x_q*y_q-2*z_q*w_q, 2*x_q*z_q+2*y_q*w_q],
                [2*x_q*y_q+2*z_q*w_q, 1-2*x_q*x_q-2*z_q*z_q, 2*y_q*z_q-2*x_q*w_q],
                [2*x_q*z_q-2*y_q*w_q, 2*y_q*z_q+2*x_q*w_q, 1-2*x_q*x_q-2*y_q*y_q]
            ])
            C = -R.T @ tvec
            images[iid] = {'qvec': qvec, 'tvec': tvec, 'cam_id': cid, 'name': name,
                           'R': R, 'C': C, 'xys': xys, 'pt3d': pt3d}
    return images

def read_points3d_bin(path):
    pts = {}
    with open(path, 'rb') as f:
        num = struct.unpack('<Q', f.read(8))[0]
        for _ in range(num):
            pid = struct.unpack('<Q', f.read(8))[0]
            xyz = np.array(struct.unpack('<3d', f.read(24)))
```

```
            rgb = np.array(struct.unpack('<3B', f.read(3)))
            err = struct.unpack('<d', f.read(8))[0]
            n = struct.unpack('<Q', f.read(8))[0]
            f.read(n * 8)
            pts[pid] = (xyz, err)
    return pts

cameras = read_cameras_bin(SPARSE14 / 'cameras.bin')
images = read_images_bin(SPARSE14 / 'images.bin')
pts3d = read_points3d_bin(SPARSE14 / 'points3D.bin')

def compute_reproj_error(img_info):
    cam = cameras[img_info['cam_id']]
    p = cam['params']
    if cam['model'] == 1:
        fx, fy, cx, cy = p[0], p[1], p[2], p[3]
    else:
        fx = fy = p[0]; cx, cy = p[1], p[2]
    R = img_info['R']
    t = img_info['tvec']
    errors = []
    for i, pid in enumerate(img_info['pt3d']):
        if pid < 0 or pid not in pts3d:
            continue
        xyz, _ = pts3d[pid]
        Xc = R @ xyz + t
        if Xc[2] <= 0:
            continue
        u_proj = fx * Xc[0] / Xc[2] + cx
        v_proj = fy * Xc[1] / Xc[2] + cy
        u_obs, v_obs = img_info['xys'][i]
        errors.append(np.sqrt((u_proj - u_obs)**2 + (v_proj - v_obs)**2))
    return float(np.mean(errors)) if errors else float('inf')

img_errors = {}
for iid, img_info in images.items():
    err = compute_reproj_error(img_info)
    img_errors[iid] = err
    print(f'  {img_info["name"]}: reproj_error={err:.3f}px')

sorted_imgs = sorted(img_errors.items(), key=lambda x: x[1])
n_imgs = len(sorted_imgs)
pick_indices = [n_imgs//5, n_imgs*2//5, n_imgs*3//5, n_imgs*4//5]
chosen_ids = [sorted_imgs[i][0] for i in pick_indices]
print(f'\nChosen cameras (near-median reprojection error):')
for iid in chosen_ids:
    print(f'  img {iid}: {images[iid]["name"]}  error={img_errors[iid]:.3f}px')

print('\nLoading meshes...')
mesh14 = o3d.io.read_triangle_mesh(str(N14_MESH))
mesh14.compute_vertex_normals()
mesh24 = o3d.io.read_triangle_mesh(str(N24_MESH))
mesh24.transform(T_icp)
mesh24.compute_vertex_normals()

def render_mesh_matplotlib(mesh, R_cam, t_cam, cam_info, W=800, H=800, title=''):
    verts = np.asarray(mesh.vertices)
    colors_v = np.asarray(mesh.vertex_colors) if mesh.has_vertex_colors() else None

    Xc = (R_cam @ verts.T + t_cam.reshape(3,1)).T
    mask = Xc[:, 2] > 0.1

    p = cam_info['params']
    if cam_info['model'] == 1:
        fx, fy, cx, cy = p[0], p[1], p[2], p[3]
    else:
        fx = fy = p[0]; cx, cy = p[1], p[2]

    scale_x = W / cam_info['w']
    scale_y = H / cam_info['h']

    u = (fx * Xc[:,0] / Xc[:,2].clip(1e-6) + cx) * scale_x
    v = (fy * Xc[:,1] / Xc[:,2].clip(1e-6) + cy) * scale_y

    in_img = mask & (u >= 0) & (u < W) & (v >= 0) & (v < H)

    img = np.ones((H, W, 3), dtype=np.float32) * 0.95

    depths = Xc[:, 2]
    d_min, d_max = depths[in_img].min() if in_img.any() else 1.0, depths[in_img].max() if in_img.any() else 10.0

    valid_u = u[in_img].astype(int).clip(0, W-1)
    valid_v = v[in_img].astype(int).clip(0, H-1)
    valid_d = depths[in_img]

    if colors_v is not None:
        valid_c = colors_v[in_img]
    else:
        norm_d = (valid_d - d_min) / (d_max - d_min + 1e-10)
        valid_c = np.stack([norm_d, norm_d, norm_d], axis=1)

    sort_order = np.argsort(-valid_d)
```

```
    for idx in sort_order:
        uu, vv = valid_u[idx], valid_v[idx]
        img[vv, uu] = valid_c[idx]

    img_uint8 = (img * 255).clip(0, 255).astype(np.uint8)
    return img_uint8

sidebyside_paths = []
for view_idx, iid in enumerate(chosen_ids, start=1):
    img_info = images[iid]
    cam = cameras[img_info['cam_id']]
    R_cam = img_info['R']
    t_cam = img_info['tvec']
    img_name = img_info['name']

    print(f'\nRendering view {view_idx}: {img_name}')

    render14 = render_mesh_matplotlib(mesh14, R_cam, t_cam, cam)
    render24 = render_mesh_matplotlib(mesh24, R_cam, t_cam, cam)

    r14_path = OUT_DIR / f'render_n14_view{view_idx}.png'
    r24_path = OUT_DIR / f'render_n24_view{view_idx}.png'
    Image.fromarray(render14).save(str(r14_path))
    Image.fromarray(render24).save(str(r24_path))
    print(f'  Saved: {r14_path.name}, {r24_path.name}')

    photo_name = img_name
    photo_path = IMAGES14 / photo_name
    has_photo = photo_path.exists()

    if has_photo:
        photo = cv2.imread(str(photo_path))
        photo = cv2.cvtColor(photo, cv2.COLOR_BGR2RGB)
        photo_resized = cv2.resize(photo, (800, 800))
        fig, axes = plt.subplots(1, 3, figsize=(24, 8))
        axes[0].imshow(render14)
        axes[0].set_title(f'N=14 Mesh', fontsize=13, color='steelblue')
        axes[0].axis('off')
        axes[1].imshow(photo_resized)
        axes[1].set_title(f'Original Photo: {Path(photo_name).name}', fontsize=13, color='goldenrod')
        axes[1].axis('off')
        axes[2].imshow(render24)
        axes[2].set_title(f'N=24 Mesh (ICP-aligned)', fontsize=13, color='mediumvioletred')
        axes[2].axis('off')
    else:
        fig, axes = plt.subplots(1, 2, figsize=(16, 8))
        axes[0].imshow(render14)
        axes[0].set_title(f'N=14 Mesh', fontsize=13, color='steelblue')
        axes[0].axis('off')
        axes[1].imshow(render24)
        axes[1].set_title(f'N=24 Mesh (ICP-aligned)', fontsize=13, color='mediumvioletred')
        axes[1].axis('off')

    fig.suptitle(f'View {view_idx}: {Path(photo_name).stem}  reproj_err={img_errors[iid]:.2f}px', fontsize=14)
    fig.tight_layout()
    sbs_path = OUT_DIR / f'sidebyside_view{view_idx}.png'
    plt.savefig(str(sbs_path), dpi=100, bbox_inches='tight')
    plt.close()
    sidebyside_paths.append(sbs_path)
    print(f'  Saved: {sbs_path.name}')

print('\nGenerating qualitative grid (2x2 of side-by-sides)...')
fig, axes = plt.subplots(2, 2, figsize=(28, 18))
for i, sbs_path in enumerate(sidebyside_paths):
    row, col = i // 2, i % 2
    img = np.array(Image.open(sbs_path))
    axes[row, col].imshow(img)
    axes[row, col].set_title(f'View {i+1}', fontsize=12)
    axes[row, col].axis('off')
fig.suptitle('Qualitative Comparison: N=14 vs N=24 Mesh (4 Viewpoints)', fontsize=16, fontweight='bold')
plt.tight_layout()
grid_path = OUT_DIR / 'qualitative_grid.png'
plt.savefig(str(grid_path), dpi=80, bbox_inches='tight')
plt.close()
print(f'Saved: {grid_path}')
print('\n===== STAGE: QUALITATIVE COMPARISON COMPLETE =====')
```

Listing 11: qualitative_compare.py

B.2.6 `recover_21036.py`

```
import sqlite3
import numpy as np
import cv2
import struct
import shutil
from pathlib import Path
```

```
7
8  DB_PATH = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/database.db')
9  SPARSE_IN = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/sparse/0')
10 SPARSE_OUT = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/sparse/0_with21036')
11 SPARSE_OUT.mkdir(parents=True, exist_ok=True)
12
13 IMG36_ID = 14
14 IMG37_ID = 15
15
16 db = sqlite3.connect(str(DB_PATH))
17 cur = db.cursor()
18
19 def read_keypoints(image_id):
20     cur.execute('SELECT rows, cols, data FROM keypoints WHERE image_id=?', (image_id,))
21     row = cur.fetchone()
22     n, cols, data = row
23     kps = np.frombuffer(data, dtype=np.float32).reshape(n, cols)
24     return kps[:, :2]
25
26 def read_matches(id1, id2):
27     pair_id = id1 * 2147483647 + id2
28     cur.execute('SELECT rows, cols, data, H FROM two_view_geometries WHERE pair_id=?', (pair_id,))
29     row = cur.fetchone()
30     n_rows, n_cols, data, H_bytes = row
31     matches = np.frombuffer(data, dtype=np.uint32).reshape(n_rows, 2)
32     H = np.frombuffer(H_bytes, dtype=np.float64).reshape(3, 3) if H_bytes else None
33     return matches, H
34
35 def read_cameras_bin(path):
36     cameras = {}
37     model_nparams = {0:3, 1:4, 2:4, 3:5, 4:8, 5:8, 6:12, 7:5}
38     with open(path, 'rb') as f:
39         num = struct.unpack('<Q', f.read(8))[0]
40         for _ in range(num):
41             cid = struct.unpack('<I', f.read(4))[0]
42             model = struct.unpack('<I', f.read(4))[0]
43             w = struct.unpack('<Q', f.read(8))[0]
44             h = struct.unpack('<Q', f.read(8))[0]
45             np_ = model_nparams.get(model, 4)
46             params = list(struct.unpack(f'<{np_}d', f.read(8*np_)))
47             cameras[cid] = {'model': model, 'w': w, 'h': h, 'params': params}
48     return cameras
49
50 def camera_K_dist(cam):
51     model = cam['model']
52     p = cam['params']
53     if model == 0:
54         f, cx, cy = p[0], p[1], p[2]
55         K = np.array([[f,0,cx],[0,f,cy],[0,0,1]])
56         dist = np.zeros(5)
57     elif model == 1:
58         fx, fy, cx, cy = p[0], p[1], p[2], p[3]
59         K = np.array([[fx,0,cx],[0,fy,cy],[0,0,1]])
60         dist = np.zeros(5)
61     elif model == 2:
62         f, cx, cy, k1 = p[0], p[1], p[2], p[3]
63         K = np.array([[f,0,cx],[0,f,cy],[0,0,1]])
64         dist = np.array([k1, 0, 0, 0, 0])
65     elif model == 3:
66         f, cx, cy, k1, k2 = p[0], p[1], p[2], p[3], p[4]
67         K = np.array([[f,0,cx],[0,f,cy],[0,0,1]])
68         dist = np.array([k1, k2, 0, 0, 0])
69     else:
70         fx, fy, cx, cy = p[0], p[1], p[2], p[3]
71         K = np.array([[fx,0,cx],[0,fy,cy],[0,0,1]])
72         dist = np.array(p[4:8] + [0]*(8-len(p[4:8])))[:5] if len(p)>4 else np.zeros(5)
73     return K.astype(np.float64), dist.astype(np.float64)
74
75 def read_images_bin(path):
76     images = {}
77     model_nparams_kp = {0:3, 1:4, 2:4, 3:5, 4:8, 5:8, 6:12, 7:5}
78     with open(path, 'rb') as f:
79         num = struct.unpack('<Q', f.read(8))[0]
80         for _ in range(num):
81             iid = struct.unpack('<I', f.read(4))[0]
82             qvec = np.array(struct.unpack('<4d', f.read(32)))
83             tvec = np.array(struct.unpack('<3d', f.read(24)))
84             cid = struct.unpack('<I', f.read(4))[0]
85             name = b''
86             while True:
87                 c = f.read(1)
88                 if c == b'\x00': break
89                 name += c
90             name = name.decode()
91             n = struct.unpack('<Q', f.read(8))[0]
92             xys = np.array(struct.unpack(f'<{2*n}d', f.read(16*n))).reshape(-1,2) if n>0 else np.zeros((0,2))
93             pt3d = np.array(struct.unpack(f'<{n}q', f.read(8*n)), dtype=np.int64) if n>0 else np.array([], dtype=np.int64)
94             w,x,y,z = qvec
95             R = np.array([[1-2*y*y-2*z*z, 2*x*y-2*z*w, 2*x*z+2*y*w],[2*x*y+2*z*w, 1-2*x*x-2*z*z, 2*y*z-2*x*w],[2*x*z-2*y*w, 2*y*z+2*x*w, 1-2*x*x-2*
       y*y]])
96             images[iid] = {'qvec': qvec, 'tvec': tvec, 'cam_id': cid, 'name': name, 'R': R, 'xys': xys, 'pt3d': pt3d}
97     return images
```

```
98
99  def qvec_from_R(R):
100     trace = R[0,0] + R[1,1] + R[2,2]
101     if trace > 0:
102         s = 0.5 / np.sqrt(trace + 1.0)
103         w = 0.25 / s
104         x = (R[2,1] - R[1,2]) * s
105         y = (R[0,2] - R[2,0]) * s
106         z = (R[1,0] - R[0,1]) * s
107     elif R[0,0] > R[1,1] and R[0,0] > R[2,2]:
108         s = 2.0 * np.sqrt(1.0 + R[0,0] - R[1,1] - R[2,2])
109         w = (R[2,1] - R[1,2]) / s
110         x = 0.25 * s
111         y = (R[0,1] + R[1,0]) / s
112         z = (R[0,2] + R[2,0]) / s
113     elif R[1,1] > R[2,2]:
114         s = 2.0 * np.sqrt(1.0 + R[1,1] - R[0,0] - R[2,2])
115         w = (R[0,2] - R[2,0]) / s
116         x = (R[0,1] + R[1,0]) / s
117         y = 0.25 * s
118         z = (R[1,2] + R[2,1]) / s
119     else:
120         s = 2.0 * np.sqrt(1.0 + R[2,2] - R[0,0] - R[1,1])
121         w = (R[1,0] - R[0,1]) / s
122         x = (R[0,2] + R[2,0]) / s
123         y = (R[1,2] + R[2,1]) / s
124         z = 0.25 * s
125     return np.array([w, x, y, z])
126
127 kps36 = read_keypoints(IMG36_ID)
128 kps37 = read_keypoints(IMG37_ID)
129 matches, H_colmap = read_matches(IMG36_ID, IMG37_ID)
130
131 pts36 = kps36[matches[:, 0]].astype(np.float64)
132 pts37 = kps37[matches[:, 1]].astype(np.float64)
133
134 cameras_orig = read_cameras_bin(SPARSE_IN / 'cameras.bin')
135 images_orig = read_images_bin(SPARSE_IN / 'images.bin')
136
137 img37 = images_orig[15]
138 R37 = img37['R']
139 t37 = img37['tvec']
140 cam_id37 = img37['cam_id']
141 K37, dist37 = camera_K_dist(cameras_orig[cam_id37])
142 K36, dist36 = camera_K_dist(cameras_orig[2])
143
144 print(f'K36: f={K36[0,0]:.2f} cx={K36[0,2]:.2f} cy={K36[1,2]:.2f}')
145 print(f'K37: f={K37[0,0]:.2f} cx={K37[0,2]:.2f} cy={K37[1,2]:.2f}')
146
147 print(f'\nHcolmap:\n{H_colmap}')
148 print(f'\nDecomposing homography...')
149 retval, rotations, translations, normals = cv2.decomposeHomographyMat(H_colmap, K36)
150 print(f'{retval} solutions found')
151
152 pts36_u = cv2.undistortPoints(pts36.reshape(-1,1,2), K36, dist36).reshape(-1,2)
153 pts37_u = cv2.undistortPoints(pts37.reshape(-1,1,2), K37, dist37).reshape(-1,2)
154
155 best_sol = None
156 best_inliers = -1
157
158 for i, (R_rel, t_rel, n_vec) in enumerate(zip(rotations, translations, normals)):
159     t_r = t_rel.flatten()
160     P1 = np.hstack([np.eye(3), np.zeros((3,1))])
161     P2 = np.hstack([R_rel, t_r.reshape(3,1)])
162     pts_in_front = 0
163     sample = min(100, len(pts36_u))
164     for pt1, pt2 in zip(pts36_u[:sample], pts37_u[:sample]):
165         A = np.array([
166             pt1[0]*P1[2] - P1[0],
167             pt1[1]*P1[2] - P1[1],
168             pt2[0]*P2[2] - P2[0],
169             pt2[1]*P2[2] - P2[1]
170         ])
171         _, _, Vt = np.linalg.svd(A)
172         X = Vt[-1]
173         X = X / X[3]
174         z1 = X[2]
175         z2 = (R_rel @ X[:3] + t_r)[2]
176         if z1 > 0 and z2 > 0:
177             pts_in_front += 1
178     print(f'  sol {i}: t_rel={np.round(t_r,3)} pts_in_front={pts_in_front}/{sample}')
179     if pts_in_front > best_inliers:
180         best_inliers = pts_in_front
181         best_sol = (R_rel, t_r)
182
183 db.close()
184
185 if best_sol is None:
186     print('No valid solution found - giving up on 21036')
187     exit(1)
188
189 R_rel, t_rel = best_sol
```

```
print(f'\nSelected: pts_in_front={best_inliers}')

R36_abs = R37 @ R_rel.T
C36 = -R37.T @ t37 + R_rel.T @ t_rel
t36_abs = -R36_abs @ (-R37.T @ t37 - R_rel.T @ t_rel)

t36_abs2 = t37 + R37 @ (-R_rel.T @ t_rel)

print(f't36_abs from 21037 pose: {np.round(t36_abs,4)}')
print(f't36_abs alternative:     {np.round(t36_abs2,4)}')
print(f't37 for reference:       {np.round(t37,4)}')

C37 = -R37.T @ t37
C36_calc = C37 + R37.T @ (R_rel.T @ t_rel)
t36_final = -R36_abs @ C36_calc

print(f'C37: {np.round(C37,4)}')
print(f'C36: {np.round(C36_calc,4)}')
print(f't36_final: {np.round(t36_final,4)}')

qvec36 = qvec_from_R(R36_abs)
print(f'qvec36: {np.round(qvec36,6)}')

shutil.copy(SPARSE_IN / 'cameras.bin', SPARSE_OUT / 'cameras.bin')
shutil.copy(SPARSE_IN / 'points3D.bin', SPARSE_OUT / 'points3D.bin')

with open(SPARSE_IN / 'images.bin', 'rb') as fin, open(SPARSE_OUT / 'images.bin', 'wb') as fout:
    num_orig = struct.unpack('<Q', fin.read(8))[0]
    fout.write(struct.pack('<Q', num_orig + 1))
    fout.write(fin.read())

    new_img_id = 16
    fout.write(struct.pack('<I', new_img_id))
    fout.write(struct.pack('<4d', *qvec36))
    fout.write(struct.pack('<3d', *t36_final))
    fout.write(struct.pack('<I', 2))
    fout.write(b'mag138/AS17-138-21036HR.png\x00')

    n_kp = len(kps36)
    fout.write(struct.pack('<Q', n_kp))
    for kp in kps36:
        fout.write(struct.pack('<2d', float(kp[0]), float(kp[1])))
        fout.write(struct.pack('<q', -1))

print(f'\nWrote extended sparse model to {SPARSE_OUT}')
print(f'21036 pose: C=[{C36_calc[0]:.3f},{C36_calc[1]:.3f},{C36_calc[2]:.3f}]')
```

Listing 12: recover_21036.py

B.3 Group C: Novel-Pose Synthesis and Rendering

B.3.1 `diagnose_novel.py`

```
from pathlib import Path
import numpy as np
import json
import struct
import collections
import subprocess

base = Path.home() / "ses598-apollo17"
sparse_dir = base / "colmap_n15" / "sparse" / "0"

CameraTuple = collections.namedtuple("CameraTuple", ["id", "model", "width", "height", "params"])
ImageTuple = collections.namedtuple("ImageTuple", ["id", "qvec", "tvec", "camera_id", "name"])

def read_next_bytes(fid, num_bytes, format_char_sequence, endian="<"):
    data = fid.read(num_bytes)
    return struct.unpack(endian + format_char_sequence, data)

def read_cameras_binary(path):
    cameras = {}
    with open(path, "rb") as fid:
        num_cameras = read_next_bytes(fid, 8, "Q")[0]
        for _ in range(num_cameras):
            cam_props = read_next_bytes(fid, 24, "iiQQ")
            camera_id = cam_props[0]
            model_id = cam_props[1]
            width = cam_props[2]
            height = cam_props[3]
            num_params = {0: 3, 1: 4, 2: 4, 3: 5, 4: 8, 5: 8, 6: 12, 7: 5, 8: 4, 9: 5, 10: 12}[model_id]
            params = read_next_bytes(fid, 8 * num_params, "d" * num_params)
            cameras[camera_id] = CameraTuple(camera_id, model_id, width, height, params)
    return cameras

def read_images_binary(path):
```

```
    images = {}
    with open(path, "rb") as fid:
        num_images = read_next_bytes(fid, 8, "Q")[0]
        for _ in range(num_images):
            image_props = read_next_bytes(fid, 64, "idddddddi")
            image_id = image_props[0]
            qvec = np.array(image_props[1:5])
            tvec = np.array(image_props[5:8])
            camera_id = image_props[8]
            name = ""
            char = read_next_bytes(fid, 1, "c")[0]
            while char != b"\x00":
                name += char.decode("utf-8")
                char = read_next_bytes(fid, 1, "c")[0]
            num_points2d = read_next_bytes(fid, 8, "Q")[0]
            read_next_bytes(fid, 24 * num_points2d, "ddq" * num_points2d)
            images[image_id] = ImageTuple(image_id, qvec, tvec, camera_id, name)
    return images

def qvec_to_rotmat(qvec):
    w, x, y, z = qvec
    return np.array([
        [1 - 2*y*y - 2*z*z, 2*x*y - 2*z*w, 2*x*z + 2*y*w],
        [2*x*y + 2*z*w, 1 - 2*x*x - 2*z*z, 2*y*z - 2*x*w],
        [2*x*z - 2*y*w, 2*y*z + 2*x*w, 1 - 2*x*x - 2*y*y]])

cameras = read_cameras_binary(sparse_dir / "cameras.bin")
images = read_images_binary(sparse_dir / "images.bin")

print(f"Cameras: {len(cameras)}")
print(f"Images: {len(images)}")
print()

centers = []
forwards = []
for img_id, img in sorted(images.items(), key=lambda kv: kv[1].name):
    R = qvec_to_rotmat(img.qvec)
    t = img.tvec
    center = -R.T @ t
    forward = R.T @ np.array([0, 0, 1])
    centers.append(center)
    forwards.append(forward)
    print(f"{img.name:40s}  center=({center[0]:7.3f},{center[1]:7.3f},{center[2]:7.3f})  cam_id={img.camera_id}")

centers = np.array(centers)
forwards = np.array(forwards)

print()
print("=== Pose cloud statistics ===")
print(f"Centroid:        {centers.mean(axis=0)}")
print(f"Bounding box:    min={centers.min(axis=0)}, max={centers.max(axis=0)}")
print(f"Spread (std):    {centers.std(axis=0)}")
print(f"Mean pairwise distance: {np.mean([np.linalg.norm(centers[i]-centers[j]) for i in range(len(centers)) for j in range(i+1,len(centers))]):.3f
      }")

mag137_mask = ["mag137" in img.name for img in sorted(images.values(), key=lambda i: i.name)]
mag138_mask = ["mag138" in img.name for img in sorted(images.values(), key=lambda i: i.name)]

mag137_centers = centers[mag137_mask]
mag138_centers = centers[mag138_mask]

print()
print(f"mag137 centroid: {mag137_centers.mean(axis=0)}")
print(f"mag138 centroid: {mag138_centers.mean(axis=0)}")
print(f"Inter-cluster distance: {np.linalg.norm(mag137_centers.mean(axis=0) - mag138_centers.mean(axis=0)):.3f}")
print(f"mag137 intra-cluster mean dist: {np.mean([np.linalg.norm(mag137_centers[i]-mag137_centers[j]) for i in range(len(mag137_centers)) for j in
      range(i+1,len(mag137_centers))]):.3f}")
print(f"mag138 intra-cluster mean dist: {np.mean([np.linalg.norm(mag138_centers[i]-mag138_centers[j]) for i in range(len(mag138_centers)) for j in
      range(i+1,len(mag138_centers))]):.3f}")

mean_forward = forwards.mean(axis=0)
mean_forward = mean_forward / np.linalg.norm(mean_forward)
look_at_target = centers.mean(axis=0) + mean_forward * np.linalg.norm(centers.std(axis=0)) * 2
print(f"\nEstimated scene look-at target: {look_at_target}")
```

Listing 13: diagnose_novel.py

B.3.2 gen_novel_poses.py

```
import numpy as np
import struct
import json
from pathlib import Path

SPARSE = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/sparse/0')
SPLAT_DIR = Path('/home/jatin-satyam/ses598-apollo17/splats_n15/apollo17_n15/splatfacto/2026-04-25_144254')
OUT_DIR = Path('/home/jatin-satyam/ses598-apollo17/novel_poses')
```

```
OUT_DIR.mkdir(parents=True, exist_ok=True)

with open(SPLAT_DIR / 'dataparser_transforms.json') as f:
    dp = json.load(f)
T34 = np.array(dp['transform'], dtype=np.float64)
scale = float(dp['scale'])
print('Transform T34:')
print(T34)
print(f'Scale: {scale}')

def read_cameras(path):
    cameras = {}
    with open(path, 'rb') as f:
        n = struct.unpack('Q', f.read(8))[0]
        for _ in range(n):
            cid = struct.unpack('I', f.read(4))[0]
            mid = struct.unpack('i', f.read(4))[0]
            w = struct.unpack('Q', f.read(8))[0]
            h = struct.unpack('Q', f.read(8))[0]
            nparams = {0:3,1:4,2:4,3:5,4:8}[mid]
            params = struct.unpack(f'{nparams}d', f.read(8*nparams))
            cameras[cid] = {'model_id':mid,'w':w,'h':h,'params':params}
    return cameras

def read_images(path):
    images = {}
    with open(path, 'rb') as f:
        n = struct.unpack('Q', f.read(8))[0]
        for _ in range(n):
            iid = struct.unpack('I', f.read(4))[0]
            qw,qx,qy,qz = struct.unpack('4d', f.read(32))
            tx,ty,tz = struct.unpack('3d', f.read(24))
            cid = struct.unpack('I', f.read(4))[0]
            name = b''
            while True:
                c = f.read(1)
                if c == b'\x00': break
                name += c
            npts = struct.unpack('Q', f.read(8))[0]
            f.read(npts*24)
            images[iid] = {'qvec':(qw,qx,qy,qz),'tvec':(tx,ty,tz),'cam_id':cid,'name':name.decode()}
    return images

def qvec_to_R(qvec):
    qw,qx,qy,qz = qvec
    return np.array([
        [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
        [2*(qx*qy+qz*qw),   1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
        [2*(qx*qz-qy*qw),   2*(qy*qz+qx*qw),   1-2*(qx**2+qy**2)]
    ])

def read_images_with_pts(path):
    images = {}
    with open(path, 'rb') as f:
        n = struct.unpack('Q', f.read(8))[0]
        for _ in range(n):
            iid = struct.unpack('I', f.read(4))[0]
            qw,qx,qy,qz = struct.unpack('4d', f.read(32))
            tx,ty,tz = struct.unpack('3d', f.read(24))
            cid = struct.unpack('I', f.read(4))[0]
            name = b''
            while True:
                c = f.read(1)
                if c == b'\x00': break
                name += c
            npts = struct.unpack('Q', f.read(8))[0]
            pt_ids = []
            for _ in range(npts):
                struct.unpack('2d', f.read(16))
                pid = struct.unpack('Q', f.read(8))[0]
                if pid != 2**64-1:
                    pt_ids.append(pid)
            images[iid] = {'name': name.decode(), 'cam_id': cid,
                           'qvec': (qw,qx,qy,qz), 'tvec': (tx,ty,tz),
                           'pt3d_ids': set(pt_ids)}
    return images

def read_all_pts(path, filter_box):
    pts = {}
    x0,x1,y0,y1,z0,z1 = filter_box
    with open(path, 'rb') as f:
        n = struct.unpack('Q', f.read(8))[0]
        for _ in range(n):
            pid = struct.unpack('Q', f.read(8))[0]
            x,y,z = struct.unpack('3d', f.read(24))
            f.read(3); f.read(8)
            ntrack = struct.unpack('Q', f.read(8))[0]
            f.read(ntrack*8)
            if x0<=x<=x1 and y0<=y<=y1 and z0<=z<=z1:
                pts[pid] = np.array([x,y,z])
    return pts

```

```
bbox = (-7.5, 1.0, -2.5, 4.5, 0.5, 11.0)
all_pts = read_all_pts(SPARSE / 'points3D.bin', bbox)

cameras = read_cameras(SPARSE / 'cameras.bin')
images = read_images_with_pts(SPARSE / 'images.bin')

mag137_imgs = [img for img in images.values() if 'mag137' in img['name']]
mag138_imgs = [img for img in images.values() if 'mag138' in img['name']]

def rock_centroid(imgs):
    ptids = set()
    for img in imgs:
        ptids.update(img['pt3d_ids'])
    pts = np.array([all_pts[pid] for pid in ptids if pid in all_pts])
    return np.median(pts, axis=0) if len(pts) > 0 else None

T_lookat137 = rock_centroid(mag137_imgs)
T_lookat138 = rock_centroid(mag138_imgs)
print(f'Rock centroid mag137: {T_lookat137.round(4)}')
print(f'Rock centroid mag138: {T_lookat138.round(4)}')

def cluster_geometry(imgs):
    centers, forwards = [], []
    for img in imgs:
        R = qvec_to_R(img['qvec'])
        t = np.array(img['tvec'])
        C = -R.T @ t
        fwd = R.T[:, 2]
        centers.append(C)
        forwards.append(fwd)
    centers = np.array(centers)
    forwards = np.array(forwards)
    centroid = centers.mean(0)
    radii = np.linalg.norm(centers - centroid, axis=1)
    radius = float(np.median(radii))
    mean_fwd = forwards.mean(0)
    mean_fwd = mean_fwd / np.linalg.norm(mean_fwd)
    return centroid, radius, mean_fwd, centers

C137, r137, F137, centers137 = cluster_geometry(mag137_imgs)
C138, r138, F138, centers138 = cluster_geometry(mag138_imgs)

T_formula137 = C137 + F137 * r137 * 1.5
T_formula138 = C138 + F138 * r138 * 1.5

print(f'\nmag137: C={C137.round(4)}  r={r137:.4f}  F={F137.round(4)}')
print(f'  lookat_target(formula)={T_formula137.round(4)}  lookat_target(actual)={T_lookat137.round(4)}')
print(f'mag138: C={C138.round(4)}  r={r138:.4f}  F={F138.round(4)}')
print(f'  lookat_target(formula)={T_formula138.round(4)}  lookat_target(actual)={T_lookat138.round(4)}')

def lookat_colmap(eye, target):
    world_up = np.array([0., 1., 0.])
    Z = target - eye
    Z = Z / np.linalg.norm(Z)
    X = np.cross(world_up, Z)
    X = X / np.linalg.norm(X)
    Y = np.cross(X, Z)
    c2w = np.eye(4)
    c2w[:3, 0] = X
    c2w[:3, 1] = Y
    c2w[:3, 2] = Z
    c2w[:3, 3] = eye
    return c2w

col_flip = np.diag([1., -1., -1., 1.])

def to_nerfstudio(c2w_44):
    c2w_34 = (T34 @ (c2w_44 @ col_flip)).copy()
    c2w_34[:, 3] *= scale
    return np.vstack([c2w_34, [0., 0., 0., 1.]])

def arc_poses(C, r, F, T_lookat, n=5, span_deg=30.0):
    F_xz = np.array([F[0], 0., F[2]])
    F_xz = F_xz / np.linalg.norm(F_xz)
    P_xz = np.array([-F_xz[2], 0., F_xz[0]])
    angles = np.linspace(-span_deg, span_deg, n)
    results = []
    for a in angles:
        th = np.radians(a)
        d = np.cos(th) * F_xz + np.sin(th) * P_xz
        eye = C + r * d
        eye[1] = C[1]
        c2w_colmap = lookat_colmap(eye, T_lookat)
        c2w_ns = to_nerfstudio(c2w_colmap)
        results.append({
            'angle_deg': float(a),
            'eye_colmap': eye.tolist(),
            'c2w_colmap': c2w_colmap.tolist(),
            'c2w_ns': c2w_ns.tolist(),
        })
    return results

```

```
poses137 = arc_poses(C137, r137, F137, T_formula137, n=5, span_deg=30.0)
poses138 = arc_poses(C138, r138, F138, T_formula138, n=5, span_deg=30.0)

cam1 = cameras[1]
f_train = cam1['params'][0] / 2.0
h_train = cam1['h'] / 2.0
vfov_deg = float(2.0 * np.degrees(np.arctan(h_train / 2.0 / f_train)))
print(f'\nVertical FOV: {vfov_deg:.4f} degrees  (f_train={f_train:.2f}, h_train={h_train:.0f})')

RENDER_W, RENDER_H = 2000, 2000

camera_path_json = {
    'camera_type': 'perspective',
    'render_height': RENDER_H,
    'render_width': RENDER_W,
    'fps': 1,
    'seconds': float(10),
    'camera_path': [],
}

all_entries = [(p, 'mag137', i+1) for i, p in enumerate(poses137)] + \
              [(p, 'mag138', i+1) for i, p in enumerate(poses138)]

for p, cluster, idx in all_entries:
    camera_path_json['camera_path'].append({
        'camera_to_world': np.array(p['c2w_ns']).flatten().tolist(),
        'fov': vfov_deg,
        'aspect': float(RENDER_W) / float(RENDER_H),
    })

cam_path_file = OUT_DIR / 'novel_camera_path.json'
with open(cam_path_file, 'w') as f:
    json.dump(camera_path_json, f, indent=2)
print(f'\nCamera path saved: {cam_path_file} ({len(camera_path_json["camera_path"])} poses)')

print('\nPose definitions:')
for p, cluster, idx in all_entries:
    eye = np.array(p['eye_colmap'])
    c_ns = np.array(p['c2w_ns'])[:3, 3]
    print(f'  {cluster}_{idx:02d}  angle={p["angle_deg"]:+6.1f}  eye_colmap={eye.round(3)}  t_ns={c_ns.round(4)}')
```

Listing 14: gen_novel_poses.py

B.3.3 make_contact_sheet.py

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from PIL import Image
from pathlib import Path

OUT_DIR = Path('/home/jatin-satyam/ses598-apollo17/novel_poses')

frames = [
    ('novel_mag137_01.png', 'mag137_01  -30 deg'),
    ('novel_mag137_02.png', 'mag137_02  -15 deg'),
    ('novel_mag137_03.png', 'mag137_03    0 deg'),
    ('novel_mag137_04.png', 'mag137_04  +15 deg'),
    ('novel_mag137_05.png', 'mag137_05  +30 deg'),
    ('novel_mag138_01.png', 'mag138_01  -30 deg'),
    ('novel_mag138_02.png', 'mag138_02  -15 deg'),
    ('novel_mag138_03.png', 'mag138_03    0 deg'),
    ('novel_mag138_04.png', 'mag138_04  +15 deg'),
    ('novel_mag138_05.png', 'mag138_05  +30 deg'),
]

fig, axes = plt.subplots(2, 5, figsize=(22, 9))
fig.suptitle('Novel View Synthesis: 10 Rendered Poses (5 per Magazine Cluster)', fontsize=14, fontweight='bold')

for idx, (fname, label) in enumerate(frames):
    row = idx // 5
    col = idx % 5
    ax = axes[row, col]
    img = np.array(Image.open(OUT_DIR / fname).convert('RGB'))
    thumb_h = 400
    scale = thumb_h / img.shape[0]
    thumb_w = int(img.shape[1] * scale)
    from PIL import Image as PILImage
    thumb = np.array(PILImage.fromarray(img).resize((thumb_w, thumb_h), PILImage.LANCZOS))
    ax.imshow(thumb)
    ax.set_title(label, fontsize=10, color='steelblue' if 'mag137' in fname else 'mediumvioletred')
    ax.axis('off')

plt.tight_layout()
out = OUT_DIR / 'contact_sheet.png'
plt.savefig(str(out), dpi=120, bbox_inches='tight')
```

```
plt.close()
print(f'Saved: {out}')
```

Listing 15: make_contact_sheet.py

B.3.4 novel_pose_smoke.py

```
import json
import numpy as np
import torch
import struct
from pathlib import Path
import cv2

SPLAT_DIR = Path('/home/jatin-satyam/ses598-apollo17/splats_n15/apollo17_n15/splatfacto/2026-04-25_144254')
SPARSE_DIR = Path('/home/jatin-satyam/ses598-apollo17/colmap_n15/dense/sparse')
OUT_DIR = Path('/home/jatin-satyam/ses598-apollo17/metrics_n15')

# Read dataparser transforms to understand coordinate mapping
transforms_path = SPLAT_DIR / 'dataparser_transforms.json'
with open(transforms_path) as f:
    dp = json.load(f)
print('Dataparser transforms:')
print(json.dumps(dp, indent=2))

# Interpolate between camera 0 and camera 6 (first and last mag137 views) at t=0.5
# We'll do this by reading the transforms.json that nerfstudio produces
# Actually nerfstudio COLMAP dataparser uses the sparse model directly.
# The dataparser_transforms.json has: transform (4x4) and scale

# Read COLMAP cameras from dense/sparse (PINHOLE format)
CAMERAS_PATH = SPARSE_DIR / 'cameras.bin'
IMAGES_PATH = SPARSE_DIR / 'images.bin'

def read_cameras_bin(path):
    cameras = {}
    with open(path, 'rb') as f:
        n = struct.unpack('Q', f.read(8))[0]
        for _ in range(n):
            cam_id = struct.unpack('I', f.read(4))[0]
            model_id = struct.unpack('i', f.read(4))[0]
            w = struct.unpack('Q', f.read(8))[0]
            h = struct.unpack('Q', f.read(8))[0]
            nparams = {0:3, 1:4, 2:4, 3:5, 4:8}[model_id]
            params = struct.unpack(f'{nparams}d', f.read(8*nparams))
            cameras[cam_id] = {'model_id': model_id, 'w': w, 'h': h, 'params': params}
    return cameras

def read_images_bin(path):
    images = {}
    with open(path, 'rb') as f:
        n = struct.unpack('Q', f.read(8))[0]
        for _ in range(n):
            img_id = struct.unpack('I', f.read(4))[0]
            qw, qx, qy, qz = struct.unpack('4d', f.read(32))
            tx, ty, tz = struct.unpack('3d', f.read(24))
            cam_id = struct.unpack('I', f.read(4))[0]
            name = b''
            while True:
                c = f.read(1)
                if c == b'\x00':
                    break
                name += c
            n_pts = struct.unpack('Q', f.read(8))[0]
            f.read(n_pts * 24)
            images[img_id] = {
                'qvec': (qw, qx, qy, qz),
                'tvec': (tx, ty, tz),
                'cam_id': cam_id,
                'name': name.decode(),
            }
    return images

cameras = read_cameras_bin(CAMERAS_PATH)
images = read_images_bin(IMAGES_PATH)

def qvec_to_R(qvec):
    qw, qx, qy, qz = qvec
    R = np.array([
        [1-2*(qy**2+qz**2),   2*(qx*qy-qz*qw),   2*(qx*qz+qy*qw)],
        [2*(qx*qy+qz*qw),   1-2*(qx**2+qz**2),   2*(qy*qz-qx*qw)],
        [2*(qx*qz-qy*qw),     2*(qy*qz+qx*qw), 1-2*(qx**2+qy**2)],
    ])
    return R

def slerp(q0, q1, t):
    q0 = np.array(q0); q1 = np.array(q1)
```

```
    dot = np.dot(q0, q1)
    if dot < 0:
        q1 = -q1; dot = -dot
    dot = min(dot, 1.0)
    theta = np.arccos(dot)
    if theta < 1e-8:
        return q0
    return (np.sin((1-t)*theta) * q0 + np.sin(t*theta) * q1) / np.sin(theta)

img_list = sorted(images.values(), key=lambda x: x['name'])
mag137 = [img for img in img_list if 'mag137' in img['name']]
i0, i1 = mag137[0], mag137[-1]
print(f'Interpolating between {i0["name"]} and {i1["name"]}')

t = 0.5
q_interp = slerp(i0['qvec'], i1['qvec'], t)
t_interp = tuple((1-t)*np.array(i0['tvec']) + t*np.array(i1['tvec']))

R = qvec_to_R(q_interp)
t_vec = np.array(t_interp)
c2w = np.eye(4)
c2w[:3, :3] = R.T
c2w[:3, 3] = -R.T @ t_vec

cam = cameras[i0['cam_id']]
fx, fy, cx, cy = cam['params']
w, h = cam['w'], cam['h']

# Apply dataparser transform
T = np.array(dp['transform'])
scale = dp['scale']
c2w_ns_34 = T @ c2w
c2w_ns_34[:3, 3] *= scale
c2w_ns = np.vstack([c2w_ns_34, [0, 0, 0, 1]])

print(f'Interpolated camera center (nerfstudio coords): {c2w_ns[:3, 3]}')
print(f'fx={fx:.1f} fy={fy:.1f} cx={cx:.1f} cy={cy:.1f}  {w}x{h}')

# Write a cameras_path JSON for ns-render camera-path
# nerfstudio camera_path format
camera_path = {
    'camera_type': 'perspective',
    'render_height': h,
    'render_width': w,
    'fps': 1,
    'seconds': 1.0,
    'camera_path': [{
        'camera_to_world': c2w_ns.flatten().tolist(),
        'fov': float(2 * np.degrees(np.arctan(0.5 * h / fy))),
        'aspect': float(w / h),
    }],
}

cam_path_file = OUT_DIR / 'novel_camera_path.json'
with open(cam_path_file, 'w') as f:
    json.dump(camera_path, f, indent=2)
print(f'Saved camera path: {cam_path_file}')
```

Listing 16: novel_pose_smoke.py

B.3.5 sanity_render.py

```
import numpy as np
import open3d as o3d
import struct
import os
os.environ['PYOPENGL_PLATFORM'] = 'egl'
import pyrender
import trimesh
from pathlib import Path
from PIL import Image

def read_images_bin(path):
    images = {}
    with open(path, 'rb') as f:
        n = struct.unpack('<Q', f.read(8))[0]
        for _ in range(n):
            iid = struct.unpack('<I', f.read(4))[0]
            qvec = np.array(struct.unpack('<4d', f.read(32)))
            tvec = np.array(struct.unpack('<3d', f.read(24)))
            cid = struct.unpack('<I', f.read(4))[0]
            name = b''
            while True:
                c = f.read(1)
                if c == b'\x00': break
                name += c
            n_pts = struct.unpack('<Q', f.read(8))[0]
```

```
            f.read(n_pts * 24)
            qw, qx, qy, qz = qvec
            R = np.array([
                [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
                [2*(qx*qy+qz*qw), 1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
                [2*(qx*qz-qy*qw), 2*(qy*qz+qx*qw), 1-2*(qx**2+qy**2)]
            ])
            images[iid] = dict(name=name.decode(), cam_id=cid, R=R, tvec=tvec)
    return images

def read_cameras_bin(path):
    cameras = {}
    nparams = {0:3,1:4,2:4,3:5,4:8}
    with open(path, 'rb') as f:
        n = struct.unpack('<Q', f.read(8))[0]
        for _ in range(n):
            cid = struct.unpack('<I', f.read(4))[0]
            mid = struct.unpack('<I', f.read(4))[0]
            w = struct.unpack('<Q', f.read(8))[0]
            h = struct.unpack('<Q', f.read(8))[0]
            np_ = nparams[mid]
            params = list(struct.unpack(f'<{np_}d', f.read(8*np_)))
            cameras[cid] = dict(model=mid, W=w, H=h, params=params)
    return cameras

def colmap_pose_to_c2w(R, tvec):
    c2w = np.eye(4)
    c2w[:3, :3] = R.T
    c2w[:3, 3] = -R.T @ tvec
    return c2w

def render_vertex_colored_ply(ply_path, sparse_dir, img_name, out_path, W=800, H=800):
    sparse_dir = Path(sparse_dir)
    cams = read_cameras_bin(sparse_dir / 'cameras.bin')
    imgs = read_images_bin(sparse_dir / 'images.bin')

    chosen = next(img for img in imgs.values() if img['name'] == img_name)
    cam_data = cams[chosen['cam_id']]
    p = cam_data['params']
    m = cam_data['model']
    if m == 1:
        fx, fy, cx, cy = p[0], p[1], p[2], p[3]
    elif m == 2:
        fx = fy = p[0]; cx = p[1]; cy = p[2]
    else:
        fx = fy = p[0]; cx = p[1]; cy = p[2]

    sx = W / cam_data['W']; sy = H / cam_data['H']
    fx_r = fx * sx; fy_r = fy * sy; cx_r = cx * sx; cy_r = cy * sy

    c2w = colmap_pose_to_c2w(chosen['R'], chosen['tvec'])
    opengl_flip = np.diag([1, -1, -1, 1])
    camera_pose = c2w @ opengl_flip

    mesh_o3d = o3d.io.read_triangle_mesh(str(ply_path))
    mesh_o3d.compute_vertex_normals()
    verts = np.asarray(mesh_o3d.vertices)
    tris = np.asarray(mesh_o3d.triangles)
    colors = np.asarray(mesh_o3d.vertex_colors)

    tm = trimesh.Trimesh(vertices=verts, faces=tris, vertex_colors=(colors * 255).astype(np.uint8))
    mesh_pr = pyrender.Mesh.from_trimesh(tm, smooth=False)

    scene = pyrender.Scene(bg_color=[0.2, 0.2, 0.2, 1.0], ambient_light=[0.8, 0.8, 0.8])
    scene.add(mesh_pr)

    camera = pyrender.IntrinsicsCamera(fx=fx_r, fy=fy_r, cx=cx_r, cy=cy_r, znear=0.01, zfar=1000.0)
    scene.add(camera, pose=camera_pose)

    light = pyrender.DirectionalLight(color=[1.0, 1.0, 1.0], intensity=2.0)
    scene.add(light, pose=camera_pose)

    renderer = pyrender.OffscreenRenderer(W, H)
    color, depth = renderer.render(scene, flags=pyrender.RenderFlags.FLAT)
    renderer.delete()

    Image.fromarray(color).save(str(out_path))
    print(f'Sanity render saved: {out_path}')
    img_arr = np.array(Image.fromarray(color))
    print(f'  Pixel stats: mean={img_arr.mean():.1f}  std={img_arr.std():.1f}')
    return img_arr

render_vertex_colored_ply(
    'colmap_n15/dense/textured/textured.ply',
    'colmap_n15/sparse/0',
    'mag137/AS17-137-20907HR.png',
    'colmap_n15/dense/textured/sanity_check.png'
)
```

Listing 17: sanity_render.py

B.3.6 test_novel_per_magazine.py

```python
from pathlib import Path
import numpy as np
import json
import subprocess
import struct
import collections

base = Path.home() / "ses598-apollo17"
sparse_dir = base / "colmap_n15" / "sparse" / "0"
splat_config = sorted((base / "splats_n15").rglob("config.yml"))[-1]
output_dir = base / "test_novel_renders"
output_dir.mkdir(exist_ok=True)

CameraTuple = collections.namedtuple("CameraTuple", ["id", "model", "width", "height", "params"])
ImageTuple = collections.namedtuple("ImageTuple", ["id", "qvec", "tvec", "camera_id", "name"])

def read_next_bytes(fid, num_bytes, format_char_sequence, endian="<"):
    data = fid.read(num_bytes)
    return struct.unpack(endian + format_char_sequence, data)

def read_cameras_binary(path):
    cameras = {}
    with open(path, "rb") as fid:
        num_cameras = read_next_bytes(fid, 8, "Q")[0]
        for _ in range(num_cameras):
            cam_props = read_next_bytes(fid, 24, "iiQQ")
            num_params = {0: 3, 1: 4, 2: 4, 3: 5, 4: 8, 5: 8, 6: 12, 7: 5, 8: 4, 9: 5, 10: 12}[cam_props[1]]
            params = read_next_bytes(fid, 8 * num_params, "d" * num_params)
            cameras[cam_props[0]] = CameraTuple(cam_props[0], cam_props[1], cam_props[2], cam_props[3], params)
    return cameras

def read_images_binary(path):
    images = {}
    with open(path, "rb") as fid:
        num_images = read_next_bytes(fid, 8, "Q")[0]
        for _ in range(num_images):
            image_props = read_next_bytes(fid, 64, "idddddddi")
            qvec = np.array(image_props[1:5])
            tvec = np.array(image_props[5:8])
            name = ""
            char = read_next_bytes(fid, 1, "c")[0]
            while char != b"\x00":
                name += char.decode("utf-8")
                char = read_next_bytes(fid, 1, "c")[0]
            num_points2d = read_next_bytes(fid, 8, "Q")[0]
            read_next_bytes(fid, 24 * num_points2d, "ddq" * num_points2d)
            images[image_props[0]] = ImageTuple(image_props[0], qvec, tvec, image_props[8], name)
    return images

def qvec_to_rotmat(qvec):
    w, x, y, z = qvec
    return np.array([
        [1 - 2*y*y - 2*z*z, 2*x*y - 2*z*w, 2*x*z + 2*y*w],
        [2*x*y + 2*z*w, 1 - 2*x*x - 2*z*z, 2*y*z - 2*x*w],
        [2*x*z - 2*y*w, 2*y*z + 2*x*w, 1 - 2*x*x - 2*y*y]])

def look_at_to_c2w(eye, target, up=np.array([0, 1, 0])):
    forward = target - eye
    forward = forward / np.linalg.norm(forward)
    right = np.cross(forward, up)
    right = right / np.linalg.norm(right)
    new_up = np.cross(right, forward)
    c2w = np.eye(4)
    c2w[:3, 0] = right
    c2w[:3, 1] = -new_up
    c2w[:3, 2] = forward
    c2w[:3, 3] = eye
    return c2w

cameras = read_cameras_binary(sparse_dir / "cameras.bin")
images = read_images_binary(sparse_dir / "images.bin")

mag137_centers = []
mag137_forwards = []
mag138_centers = []
mag138_forwards = []
for img in images.values():
    R = qvec_to_rotmat(img.qvec)
    center = -R.T @ img.tvec
    forward = R.T @ np.array([0, 0, 1])
    if "mag137" in img.name:
        mag137_centers.append(center)
        mag137_forwards.append(forward)
    else:
        mag138_centers.append(center)
        mag138_forwards.append(forward)

mag137_centers = np.array(mag137_centers)
mag138_centers = np.array(mag138_centers)
mag137_forwards = np.array(mag137_forwards)
```

```
mag138_forwards = np.array(mag138_forwards)

mag137_centroid = mag137_centers.mean(axis=0)
mag138_centroid = mag138_centers.mean(axis=0)
mag137_radius = np.median(np.linalg.norm(mag137_centers - mag137_centroid, axis=1))
mag138_radius = np.median(np.linalg.norm(mag138_centers - mag138_centroid, axis=1))
mag137_mean_fwd = mag137_forwards.mean(axis=0)
mag137_mean_fwd = mag137_mean_fwd / np.linalg.norm(mag137_mean_fwd)
mag138_mean_fwd = mag138_forwards.mean(axis=0)
mag138_mean_fwd = mag138_mean_fwd / np.linalg.norm(mag138_mean_fwd)

mag137_target = mag137_centroid + mag137_mean_fwd * 3.0
mag138_target = mag138_centroid + mag138_mean_fwd * 3.0

print(f"mag137 centroid={mag137_centroid}, radius={mag137_radius:.3f}, target={mag137_target}")
print(f"mag138 centroid={mag138_centroid}, radius={mag138_radius:.3f}, target={mag138_target}")

test_poses = []
test_poses.append(("mag137_test_a", mag137_centroid, mag137_target))
test_poses.append(("mag137_test_b", mag137_centroid + np.array([0, 0, 0.5]), mag137_target))
test_poses.append(("mag138_test_a", mag138_centroid, mag138_target))
test_poses.append(("mag138_test_b", mag138_centroid + np.array([0, 0, 0.8]), mag138_target))

camera_path = {
    "render_height": 1024,
    "render_width": 1024,
    "camera_path": [],
    "fps": 1,
    "seconds": len(test_poses),
    "smoothness_value": 0,
    "is_cycle": False,
    "crop": None
}

for name, eye, target in test_poses:
    c2w = look_at_to_c2w(eye, target)
    camera_path["camera_path"].append({
        "camera_to_world": c2w.flatten().tolist(),
        "fov": 50,
        "aspect": 1.0
    })

cam_path_file = output_dir / "test_path.json"
with open(cam_path_file, "w") as f:
    json.dump(camera_path, f, indent=2)

print(f"\nWrote camera path to {cam_path_file}")
print(f"Splat config: {splat_config}")
print(f"\nNow run:\nns-render camera-path --load-config {splat_config} --camera-path-filename {cam_path_file} --output-path {output_dir / 'test.mp4
    '}")
```

Listing 18: test_novel_per_magazine.py

B.4 Group D: Texturing and Qualitative Comparison

B.4.1 `make_comparison_grid.py`

```
import cv2
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from pathlib import Path

RENDERS_DIR = Path('/home/jatin-satyam/ses598-apollo17/renders_n15/train/rgb')
RGB_DIR = Path('/home/jatin-satyam/ses598-apollo17/data/rgb')
METRICS_DIR = Path('/home/jatin-satyam/ses598-apollo17/metrics_n15')

IMAGE_MAP = {
    'AS17-137-20903HR': 'mag137/AS17-137-20903HR.png',
    'AS17-137-20904HR': 'mag137/AS17-137-20904HR.png',
    'AS17-137-20905HR': 'mag137/AS17-137-20905HR.png',
    'AS17-137-20906HR': 'mag137/AS17-137-20906HR.png',
    'AS17-137-20907HR': 'mag137/AS17-137-20907HR.png',
    'AS17-137-20908HR': 'mag137/AS17-137-20908HR.png',
    'AS17-137-20909HR': 'mag137/AS17-137-20909HR.png',
    'AS17-138-21030HR': 'mag138/AS17-138-21030HR.png',
    'AS17-138-21031HR': 'mag138/AS17-138-21031HR.png',
    'AS17-138-21032HR': 'mag138/AS17-138-21032HR.png',
    'AS17-138-21033HR': 'mag138/AS17-138-21033HR.png',
    'AS17-138-21034HR': 'mag138/AS17-138-21034HR.png',
    'AS17-138-21035HR': 'mag138/AS17-138-21035HR.png',
    'AS17-138-21037HR': 'mag138/AS17-138-21037HR.png',
}

SELECTED = [
```

```
    ('AS17-137-20904HR', 30.18, 0.9312, 'Worst PSNR'),
    ('AS17-137-20907HR', 34.52, 0.9590, 'Median PSNR'),
    ('AS17-138-21031HR', 35.30, 0.9485, 'Random'),
    ('AS17-138-21037HR', 37.36, 0.9493, 'Best PSNR'),
]

def load_pair(key):
    rel = IMAGE_MAP[key]
    mag = rel.split('/')[0]
    render_path = RENDERS_DIR / mag / f'{key}.png'
    orig_path = RGB_DIR / rel
    render = cv2.cvtColor(cv2.imread(str(render_path)), cv2.COLOR_BGR2RGB)
    orig = cv2.cvtColor(cv2.imread(str(orig_path)), cv2.COLOR_BGR2RGB)
    rh, rw = render.shape[:2]
    orig_r = cv2.resize(orig, (rw, rh), interpolation=cv2.INTER_AREA)
    return orig_r, render

fig, axes = plt.subplots(4, 2, figsize=(14, 22))
fig.suptitle('Gaussian Splatting Reconstruction: Original vs Rendered\n(Training Views)', fontsize=14, fontweight='bold')

for row, (key, psnr, ssim, label) in enumerate(SELECTED):
    orig, render = load_pair(key)
    short = key.replace('AS17-', '').replace('HR', '')
    axes[row, 0].imshow(orig)
    axes[row, 0].set_title(f'{short} - Original', fontsize=10)
    axes[row, 0].axis('off')
    axes[row, 1].imshow(render)
    axes[row, 1].set_title(f'{short} - Rendered  [{label}]  PSNR={psnr:.2f} dB  SSIM={ssim:.4f}', fontsize=10)
    axes[row, 1].axis('off')

plt.tight_layout()
out = METRICS_DIR / 'comparison_grid.png'
plt.savefig(str(out), dpi=120, bbox_inches='tight')
plt.close()
print(f'Saved: {out}')
```

Listing 19: make_comparison_grid.py

B.4.2 qualitative_compare_textured.py

```
import numpy as np
import open3d as o3d
import struct
import json
import os
os.environ['PYOPENGL_PLATFORM'] = 'egl'
import pyrender
import trimesh
from pathlib import Path
from PIL import Image
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

SPARSE14 = Path('colmap_n15/sparse/0')
SPARSE24 = Path('colmap_n24/sparse/0')
PLY14 = Path('colmap_n15/dense/textured/textured.ply')
PLY24 = Path('colmap_n24/dense/textured/textured.ply')
METRICS_JSON = Path('comparison/quantitative_metrics.json')
IMAGES14 = Path('data/rgb')
OUT_DIR = Path('comparison')

RENDER_W, RENDER_H = 800, 800

with open(METRICS_JSON) as f:
    T_icp = np.array(json.load(f)['icp_transform'])

def read_cameras_bin(path):
    cameras = {}
    nparams = {0:3,1:4,2:4,3:5,4:8}
    with open(path, 'rb') as f:
        n = struct.unpack('<Q', f.read(8))[0]
        for _ in range(n):
            cid = struct.unpack('<I', f.read(4))[0]
            mid = struct.unpack('<I', f.read(4))[0]
            w = struct.unpack('<Q', f.read(8))[0]
            h = struct.unpack('<Q', f.read(8))[0]
            np_ = nparams[mid]
            params = list(struct.unpack(f'<{np_}d', f.read(8*np_)))
            cameras[cid] = dict(model=mid, W=w, H=h, params=params)
    return cameras

def read_images_bin(path):
    images = {}
    with open(path, 'rb') as f:
        n = struct.unpack('<Q', f.read(8))[0]
        for _ in range(n):
```

```
            iid = struct.unpack('<I', f.read(4))[0]
            qvec = np.array(struct.unpack('<4d', f.read(32)))
            tvec = np.array(struct.unpack('<3d', f.read(24)))
            cid = struct.unpack('<I', f.read(4))[0]
            name = b''
            while True:
                c = f.read(1)
                if c == b'\x00': break
                name += c
            n_pts = struct.unpack('<Q', f.read(8))[0]
            f.read(n_pts * 24)
            qw, qx, qy, qz = qvec
            R = np.array([
                [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
                [2*(qx*qy+qz*qw), 1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
                [2*(qx*qz-qy*qw), 2*(qy*qz+qx*qw), 1-2*(qx**2+qy**2)]
            ])
            images[iid] = dict(name=name.decode(), cam_id=cid, R=R, tvec=tvec)
    return images

def build_c2w_opengl(R, tvec):
    c2w = np.eye(4)
    c2w[:3, :3] = R.T
    c2w[:3, 3] = -R.T @ tvec
    flip = np.diag([1.0, -1.0, -1.0, 1.0])
    return c2w @ flip

def colmap_intrinsics_to_pyrender(cam_data, out_W, out_H):
    p = cam_data['params']
    m = cam_data['model']
    if m == 1:
        fx, fy = p[0], p[1]; cx, cy = p[2], p[3]
    elif m == 2:
        fx = fy = p[0]; cx = p[1]; cy = p[2]
    else:
        fx = fy = p[0]; cx = p[1]; cy = p[2]
    sx = out_W / cam_data['W']; sy = out_H / cam_data['H']
    return fx*sx, fy*sy, cx*sx, cy*sy

def load_vertex_colored_mesh(ply_path, transform=None):
    mesh_o3d = o3d.io.read_triangle_mesh(str(ply_path))
    if transform is not None:
        mesh_o3d.transform(transform)
    mesh_o3d.compute_vertex_normals()
    verts = np.asarray(mesh_o3d.vertices)
    tris = np.asarray(mesh_o3d.triangles)
    colors = (np.asarray(mesh_o3d.vertex_colors) * 255).astype(np.uint8)
    tm = trimesh.Trimesh(vertices=verts, faces=tris, vertex_colors=colors)
    return pyrender.Mesh.from_trimesh(tm, smooth=False)

def render_mesh(mesh_pr, pose, fx, fy, cx, cy, W=RENDER_W, H=RENDER_H):
    scene = pyrender.Scene(bg_color=[0.15, 0.15, 0.15, 1.0], ambient_light=[1.0, 1.0, 1.0])
    scene.add(mesh_pr)
    camera = pyrender.IntrinsicsCamera(fx=fx, fy=fy, cx=cx, cy=cy, znear=0.01, zfar=1000.0)
    scene.add(camera, pose=pose)
    light = pyrender.DirectionalLight(color=[1.0, 1.0, 1.0], intensity=3.0)
    scene.add(light, pose=pose)
    renderer = pyrender.OffscreenRenderer(W, H)
    color, _ = renderer.render(scene, flags=pyrender.RenderFlags.FLAT)
    renderer.delete()
    return color

print('Loading cameras from N=14 sparse model...')
cams14 = read_cameras_bin(SPARSE14 / 'cameras.bin')
imgs14 = read_images_bin(SPARSE14 / 'images.bin')

chosen_names = [
    'mag137/AS17-137-20904HR.png',
    'mag137/AS17-137-20907HR.png',
    'mag138/AS17-138-21030HR.png',
    'mag138/AS17-138-21033HR.png',
]
chosen_imgs = [next(v for v in imgs14.values() if v['name'] == nm) for nm in chosen_names]

print('Loading N=14 textured mesh...')
mesh14_pr = load_vertex_colored_mesh(PLY14)

print('Loading N=24 textured mesh (ICP-aligned to N=14 frame)...')
mesh24_pr = load_vertex_colored_mesh(PLY24, transform=T_icp)

sidebyside_paths = []
for vi, (img_info, img_name) in enumerate(zip(chosen_imgs, chosen_names), start=1):
    print(f'Rendering view {vi}: {img_name}')

    cam_data = cams14[img_info['cam_id']]
    pose = build_c2w_opengl(img_info['R'], img_info['tvec'])
    fx, fy, cx, cy = colmap_intrinsics_to_pyrender(cam_data, RENDER_W, RENDER_H)

    color14 = render_mesh(mesh14_pr, pose, fx, fy, cx, cy)
    color24 = render_mesh(mesh24_pr, pose, fx, fy, cx, cy)

    r14_path = OUT_DIR / f'textured_render_n14_view{vi}.png'
```

```
    r24_path = OUT_DIR / f'textured_render_n24_view{vi}.png'
    Image.fromarray(color14).save(str(r14_path))
    Image.fromarray(color24).save(str(r24_path))
    print(f'  N14: mean={color14.mean():.1f} std={color14.std():.1f}')
    print(f'  N24: mean={color24.mean():.1f} std={color24.std():.1f}')

    photo_path = IMAGES14 / img_name
    if photo_path.exists():
        photo = np.array(Image.open(photo_path).convert('RGB').resize((RENDER_W, RENDER_H), Image.LANCZOS))
        fig, axes = plt.subplots(1, 3, figsize=(24, 8))
        axes[0].imshow(color14)
        axes[0].set_title(f'N=14 Textured Mesh', fontsize=13, color='steelblue')
        axes[0].axis('off')
        axes[1].imshow(photo)
        axes[1].set_title(f'Source Photo: {Path(img_name).stem}', fontsize=13, color='goldenrod')
        axes[1].axis('off')
        axes[2].imshow(color24)
        axes[2].set_title(f'N=24 Textured Mesh (ICP-aligned)', fontsize=13, color='mediumvioletred')
        axes[2].axis('off')
    else:
        fig, axes = plt.subplots(1, 2, figsize=(16, 8))
        axes[0].imshow(color14)
        axes[0].set_title('N=14 Textured Mesh', fontsize=13, color='steelblue')
        axes[0].axis('off')
        axes[1].imshow(color24)
        axes[1].set_title('N=24 Textured Mesh (ICP-aligned)', fontsize=13, color='mediumvioletred')
        axes[1].axis('off')

    fig.suptitle(f'Textured Mesh Comparison View {vi}: {Path(img_name).stem}', fontsize=14, fontweight='bold')
    fig.tight_layout()
    sbs_path = OUT_DIR / f'textured_sidebyside_view{vi}.png'
    plt.savefig(str(sbs_path), dpi=100, bbox_inches='tight')
    plt.close()
    sidebyside_paths.append(sbs_path)
    print(f'  Side-by-side: {sbs_path}')

print('\nGenerating 2x2 qualitative grid...')
fig, axes = plt.subplots(2, 2, figsize=(28, 18))
for i, sbs_path in enumerate(sidebyside_paths):
    row, col = i // 2, i % 2
    img = np.array(Image.open(sbs_path))
    axes[row, col].imshow(img)
    axes[row, col].set_title(f'View {i+1}: {Path(chosen_names[i]).stem}', fontsize=12)
    axes[row, col].axis('off')
fig.suptitle('Textured Mesh Comparison: N=14 vs N=24 Photogrammetry (4 Viewpoints)', fontsize=16, fontweight='bold')
plt.tight_layout()
grid_path = OUT_DIR / 'textured_qualitative_grid.png'
plt.savefig(str(grid_path), dpi=80, bbox_inches='tight')
plt.close()
print(f'Saved: {grid_path}')
print('\n===== STAGE: TEXTURED QUALITATIVE COMPARISON COMPLETE =====')
```

Listing 20: qualitative_compare_textured.py

B.4.3 `texture_mesh_uv.py`

```
import numpy as np
import open3d as o3d
import cv2
import xatlas
import struct
import json
import time
from pathlib import Path
from PIL import Image

ATLAS_SIZE = 4096
MAX_FACES_XATLAS = 2500000

def read_cameras_bin(path):
    cameras = {}
    nparams = {0: 3, 1: 4, 2: 4, 3: 5, 4: 8}
    with open(path, 'rb') as f:
        n = struct.unpack('<Q', f.read(8))[0]
        for _ in range(n):
            cid = struct.unpack('<I', f.read(4))[0]
            mid = struct.unpack('<I', f.read(4))[0]
            w = struct.unpack('<Q', f.read(8))[0]
            h = struct.unpack('<Q', f.read(8))[0]
            np_ = nparams[mid]
            params = list(struct.unpack(f'<{np_}d', f.read(8 * np_)))
            cameras[cid] = dict(model=mid, W=w, H=h, params=params)
    return cameras

def read_images_bin(path):
    images = {}
    with open(path, 'rb') as f:
```

```
        n = struct.unpack('<Q', f.read(8))[0]
        for _ in range(n):
            iid = struct.unpack('<I', f.read(4))[0]
            qvec = np.array(struct.unpack('<4d', f.read(32)))
            tvec = np.array(struct.unpack('<3d', f.read(24)))
            cid = struct.unpack('<I', f.read(4))[0]
            name = b''
            while True:
                c = f.read(1)
                if c == b'\x00': break
                name += c
            n_pts = struct.unpack('<Q', f.read(8))[0]
            f.read(n_pts * 24)
            qw, qx, qy, qz = qvec
            R = np.array([
                [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
                [2*(qx*qy+qz*qw), 1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
                [2*(qx*qz-qy*qw), 2*(qy*qz+qx*qw), 1-2*(qx**2+qy**2)]
            ])
            images[iid] = dict(name=name.decode(), cam_id=cid, R=R, tvec=tvec, pos=-R.T @ tvec)
    return images

def build_cameras(sparse_dir, images_dir):
    sparse_dir = Path(sparse_dir)
    images_dir = Path(images_dir)
    raw_cams = read_cameras_bin(sparse_dir / 'cameras.bin')
    raw_imgs = read_images_bin(sparse_dir / 'images.bin')
    cameras = []
    for iid, img in raw_imgs.items():
        cam_data = raw_cams[img['cam_id']]
        p = cam_data['params']
        model = cam_data['model']
        if model == 1:
            fx, fy, cx, cy = p[0], p[1], p[2], p[3]
            k = 0.0
            f = fx
        elif model == 2:
            f, cx, cy, k = p[0], p[1], p[2], p[3]
        else:
            f = p[0]; cx = p[1]; cy = p[2]; k = 0.0
        img_path = images_dir / img['name']
        if not img_path.exists():
            print(f'  Warning: image not found: {img_path}')
            continue
        src_img = np.array(Image.open(img_path).convert('RGB'), dtype=np.uint8)
        cameras.append(dict(
            name=img['name'],
            R=img['R'],
            t=img['tvec'],
            pos=img['pos'],
            f=f, cx=cx, cy=cy, k=k,
            W=cam_data['W'], H=cam_data['H'],
            img=src_img,
            model=model
        ))
    print(f'  Loaded {len(cameras)} cameras with images')
    return cameras

def project_pts(pts3d, cam):
    Xc = (cam['R'] @ pts3d.T + cam['t'][:, None]).T
    valid = Xc[:, 2] > 0.1
    eps = 1e-8
    xn = Xc[:, 0] / Xc[:, 2].clip(eps)
    yn = Xc[:, 1] / Xc[:, 2].clip(eps)
    if cam['k'] != 0.0:
        r2 = xn**2 + yn**2
        scale = 1.0 + cam['k'] * r2
        xd = xn * scale
        yd = yn * scale
    else:
        xd, yd = xn, yn
    u = cam['f'] * xd + cam['cx']
    v = cam['f'] * yd + cam['cy']
    in_img = valid & (u >= 0) & (u < cam['W']) & (v >= 0) & (v < cam['H'])
    return u, v, in_img

def bilinear_sample(img, u_px, v_px):
    H, W = img.shape[:2]
    u0 = np.floor(u_px).astype(np.int32).clip(0, W - 2)
    v0 = np.floor(v_px).astype(np.int32).clip(0, H - 2)
    fu = (u_px - u0).astype(np.float32)
    fv = (v_px - v0).astype(np.float32)
    c00 = img[v0, u0].astype(np.float32)
    c01 = img[v0, u0 + 1].astype(np.float32)
    c10 = img[v0 + 1, u0].astype(np.float32)
    c11 = img[v0 + 1, u0 + 1].astype(np.float32)
    fu = fu[:, None]; fv = fv[:, None]
    return (1-fu)*(1-fv)*c00 + fu*(1-fv)*c01 + (1-fu)*fv*c10 + fu*fv*c11

def assign_cameras(centroids, tri_norms, cameras, scene):
    N = len(centroids)
    N_cams = len(cameras)
```

```
    cam_positions = np.array([c['pos'] for c in cameras], dtype=np.float64)

    vecs = cam_positions[None] - centroids[:, None]
    dists = np.linalg.norm(vecs, axis=2).clip(1e-10)
    vecs_norm = vecs / dists[:, :, None]
    cos_a = (vecs_norm * tri_norms[:, None]).sum(axis=2)

    in_fov = np.zeros((N, N_cams), dtype=bool)
    for ci, cam in enumerate(cameras):
        _, _, valid = project_pts(centroids, cam)
        in_fov[:, ci] = valid

    scores = np.where(in_fov & (cos_a > 0), cos_a / dists**2, -1.0)
    sorted_cams = np.argsort(-scores, axis=1)
    cam_per_tri = np.full(N, -1, dtype=np.int32)

    for rank in range(min(5, N_cams)):
        need = cam_per_tri == -1
        if not need.any():
            break
        cand = sorted_cams[need, rank]
        valid_cand = scores[need, cand] > 0
        if not valid_cand.any():
            continue
        check_idx = np.where(need)[0][valid_cand]
        check_cam = cand[valid_cand]

        origins = centroids[check_idx].astype(np.float32)
        targets = cam_positions[check_cam].astype(np.float32)
        raw_dirs = targets - origins
        raw_dists = np.linalg.norm(raw_dirs, axis=1, keepdims=True).clip(1e-10)
        dirs_n = (raw_dirs / raw_dists).astype(np.float32)
        offset_o = origins + dirs_n * 0.005

        rays = np.concatenate([offset_o, dirs_n], axis=1)
        rays_t = o3d.core.Tensor(rays, dtype=o3d.core.Dtype.Float32)
        hit = scene.cast_rays(rays_t)
        hit_d = hit['t_hit'].numpy()
        unoccluded = hit_d >= raw_dists.flatten() * 0.98

        cam_per_tri[check_idx[unoccluded]] = check_cam[unoccluded]

    still_none = cam_per_tri == -1
    if still_none.any():
        best = np.argmax(scores[still_none], axis=1)
        cam_per_tri[still_none] = best

    return cam_per_tri

def save_obj_mtl(out_dir, vmapping, verts, xatlas_tris, uvs, atlas_name):
    out_dir = Path(out_dir)
    obj_path = out_dir / 'textured.obj'
    mtl_path = out_dir / 'textured.mtl'

    with open(mtl_path, 'w') as f:
        f.write('newmtl atlas_mat\n')
        f.write(f'map_Kd {atlas_name}\n')
        f.write('Kd 1.0 1.0 1.0\n')
        f.write('Ka 0.0 0.0 0.0\n')
        f.write('Ks 0.0 0.0 0.0\n')

    N_out = len(vmapping)
    out_verts = verts[vmapping]

    chunk = 500000
    with open(obj_path, 'w') as f:
        f.write(f'mtllib textured.mtl\n')
        for i in range(0, N_out, chunk):
            block = out_verts[i:i+chunk]
            lines = '\n'.join(f'v {v[0]:.6f} {v[1]:.6f} {v[2]:.6f}' for v in block)
            f.write(lines + '\n')
        for i in range(0, N_out, chunk):
            block = uvs[i:i+chunk]
            lines = '\n'.join(f'vt {u:.6f} {v:.6f}' for u, v in block)
            f.write(lines + '\n')
        f.write('usemtl atlas_mat\n')
        for i in range(0, len(xatlas_tris), chunk):
            block = xatlas_tris[i:i+chunk]
            lines = '\n'.join(
                f'f {v0+1}/{v0+1} {v1+1}/{v1+1} {v2+1}/{v2+1}'
                for v0, v1, v2 in block
            )
            f.write(lines + '\n')

    print(f'  Saved OBJ: {obj_path}  ({obj_path.stat().st_size/1e6:.1f} MB)')
    print(f'  Saved MTL: {mtl_path}')

def texture_mesh(label, mesh_path, sparse_dir, images_dir, out_dir):
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    print(f'\n{"="*60}')
    print(f'Texturing {label}: {mesh_path}')
```

```
    mesh = o3d.io.read_triangle_mesh(str(mesh_path))
    mesh.compute_vertex_normals()
    verts_in = np.asarray(mesh.vertices, dtype=np.float64)
    tris_in = np.asarray(mesh.triangles, dtype=np.int32)
    orig_n_verts = len(verts_in)
    orig_n_faces = len(tris_in)
    print(f'  Input mesh: {orig_n_verts:,} verts, {orig_n_faces:,} faces')

    decimate_factor = 1.0
    if len(tris_in) > MAX_FACES_XATLAS:
        decimate_factor = MAX_FACES_XATLAS / len(tris_in)
        print(f'  Decimating to ~{MAX_FACES_XATLAS:,} faces (factor={decimate_factor:.4f})')
        mesh = mesh.simplify_quadric_decimation(MAX_FACES_XATLAS)
        mesh.compute_vertex_normals()
        verts_in = np.asarray(mesh.vertices, dtype=np.float64)
        tris_in = np.asarray(mesh.triangles, dtype=np.int32)
        print(f'  After decimation: {len(verts_in):,} verts, {len(tris_in):,} faces')

    print('  Loading cameras...')
    cameras = build_cameras(sparse_dir, images_dir)

    print('  Building raycasting scene...')
    scene = o3d.t.geometry.RaycastingScene()
    mesh_t = o3d.t.geometry.TriangleMesh.from_legacy(mesh)
    scene.add_triangles(mesh_t)

    v0c = verts_in[tris_in[:, 0]]
    v1c = verts_in[tris_in[:, 1]]
    v2c = verts_in[tris_in[:, 2]]
    centroids = (v0c + v1c + v2c) / 3.0
    cross = np.cross(v1c - v0c, v2c - v0c)
    nlen = np.linalg.norm(cross, axis=1, keepdims=True).clip(1e-10)
    tri_norms = cross / nlen

    print('  Assigning cameras to triangles...')
    t0 = time.time()
    cam_per_tri = assign_cameras(centroids, tri_norms, cameras, scene)
    print(f'  Camera assignment: {time.time()-t0:.1f}s  unassigned={( cam_per_tri==-1).sum()}')

    print('  xatlas UV parametrization...')
    t0 = time.time()
    vmapping, xatlas_tris, uvs = xatlas.parametrize(
        verts_in.astype(np.float32), tris_in.astype(np.uint32)
    )
    print(f'  xatlas: {len(vmapping):,} output verts, {time.time()-t0:.1f}s')
    print(f'  UV range: [{uvs.min():.4f}, {uvs.max():.4f}]')

    H = W = ATLAS_SIZE
    uvs_px = uvs * np.array([W, H], dtype=np.float32)
    tri_map = np.full((H, W), -1, dtype=np.int32)
    bary_u = np.zeros((H, W), dtype=np.float32)
    bary_v = np.zeros((H, W), dtype=np.float32)

    print(f'  Rasterizing {len(xatlas_tris):,} triangles to {W}x{H} atlas...')
    t0 = time.time()
    N_tris = len(xatlas_tris)
    for ti in range(N_tris):
        if ti % 300000 == 0 and ti > 0:
            elapsed = time.time() - t0
            eta = elapsed / ti * (N_tris - ti)
            print(f'    {ti:,}/{N_tris:,}  elapsed={elapsed:.0f}s  eta={eta:.0f}s')

        i0, i1, i2 = xatlas_tris[ti]
        p0 = uvs_px[i0]; p1 = uvs_px[i1]; p2 = uvs_px[i2]

        x_lo = max(0, int(np.floor(min(p0[0], p1[0], p2[0]))))
        x_hi = min(W - 1, int(np.ceil(max(p0[0], p1[0], p2[0]))))
        y_lo = max(0, int(np.floor(min(p0[1], p1[1], p2[1]))))
        y_hi = min(H - 1, int(np.ceil(max(p0[1], p1[1], p2[1]))))
        if x_lo > x_hi or y_lo > y_hi:
            continue

        xs = np.arange(x_lo, x_hi + 1, dtype=np.float32) + 0.5
        ys = np.arange(y_lo, y_hi + 1, dtype=np.float32) + 0.5
        xx, yy = np.meshgrid(xs, ys)
        pts = np.stack([xx.ravel(), yy.ravel()], axis=1)

        e01 = p1 - p0; e02 = p2 - p0
        d00 = float(np.dot(e01, e01)); d01 = float(np.dot(e01, e02)); d11 = float(np.dot(e02, e02))
        det = d00 * d11 - d01 * d01
        if abs(det) < 1e-10:
            continue
        inv = 1.0 / det
        vp = pts - p0
        dp0 = vp @ e01; dp1 = vp @ e02
        u_b = (d11 * dp0 - d01 * dp1) * inv
        v_b = (d00 * dp1 - d01 * dp0) * inv
        inside = (u_b >= 0) & (v_b >= 0) & (u_b + v_b <= 1.0)
        if not inside.any():
            continue
```

```
        px = xx.ravel()[inside].astype(np.int32).clip(0, W - 1)
        py = yy.ravel()[inside].astype(np.int32).clip(0, H - 1)
        tri_map[py, px] = ti
        bary_u[py, px] = u_b[inside]
        bary_v[py, px] = v_b[inside]

    print(f'  Rasterization: {time.time()-t0:.1f}s')

    valid_y, valid_x = np.where(tri_map >= 0)
    valid_tris = tri_map[valid_y, valid_x]
    u_b_vals = bary_u[valid_y, valid_x]
    v_b_vals = bary_v[valid_y, valid_x]
    w_b_vals = 1.0 - u_b_vals - v_b_vals

    xv0 = xatlas_tris[valid_tris, 0]; xv1 = xatlas_tris[valid_tris, 1]; xv2 = xatlas_tris[valid_tris, 2]
    pos3d = (w_b_vals[:, None] * verts_in[vmapping[xv0]] +
             u_b_vals[:, None] * verts_in[vmapping[xv1]] +
             v_b_vals[:, None] * verts_in[vmapping[xv2]])

    pixel_cams = cam_per_tri[valid_tris]
    atlas = np.full((H, W, 3), 128, dtype=np.uint8)

    print(f'  Baking {len(valid_y):,} valid atlas pixels across {len(cameras)} cameras...')
    t0 = time.time()
    covered = 0
    for ci, cam in enumerate(cameras):
        mask = pixel_cams == ci
        if not mask.any():
            continue
        u_px, v_px, in_img = project_pts(pos3d[mask], cam)
        sampled = bilinear_sample(cam['img'], u_px[in_img], v_px[in_img])
        ay = valid_y[mask][in_img]
        ax = valid_x[mask][in_img]
        atlas[ay, ax] = sampled.clip(0, 255).astype(np.uint8)
        covered += in_img.sum()

    coverage = covered / max(1, len(valid_y))
    print(f'  Baking: {time.time()-t0:.1f}s  coverage={coverage:.1%}')

    print('  Inpainting atlas holes...')
    fill_mask = (tri_map < 0).astype(np.uint8) * 255
    atlas = cv2.inpaint(atlas, fill_mask, 3, cv2.INPAINT_TELEA)

    atlas_path = out_dir / 'texture_atlas.png'
    Image.fromarray(atlas).save(str(atlas_path))
    print(f'  Saved atlas: {atlas_path}  ({atlas_path.stat().st_size/1e6:.1f} MB)')

    save_obj_mtl(out_dir, vmapping, verts_in, xatlas_tris, uvs, 'texture_atlas.png')

    summary = dict(
        label=label,
        tier='A',
        format='UV-textured OBJ + 4096x4096 atlas PNG',
        atlas_resolution=ATLAS_SIZE,
        input_vertex_count=orig_n_verts,
        input_face_count=orig_n_faces,
        decimation_factor=round(decimate_factor, 4),
        output_vertex_count=int(len(vmapping)),
        output_face_count=int(len(xatlas_tris)),
        coverage_rate=round(float(coverage), 4),
        out_dir=str(out_dir),
        files=['textured.obj', 'textured.mtl', 'texture_atlas.png']
    )
    summary_path = out_dir / 'summary.json'
    with open(summary_path, 'w') as f:
        json.dump(summary, f, indent=2)
    print(f'  Summary: {summary_path}')
    print(f'  Done: {label}')
    return summary

t_start = time.time()

summary_n14 = texture_mesh(
    'N=14',
    'colmap_n15/dense/meshed-poisson.ply',
    'colmap_n15/sparse/0',
    'data/rgb',
    'colmap_n15/dense/textured'
)

summary_n24 = texture_mesh(
    'N=24',
    'colmap_n24/dense/meshed-poisson.ply',
    'colmap_n24/sparse/0',
    'data/rgb_n24',
    'colmap_n24/dense/textured'
)

combined = dict(
    tier='A',
    output_format='UV-textured OBJ + atlas PNG',
    atlas_resolution=ATLAS_SIZE,
```

```
    meshes=[summary_n14, summary_n24]
)
with open('comparison/textured_summary.json', 'w') as f:
    json.dump(combined, f, indent=2)
print(json.dumps(combined, indent=2))
print(f'\nTotal time: {(time.time()-t_start)/60:.1f} minutes')
print('\n===== STAGE: UV TEXTURING COMPLETE =====')
```

Listing 21: texture_mesh_uv.py

B.4.4 texture_mesh_vertex.py

```
import numpy as np
import open3d as o3d
import struct
import json
import time
from pathlib import Path
from PIL import Image

def read_cameras_bin(path):
    cameras = {}
    nparams = {0: 3, 1: 4, 2: 4, 3: 5, 4: 8}
    with open(path, 'rb') as f:
        n = struct.unpack('<Q', f.read(8))[0]
        for _ in range(n):
            cid = struct.unpack('<I', f.read(4))[0]
            mid = struct.unpack('<I', f.read(4))[0]
            w = struct.unpack('<Q', f.read(8))[0]
            h = struct.unpack('<Q', f.read(8))[0]
            np_ = nparams[mid]
            params = list(struct.unpack(f'<{np_}d', f.read(8 * np_)))
            cameras[cid] = dict(model=mid, W=w, H=h, params=params)
    return cameras

def read_images_bin(path):
    images = {}
    with open(path, 'rb') as f:
        n = struct.unpack('<Q', f.read(8))[0]
        for _ in range(n):
            iid = struct.unpack('<I', f.read(4))[0]
            qvec = np.array(struct.unpack('<4d', f.read(32)))
            tvec = np.array(struct.unpack('<3d', f.read(24)))
            cid = struct.unpack('<I', f.read(4))[0]
            name = b''
            while True:
                c = f.read(1)
                if c == b'\x00': break
                name += c
            n_pts = struct.unpack('<Q', f.read(8))[0]
            f.read(n_pts * 24)
            qw, qx, qy, qz = qvec
            R = np.array([
                [1-2*(qy**2+qz**2), 2*(qx*qy-qz*qw), 2*(qx*qz+qy*qw)],
                [2*(qx*qy+qz*qw), 1-2*(qx**2+qz**2), 2*(qy*qz-qx*qw)],
                [2*(qx*qz-qy*qw), 2*(qy*qz+qx*qw), 1-2*(qx**2+qy**2)]
            ])
            images[iid] = dict(name=name.decode(), cam_id=cid, R=R, tvec=tvec, pos=-R.T @ tvec)
    return images

def build_cameras(sparse_dir, images_dir):
    sparse_dir = Path(sparse_dir)
    images_dir = Path(images_dir)
    raw_cams = read_cameras_bin(sparse_dir / 'cameras.bin')
    raw_imgs = read_images_bin(sparse_dir / 'images.bin')
    cameras = []
    for iid, img in raw_imgs.items():
        cam = raw_cams[img['cam_id']]
        p = cam['params']
        m = cam['model']
        if m == 1:
            f = p[0]; cx = p[2]; cy = p[3]; k = 0.0
        elif m == 2:
            f = p[0]; cx = p[1]; cy = p[2]; k = p[3]
        else:
            f = p[0]; cx = p[1]; cy = p[2]; k = 0.0
        img_path = images_dir / img['name']
        if not img_path.exists():
            continue
        src = np.array(Image.open(img_path).convert('RGB'), dtype=np.uint8)
        cameras.append(dict(
            name=img['name'], R=img['R'], t=img['tvec'], pos=img['pos'],
            f=f, cx=cx, cy=cy, k=k, W=cam['W'], H=cam['H'], img=src, model=m
        ))
    return cameras

def project_pts(pts3d, cam):
```

```
    Xc = (cam['R'] @ pts3d.T + cam['t'][:, None]).T
    valid = Xc[:, 2] > 0.1
    eps = 1e-8
    xn = Xc[:, 0] / Xc[:, 2].clip(eps)
    yn = Xc[:, 1] / Xc[:, 2].clip(eps)
    if cam['k'] != 0.0:
        r2 = xn**2 + yn**2
        sc = 1.0 + cam['k'] * r2
        xd = xn * sc; yd = yn * sc
    else:
        xd = xn; yd = yn
    u = cam['f'] * xd + cam['cx']
    v = cam['f'] * yd + cam['cy']
    in_img = valid & (u >= 0) & (u < cam['W']) & (v >= 0) & (v < cam['H'])
    return u, v, in_img, Xc[:, 2]

def bilinear_sample(img, u_px, v_px):
    H, W = img.shape[:2]
    u0 = np.floor(u_px).astype(np.int32).clip(0, W - 2)
    v0 = np.floor(v_px).astype(np.int32).clip(0, H - 2)
    fu = (u_px - u0).astype(np.float32)[:, None]
    fv = (v_px - v0).astype(np.float32)[:, None]
    c00 = img[v0, u0].astype(np.float32)
    c01 = img[v0, u0 + 1].astype(np.float32)
    c10 = img[v0 + 1, u0].astype(np.float32)
    c11 = img[v0 + 1, u0 + 1].astype(np.float32)
    return (1-fu)*(1-fv)*c00 + fu*(1-fv)*c01 + (1-fu)*fv*c10 + fu*fv*c11

def color_vertices(mesh, cameras, scene, batch=200000):
    verts = np.asarray(mesh.vertices, dtype=np.float64)
    normals = np.asarray(mesh.vertex_normals, dtype=np.float64)
    N = len(verts)

    accum_color = np.zeros((N, 3), dtype=np.float64)
    accum_weight = np.zeros(N, dtype=np.float64)

    cam_positions = np.array([c['pos'] for c in cameras], dtype=np.float64)

    for start in range(0, N, batch):
        end = min(start + batch, N)
        vb = verts[start:end]
        nb = normals[start:end]
        M = end - start

        if start % 500000 == 0:
            print(f'  Vertex {start:,}/{N:,}')

        for ci, cam in enumerate(cameras):
            u_px, v_px, in_img, depths = project_pts(vb, cam)

            if not in_img.any():
                continue

            cam_dir = cam['pos'] - vb
            dists = np.linalg.norm(cam_dir, axis=1).clip(1e-10)
            cam_dir_n = cam_dir / dists[:, None]
            cos_a = (cam_dir_n * nb).sum(axis=1)

            good = in_img & (cos_a > 0)
            if not good.any():
                continue

            check_idx = np.where(good)[0]
            origins = vb[check_idx].astype(np.float32)
            dirs = (cam['pos'] - origins).astype(np.float32)
            d_to_cam = np.linalg.norm(dirs, axis=1, keepdims=True).clip(1e-10)
            dirs_n = (dirs / d_to_cam).astype(np.float32)
            offset_o = origins + dirs_n * 0.005

            rays = np.concatenate([offset_o, dirs_n], axis=1)
            rays_t = o3d.core.Tensor(rays, dtype=o3d.core.Dtype.Float32)
            hit = scene.cast_rays(rays_t)
            hit_d = hit['t_hit'].numpy()
            unoccluded = hit_d >= d_to_cam.flatten() * 0.98

            if not unoccluded.any():
                continue

            final_idx = check_idx[unoccluded]
            w = cos_a[final_idx] / dists[final_idx]**2
            sampled = bilinear_sample(cam['img'], u_px[final_idx], v_px[final_idx])

            global_idx = np.arange(start, end)[final_idx]
            np.add.at(accum_color, global_idx, sampled * w[:, None])
            np.add.at(accum_weight, global_idx, w)

    has_color = accum_weight > 0
    vertex_colors = np.zeros((N, 3), dtype=np.float32)
    vertex_colors[has_color] = (accum_color[has_color] / accum_weight[has_color, None]).clip(0, 255) / 255.0
    vertex_colors[~has_color] = 0.5

    coverage = has_color.mean()
```

```
    return vertex_colors, coverage

def texture_mesh_vertex(label, mesh_path, sparse_dir, images_dir, out_dir):
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    print(f'\n{"="*60}')
    print(f'Vertex-color texturing {label}: {mesh_path}')

    t0 = time.time()
    mesh = o3d.io.read_triangle_mesh(str(mesh_path))
    mesh.compute_vertex_normals()
    verts = np.asarray(mesh.vertices)
    N_verts = len(verts)
    N_faces = len(np.asarray(mesh.triangles))
    print(f'  Mesh: {N_verts:,} verts, {N_faces:,} faces')

    cameras = build_cameras(sparse_dir, images_dir)
    print(f'  {len(cameras)} cameras loaded')

    print('  Building raycasting scene...')
    scene = o3d.t.geometry.RaycastingScene()
    mesh_t = o3d.t.geometry.TriangleMesh.from_legacy(mesh)
    scene.add_triangles(mesh_t)

    print('  Computing vertex colors...')
    vertex_colors, coverage = color_vertices(mesh, cameras, scene)
    print(f'  Coverage: {coverage:.1%}  time: {time.time()-t0:.1f}s')

    mesh.vertex_colors = o3d.utility.Vector3dVector(vertex_colors)
    out_ply = out_dir / 'textured.ply'
    o3d.io.write_triangle_mesh(str(out_ply), mesh)
    print(f'  Saved: {out_ply}  ({out_ply.stat().st_size/1e6:.1f} MB)')

    summary = dict(
        label=label,
        tier='B',
        format='per-vertex-colored PLY',
        output_vertex_count=N_verts,
        output_face_count=N_faces,
        coverage_rate=round(float(coverage), 4),
        out_dir=str(out_dir),
        files=['textured.ply']
    )
    with open(out_dir / 'summary.json', 'w') as f:
        json.dump(summary, f, indent=2)
    return summary

t_all = time.time()

summary_n14 = texture_mesh_vertex(
    'N=14',
    'colmap_n15/dense/meshed-poisson.ply',
    'colmap_n15/sparse/0',
    'data/rgb',
    'colmap_n15/dense/textured'
)

summary_n24 = texture_mesh_vertex(
    'N=24',
    'colmap_n24/dense/meshed-poisson.ply',
    'colmap_n24/sparse/0',
    'data/rgb_n24',
    'colmap_n24/dense/textured'
)

combined = dict(
    tier='B',
    output_format='per-vertex-colored PLY',
    note='Per-vertex RGB coloring. Recognized as textured mesh in photogrammetry tools (Metashape, CloudCompare). Chosen because xatlas UV
      parametrization exceeded 45-minute per-mesh limit on 2.1M-face input.',
    meshes=[summary_n14, summary_n24]
)

with open('comparison/textured_summary.json', 'w') as f:
    json.dump(combined, f, indent=2)
print(json.dumps(combined, indent=2))
print(f'\nTotal time: {(time.time()-t_all)/60:.1f} minutes')
print('\n===== STAGE: VERTEX TEXTURING COMPLETE =====')
```

Listing 22: texture_mesh_vertex.py